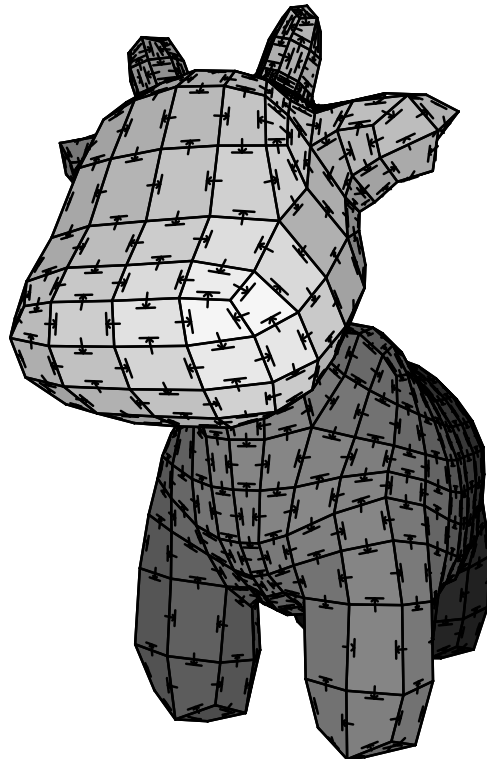
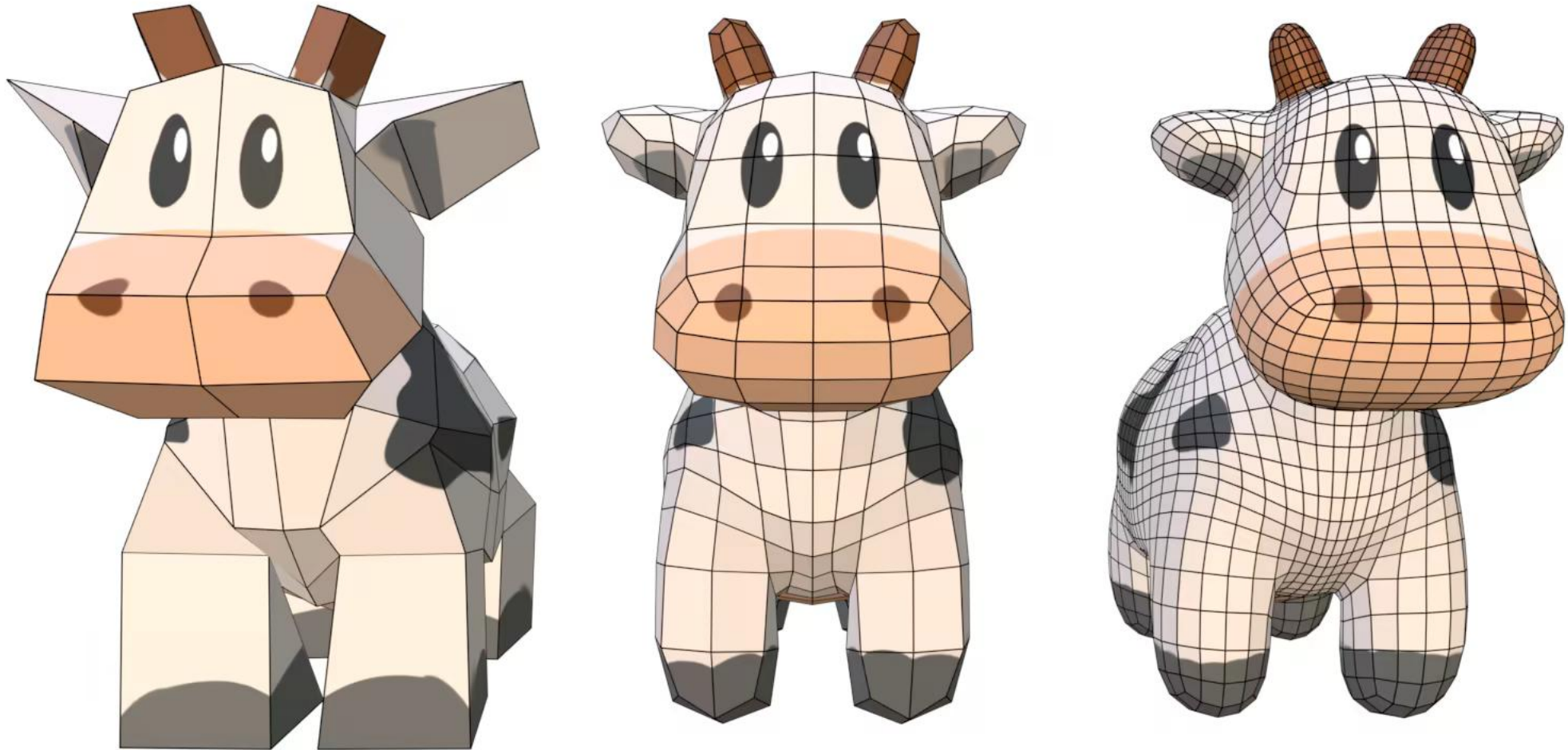


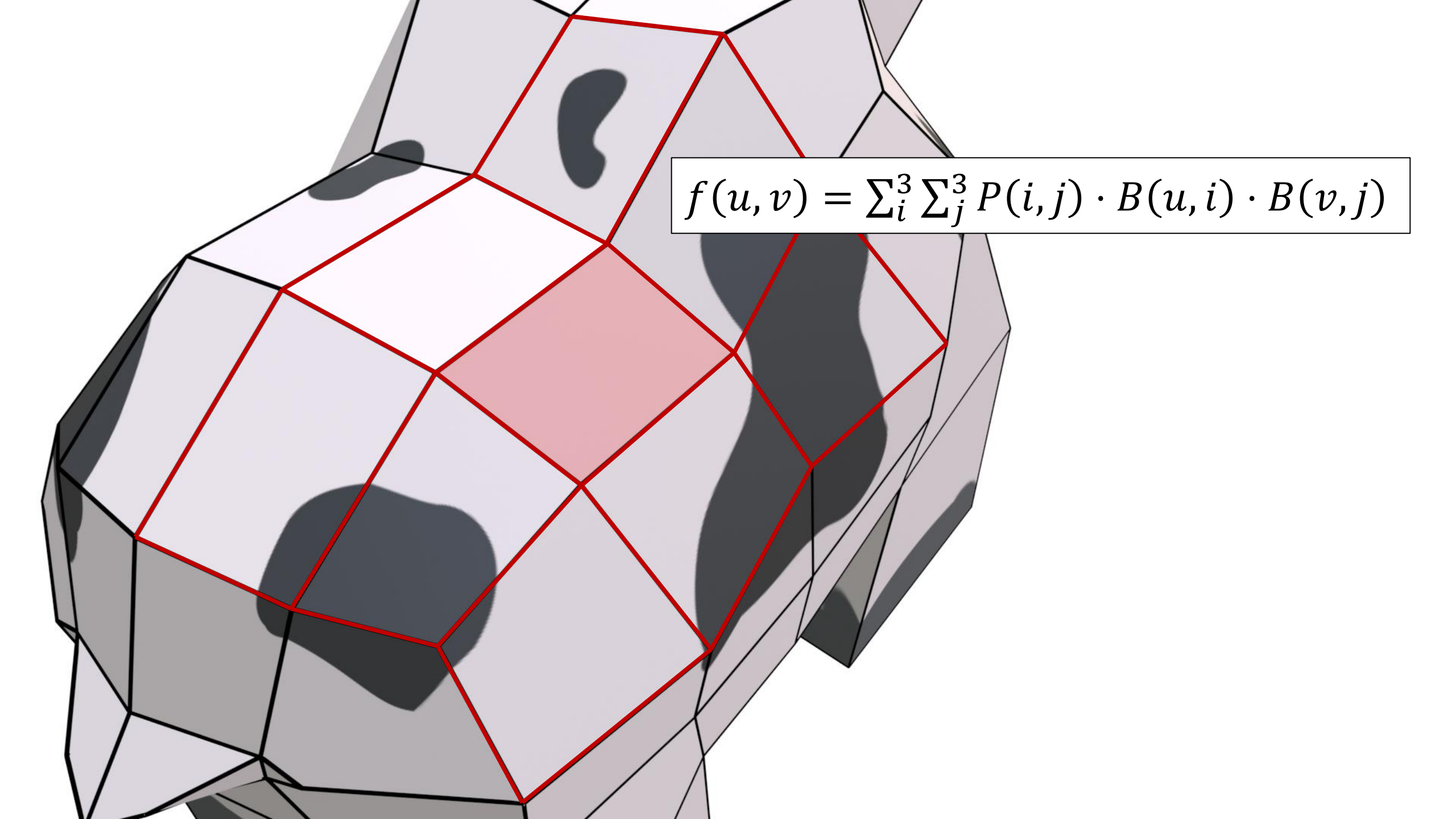
Edge-Friend: Fast and Deterministic Catmull-Clark Subdivision Surfaces

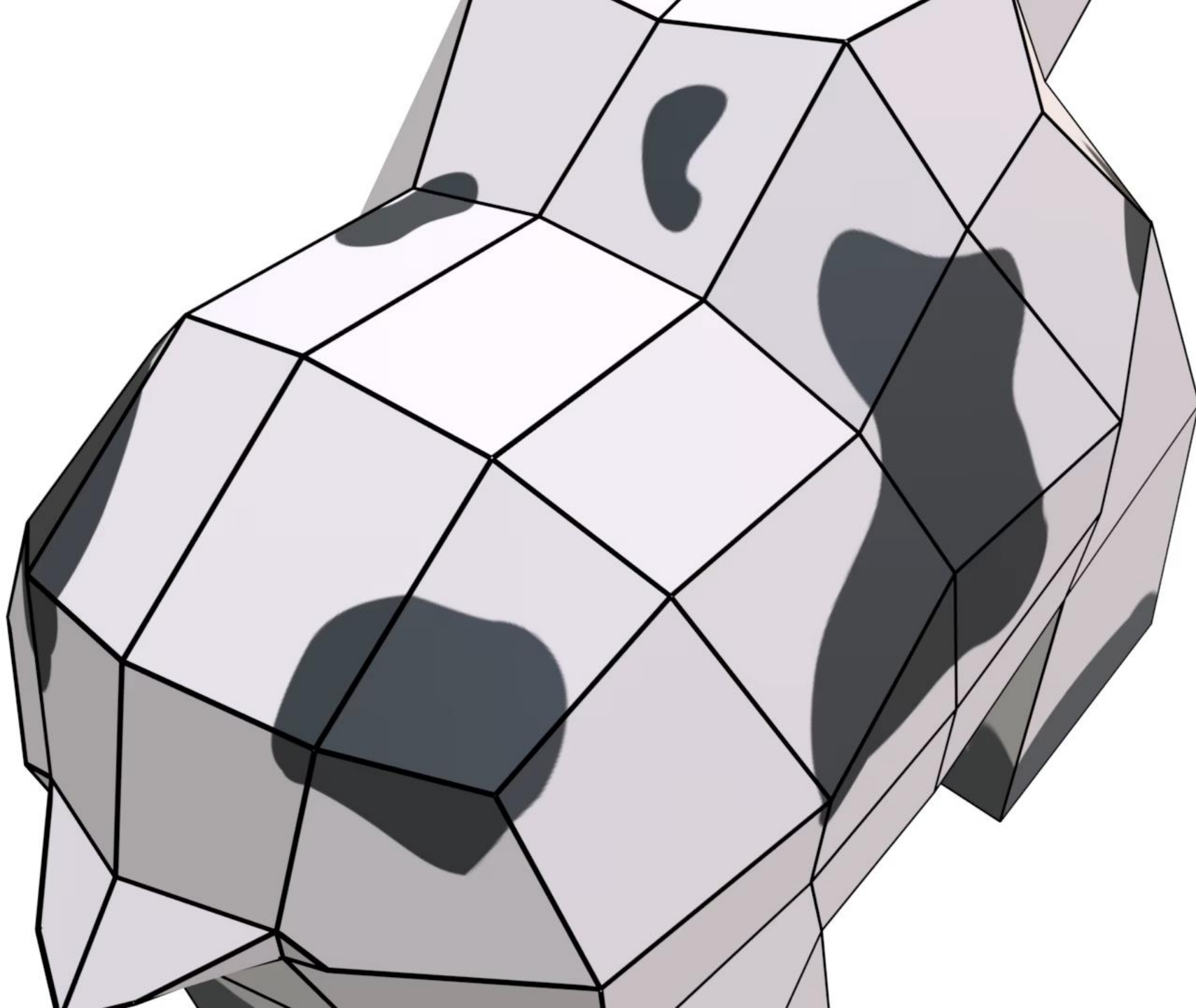
BASTIAN KUTH, MAX OBERBERGER, MATTHÄUS CHAJDAS, QUIRIN MEYER

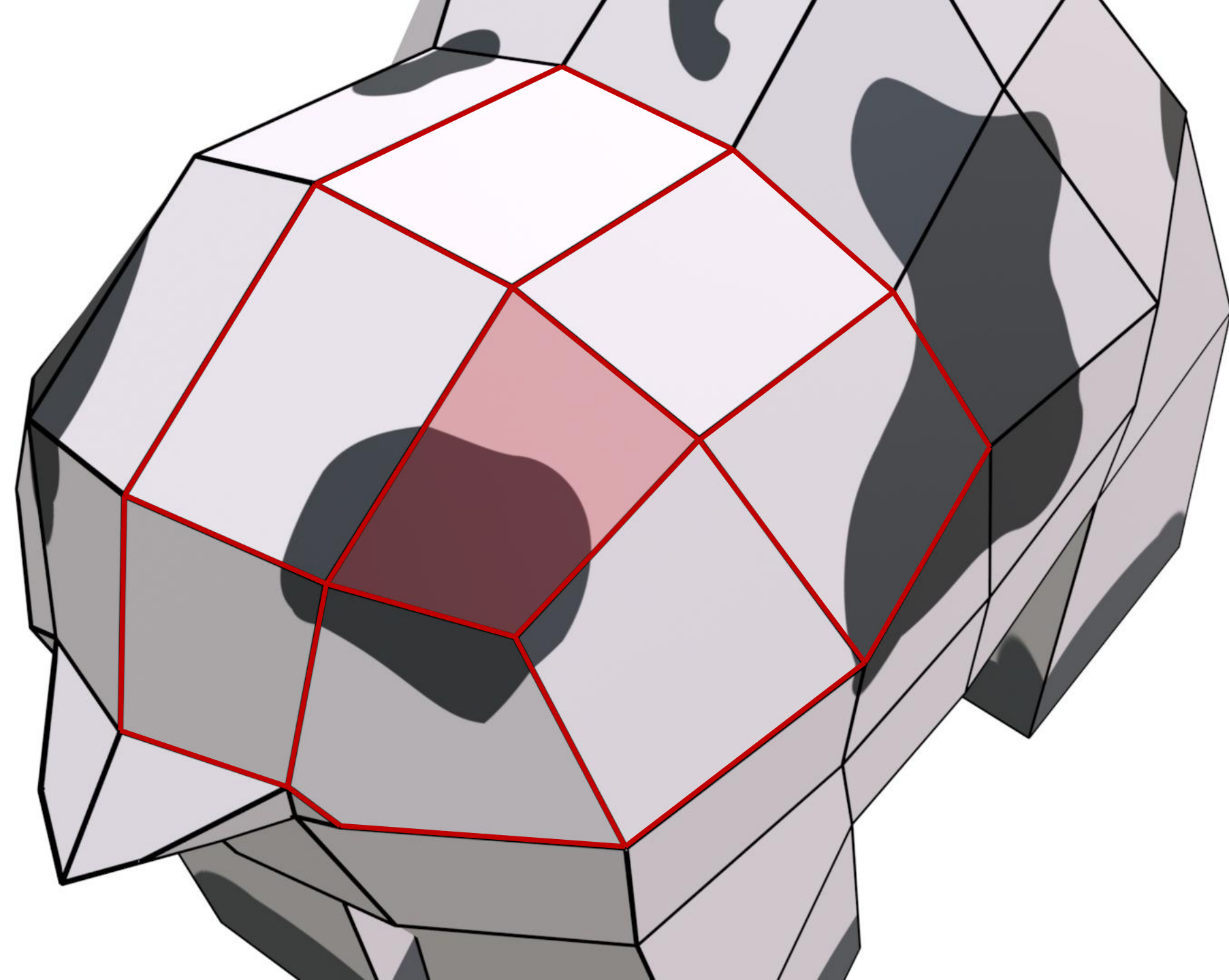


Catmull-Clark Subdivision

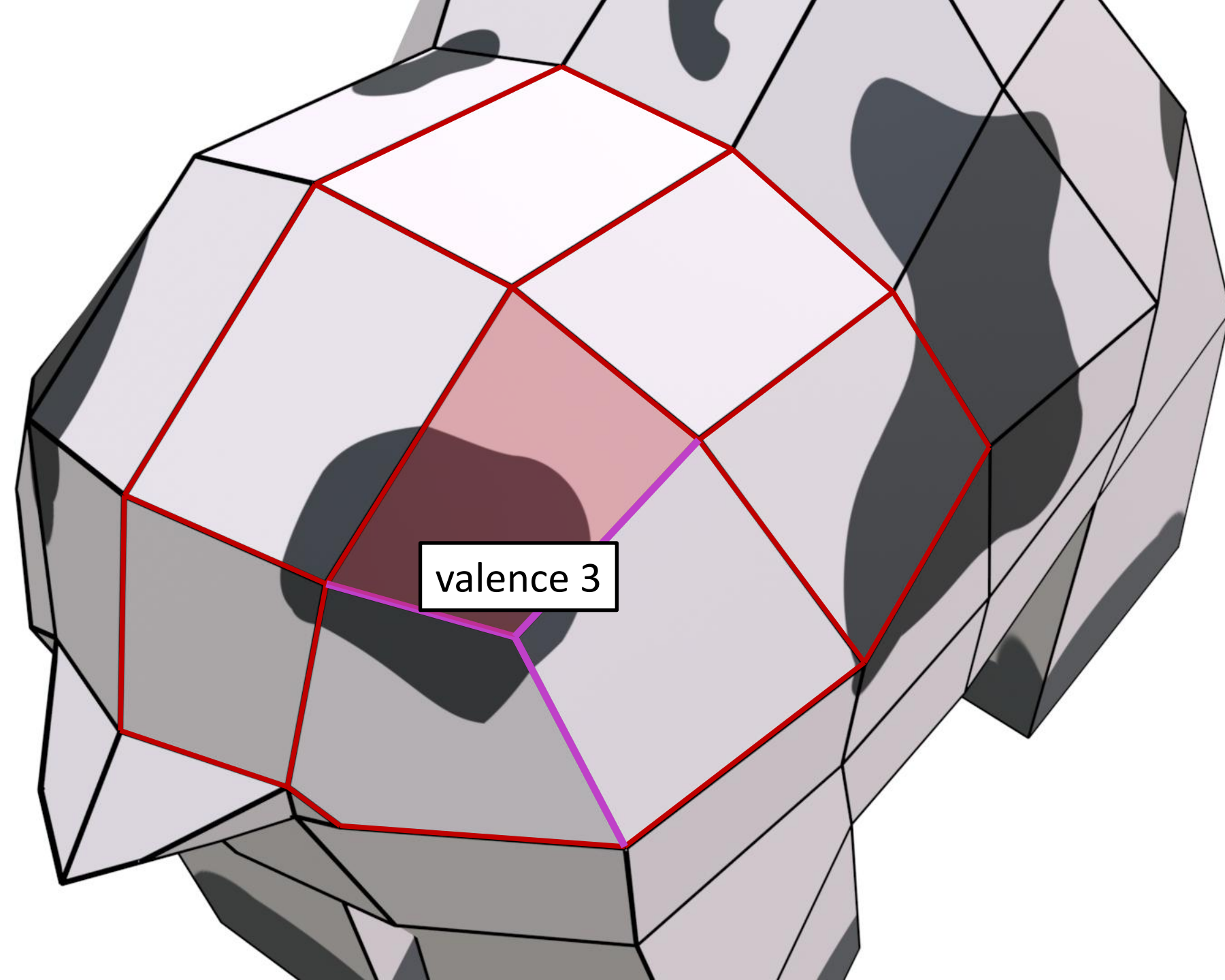


A 3D visualization of a complex polyhedral mesh, possibly representing a crystal structure or a computational domain. The mesh is composed of many triangular and quadrilateral faces. A specific face, a diamond-shaped quadrilateral, is highlighted in red. The background shows a grayscale image of a textured surface, possibly a rock or a biological specimen, with some dark, irregular shapes. The red mesh is overlaid on this background.
$$f(u, v) = \sum_i^3 \sum_j^3 P(i, j) \cdot B(u, i) \cdot B(v, j)$$



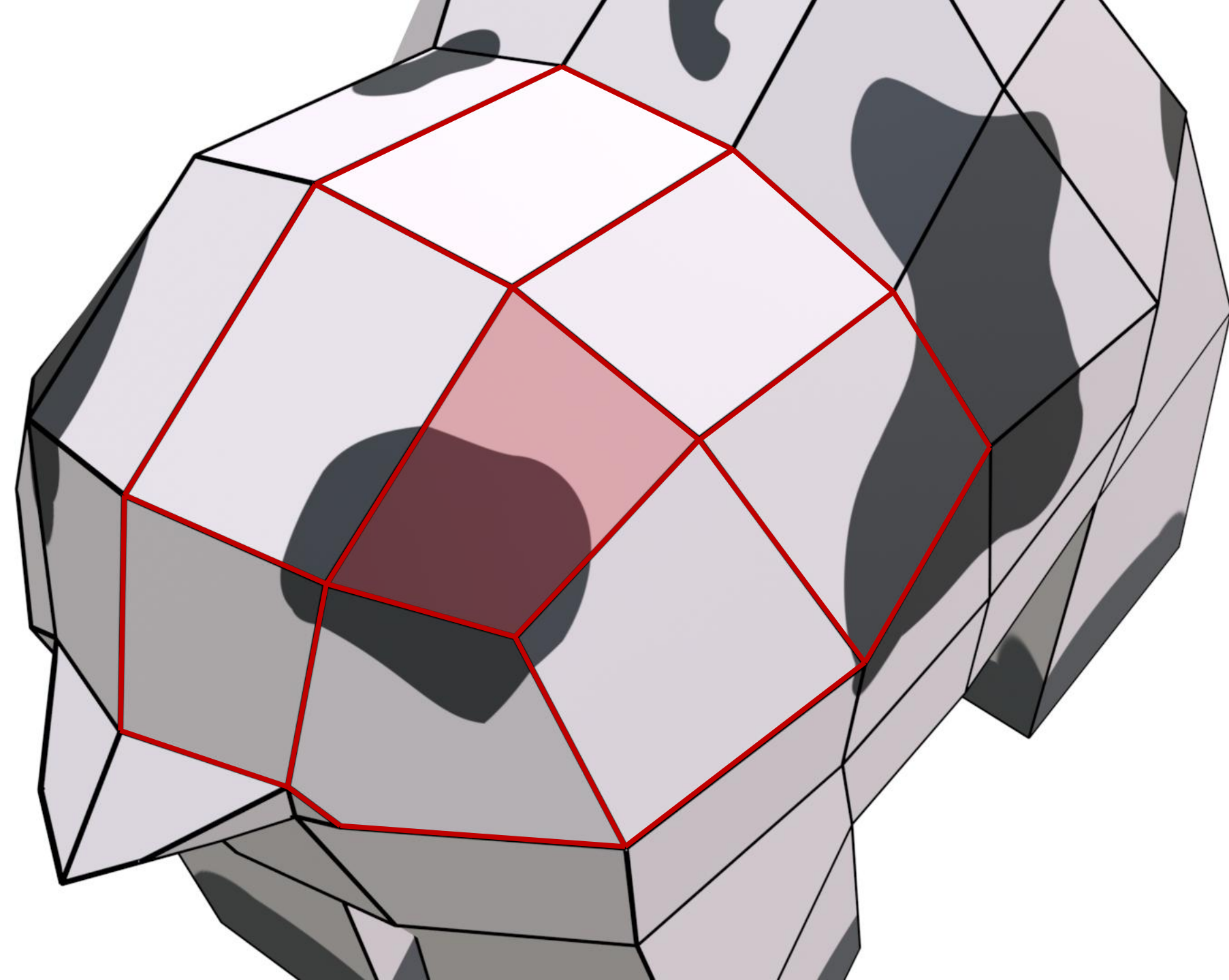


?

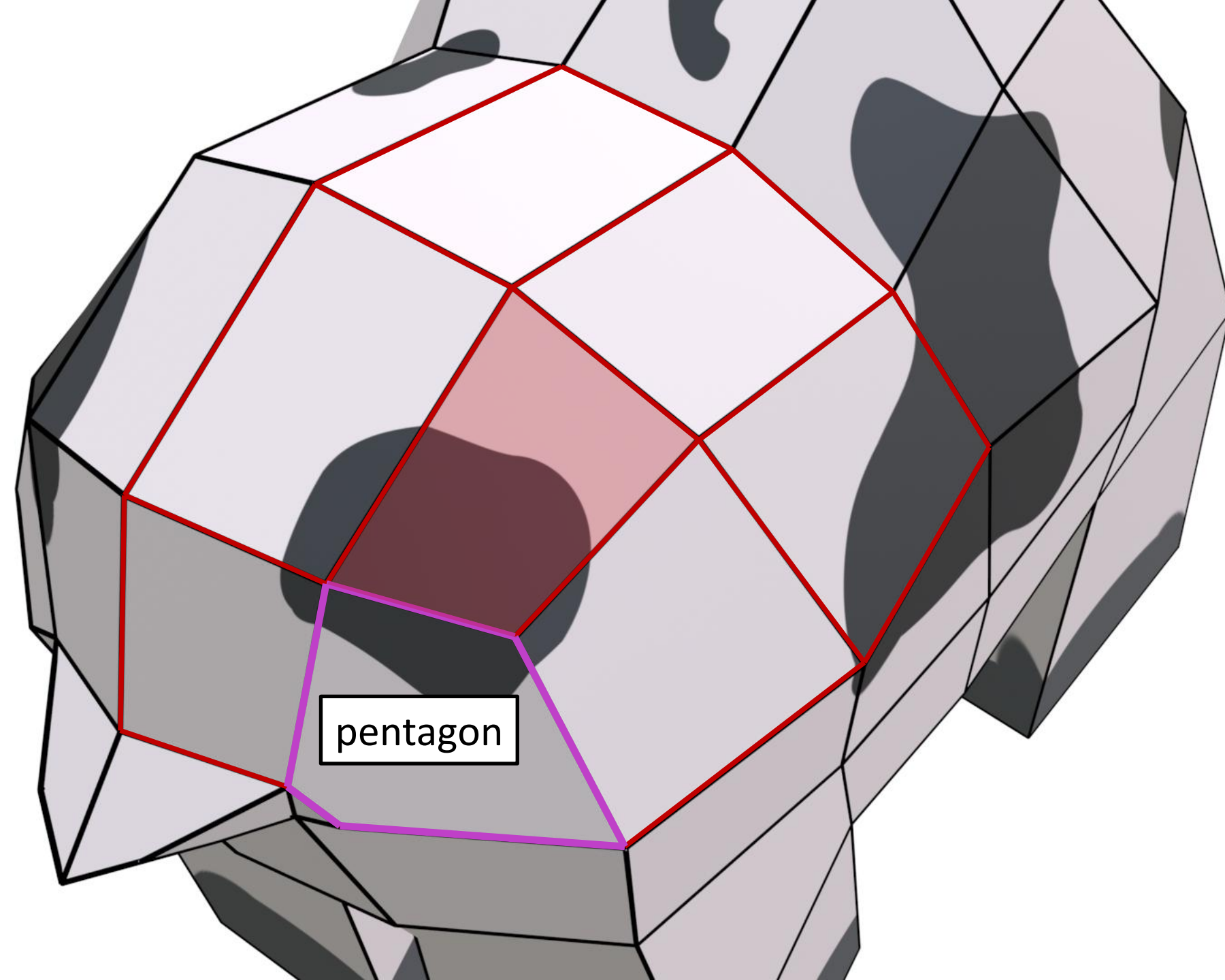


valence 3

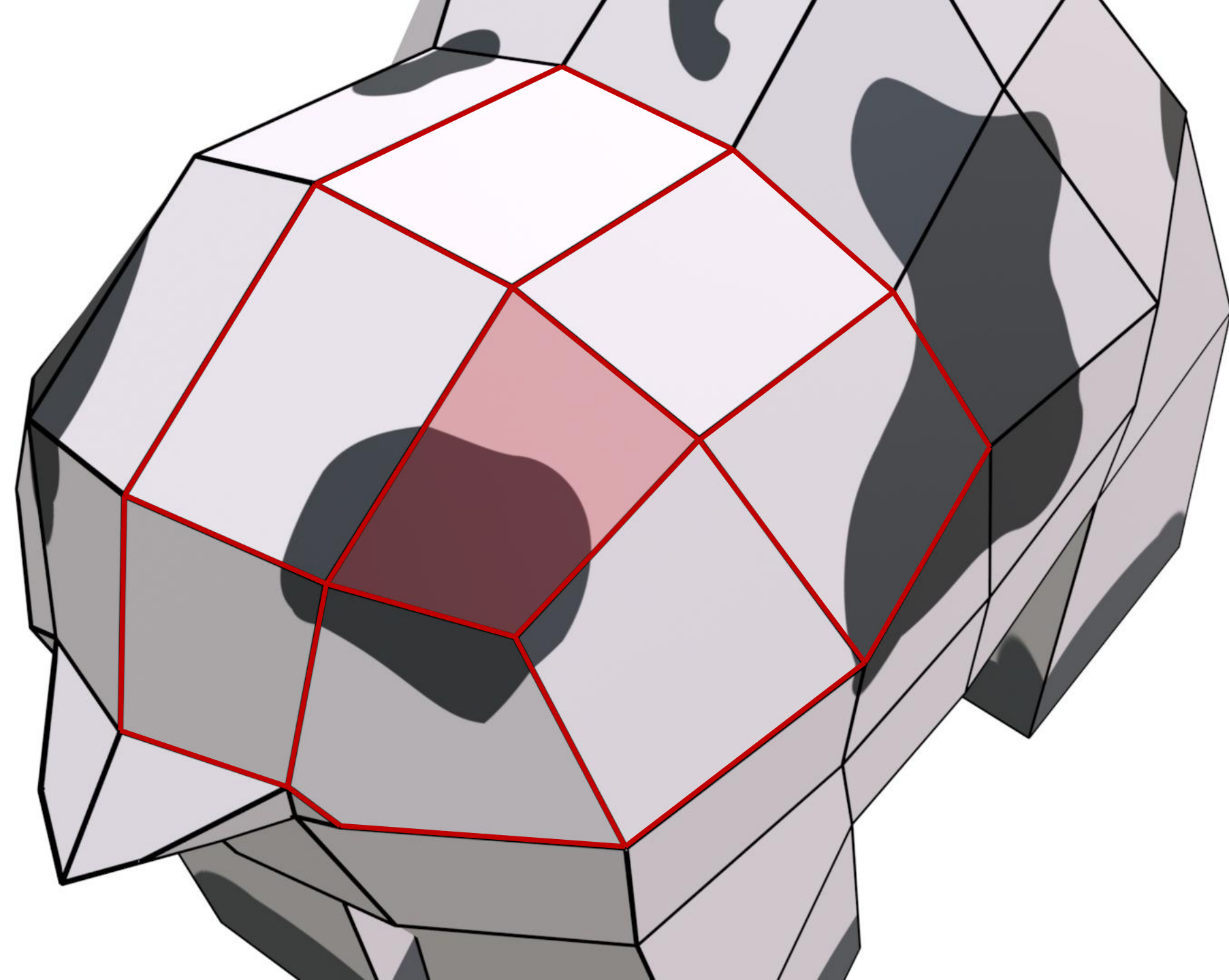
?



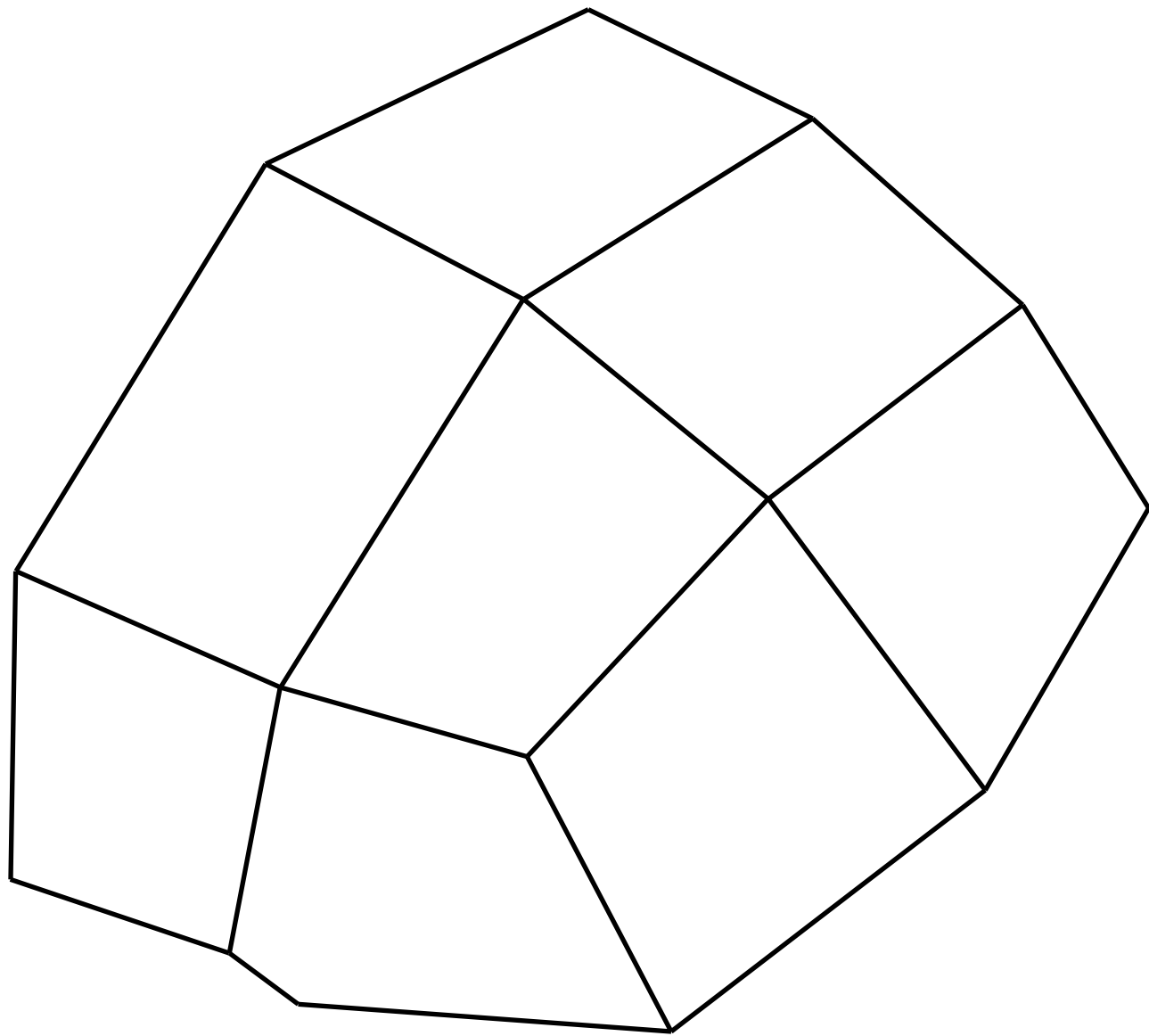
?

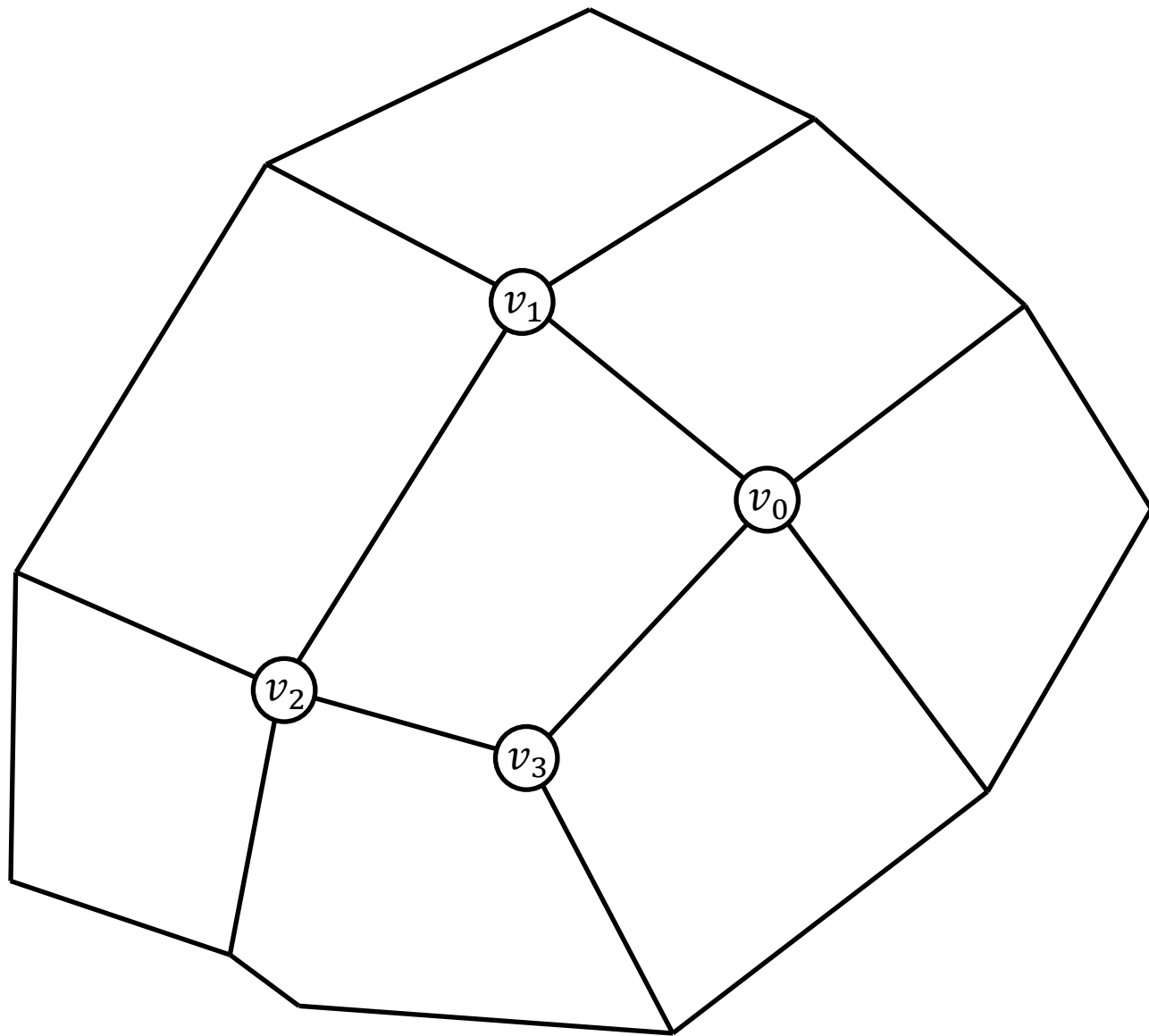


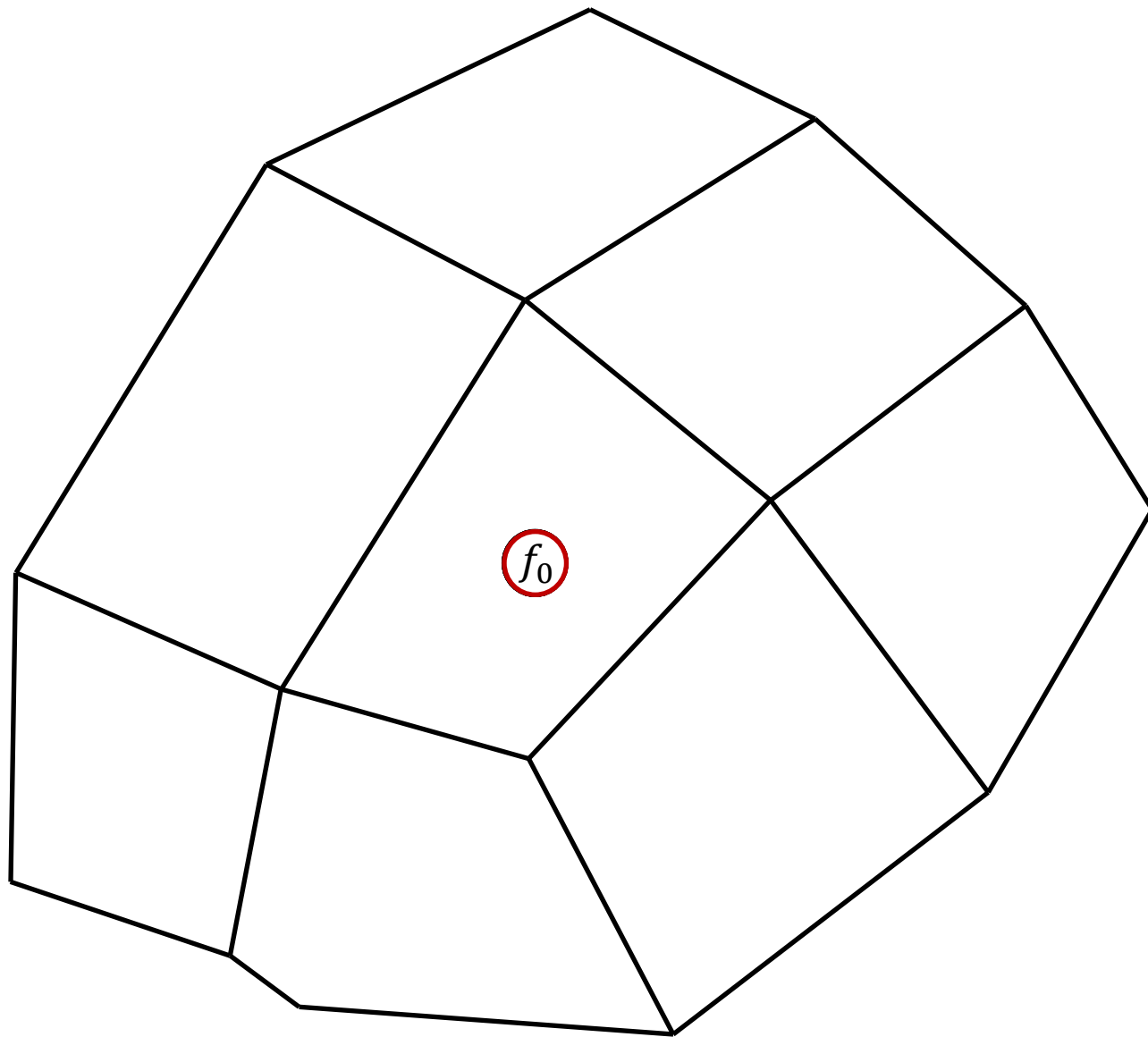
?



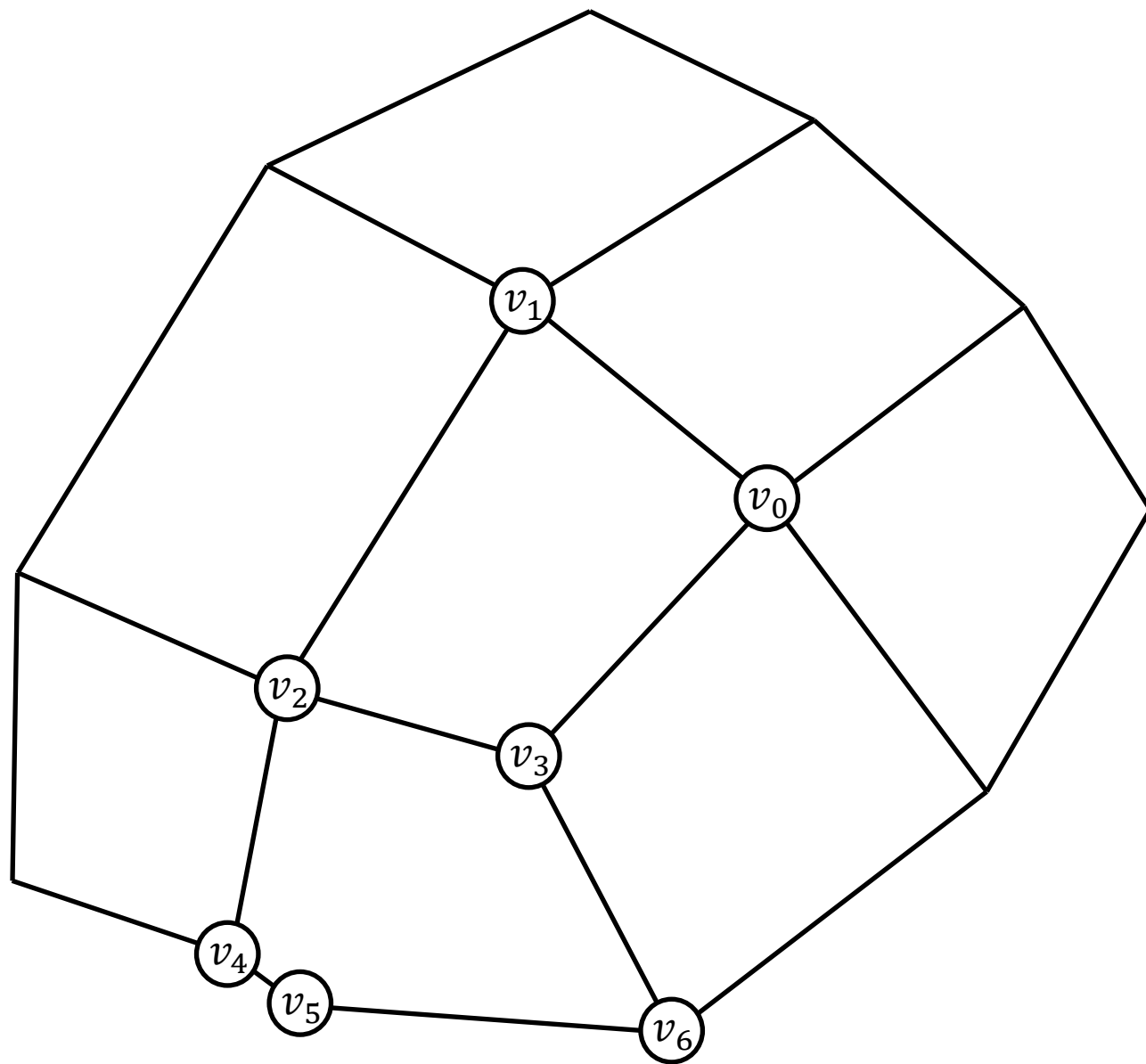
?



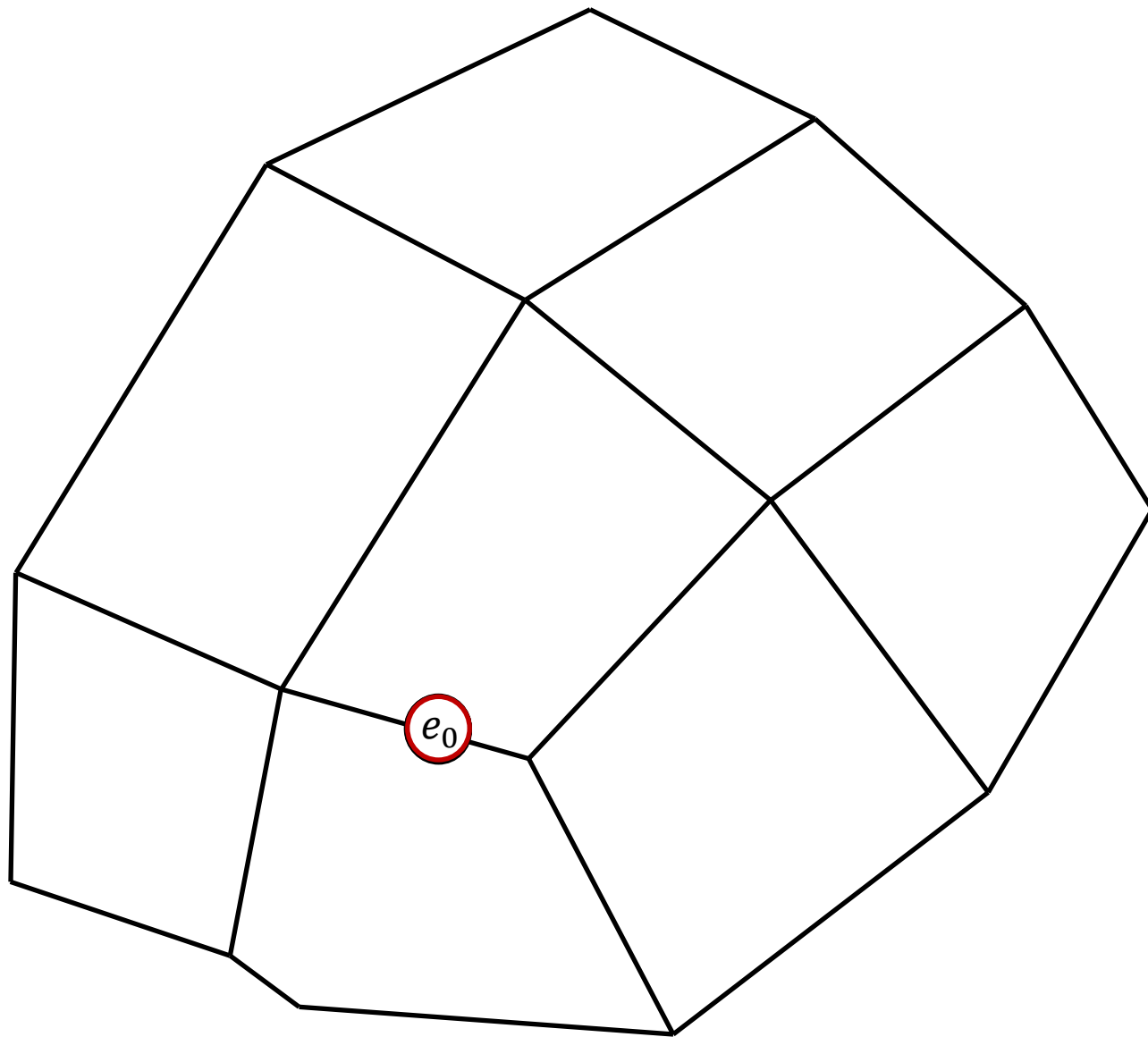




$$f = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

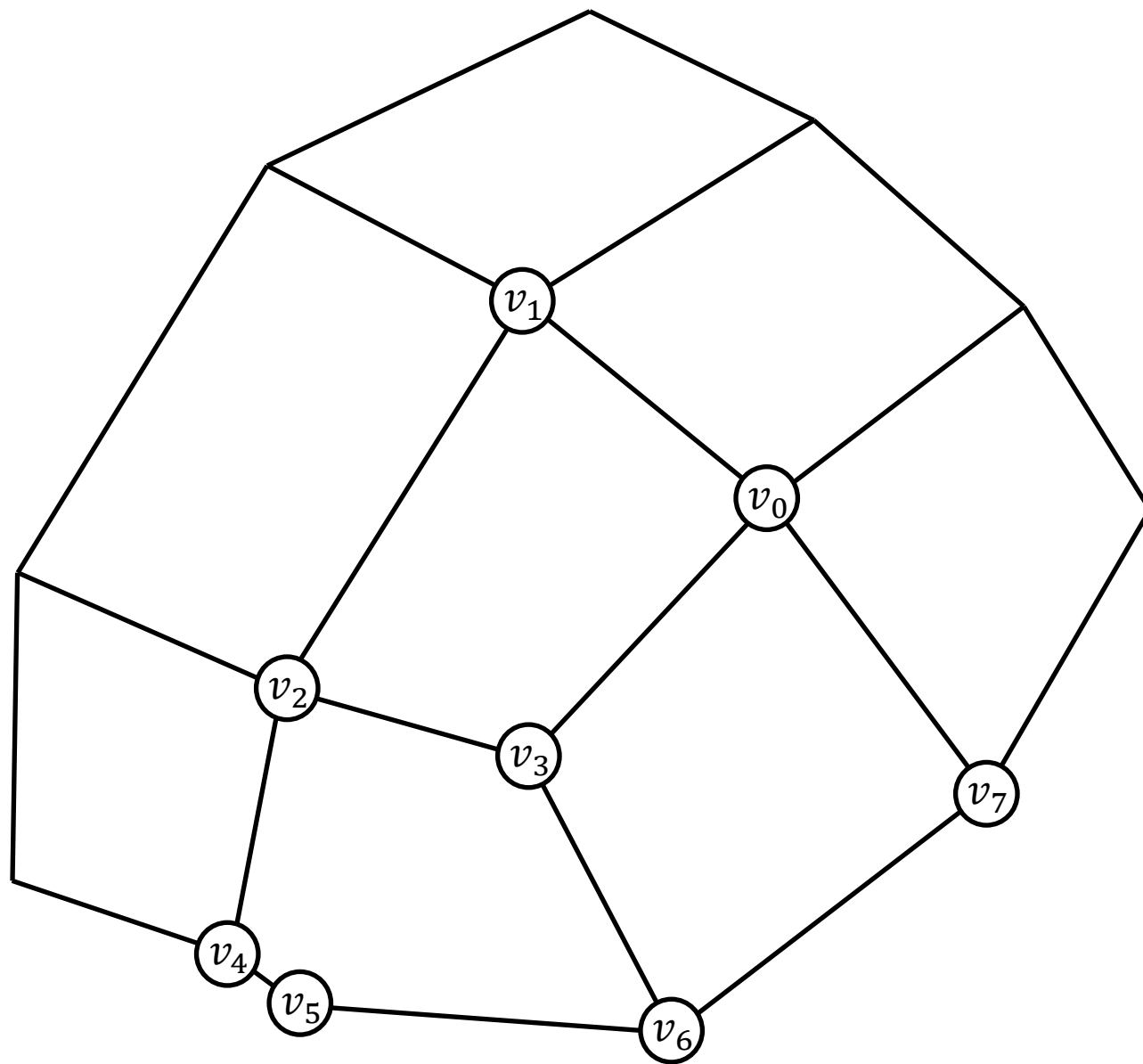


$$f = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$



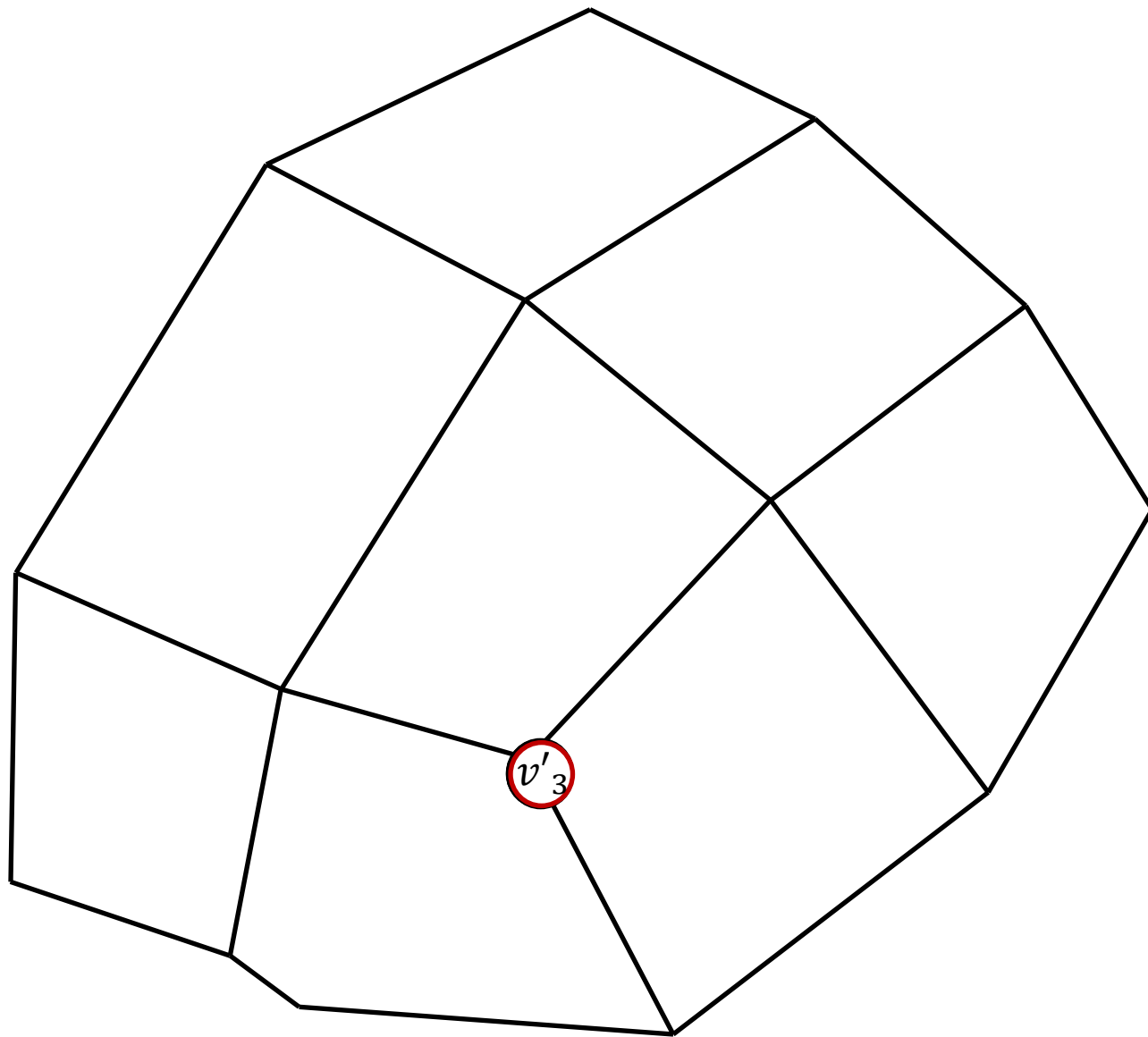
$$f = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

$$e = \frac{1}{4}(v_0 + v_1 + f_0 + f_1)$$



$$f = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

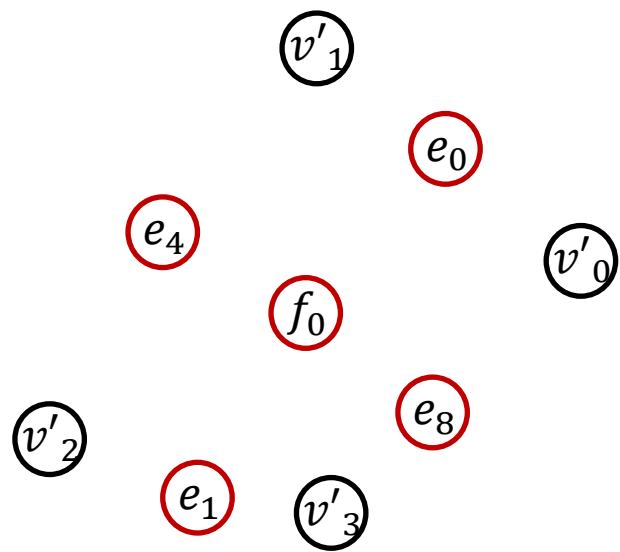
$$e = \frac{1}{4}(v_0 + v_1 + f_0 + f_1)$$

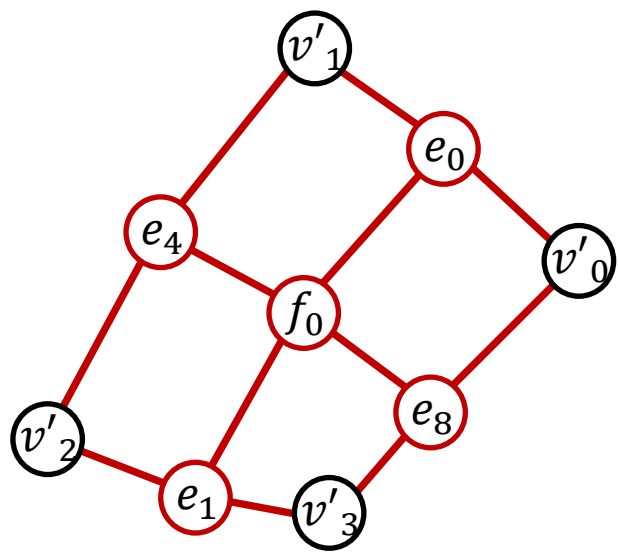


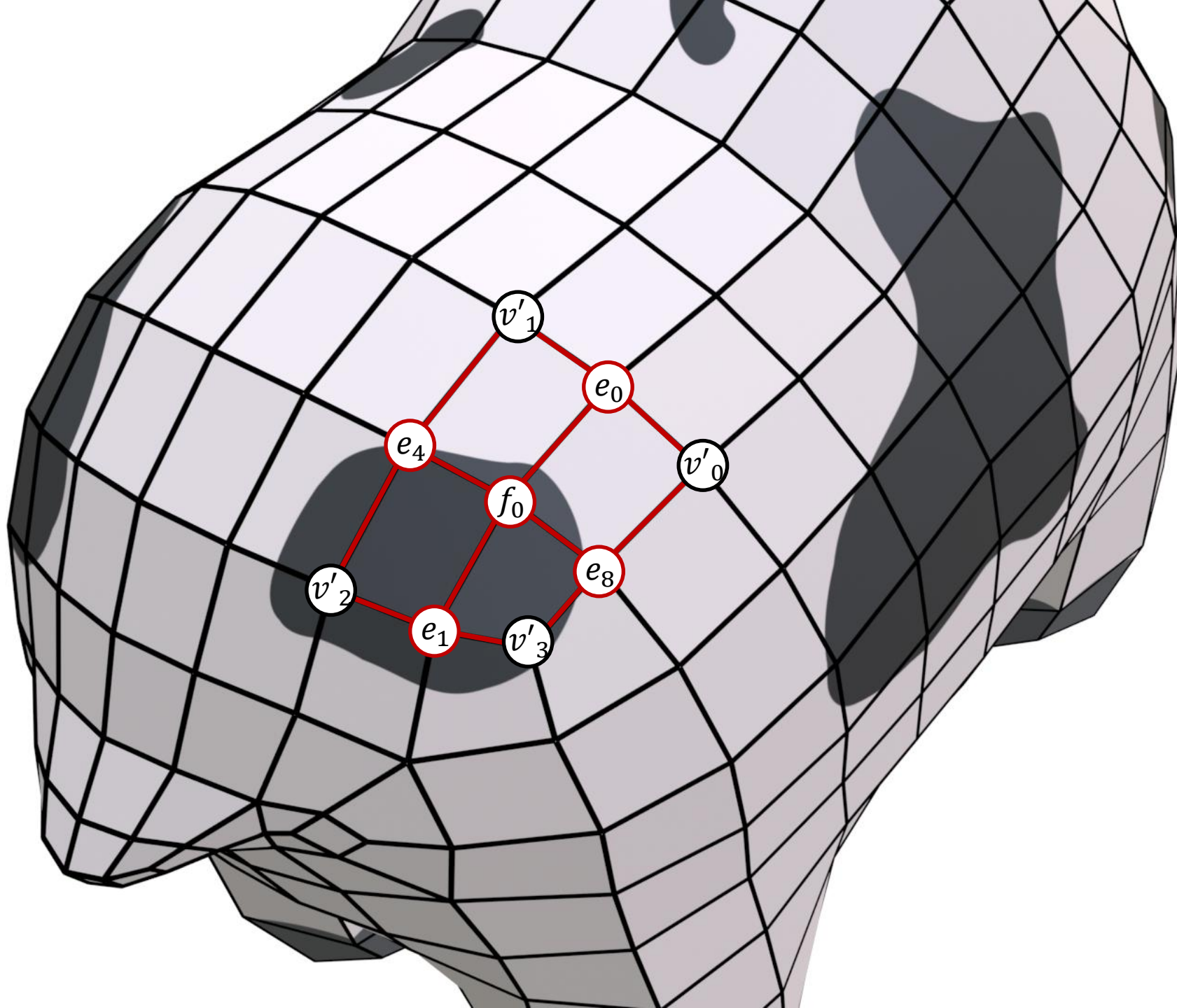
$$f = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

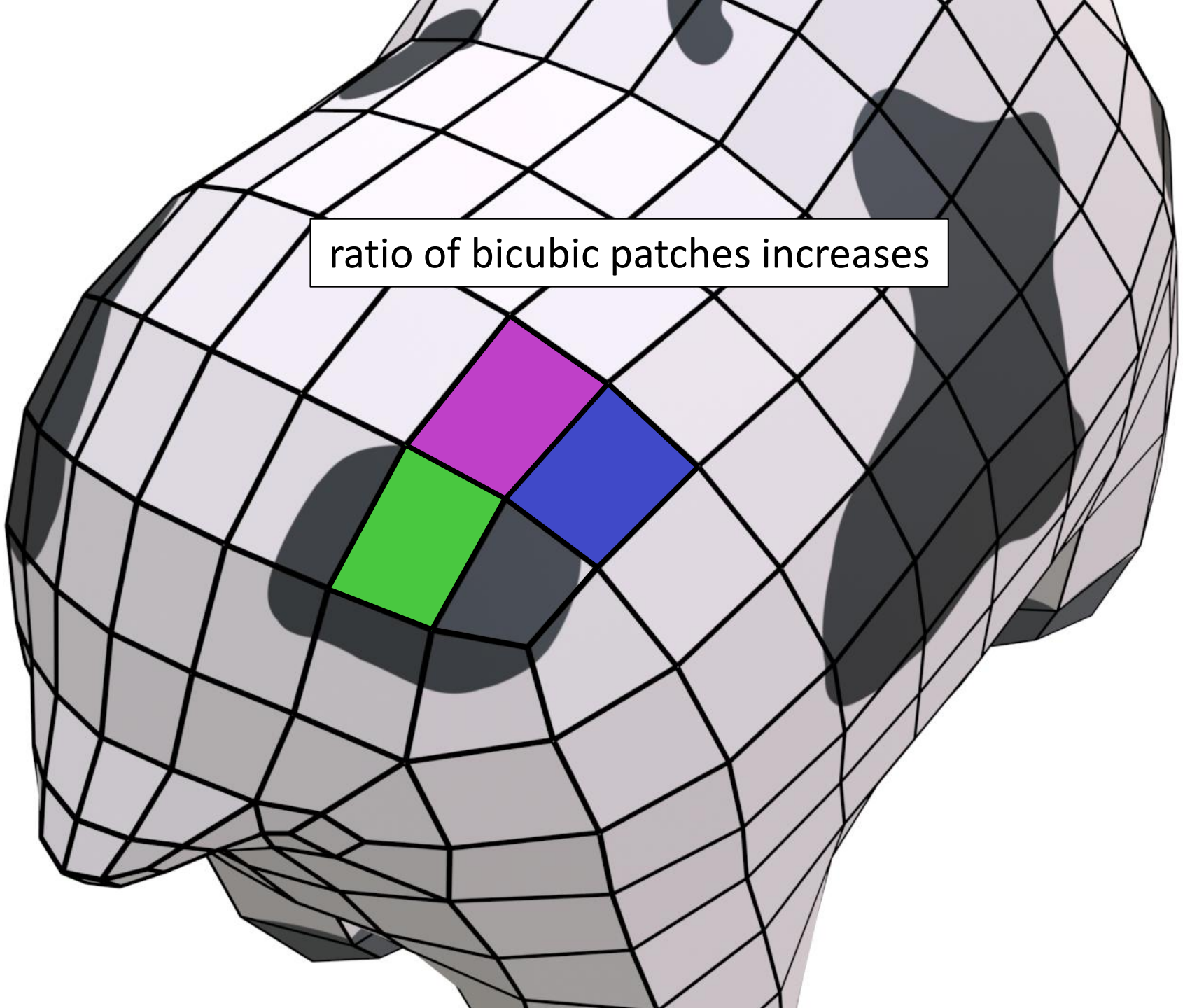
$$e = \frac{1}{4}(v_0 + v_1 + f_0 + f_1)$$

$$v' = \frac{1}{n}(Q + 2R + (n - 3)v)$$



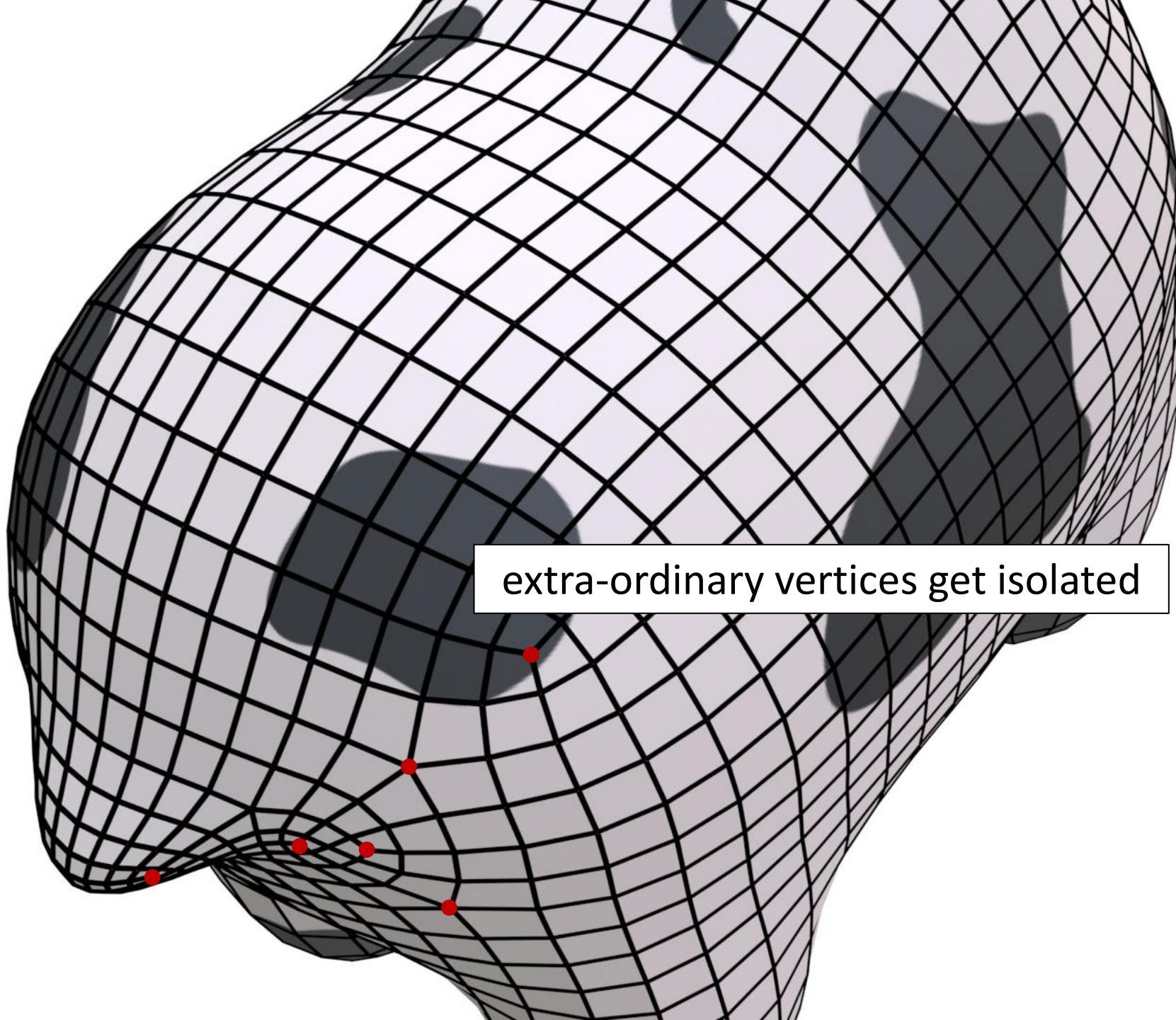






ratio of bicubic patches increases

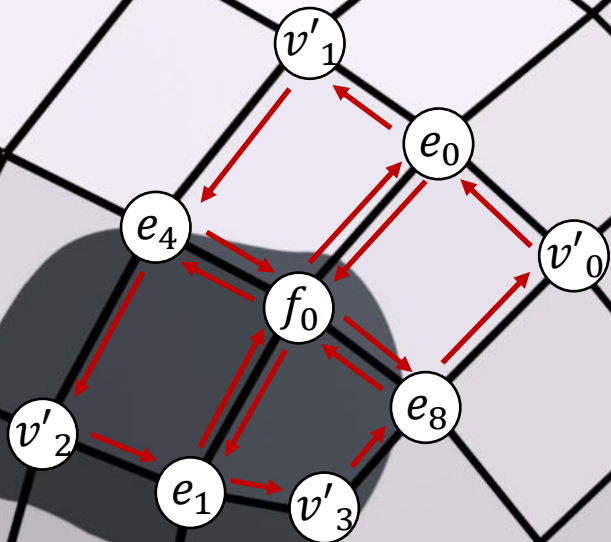
[NLMD12, BFK*16]



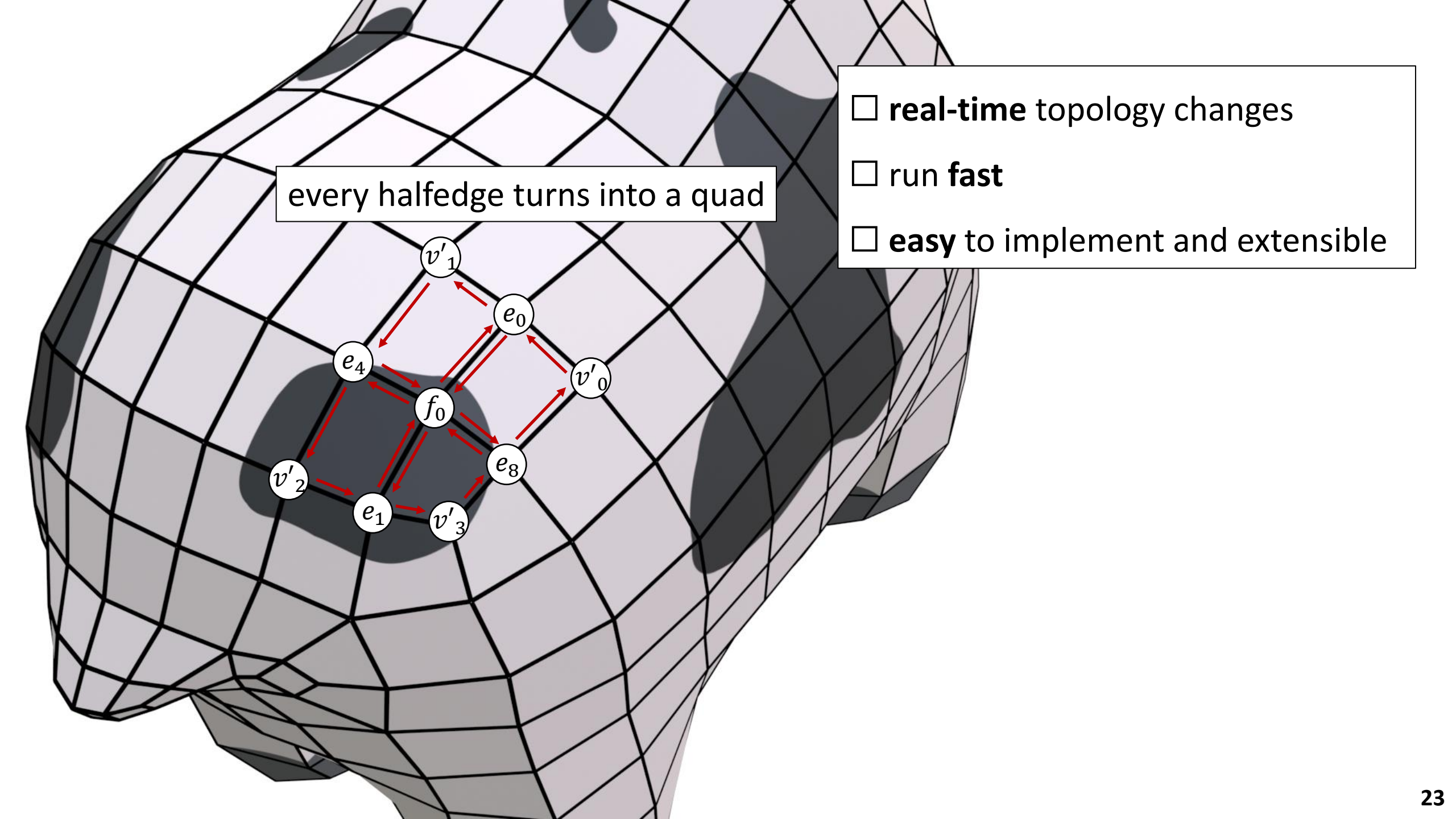
extra-ordinary vertices get isolated

[Sta98, LB07]

every halfedge turns into a quad

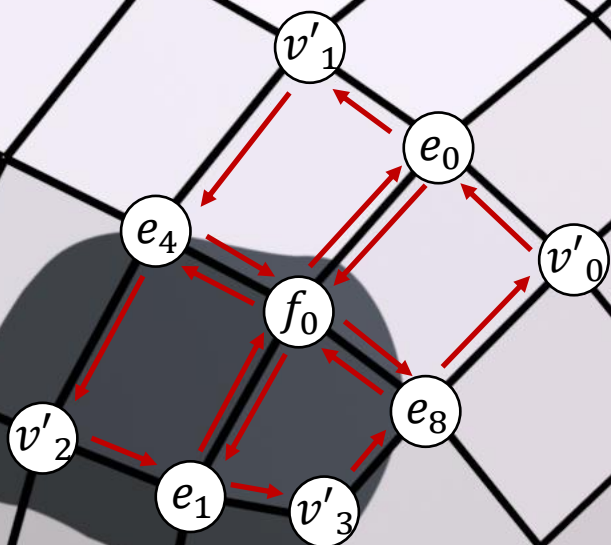


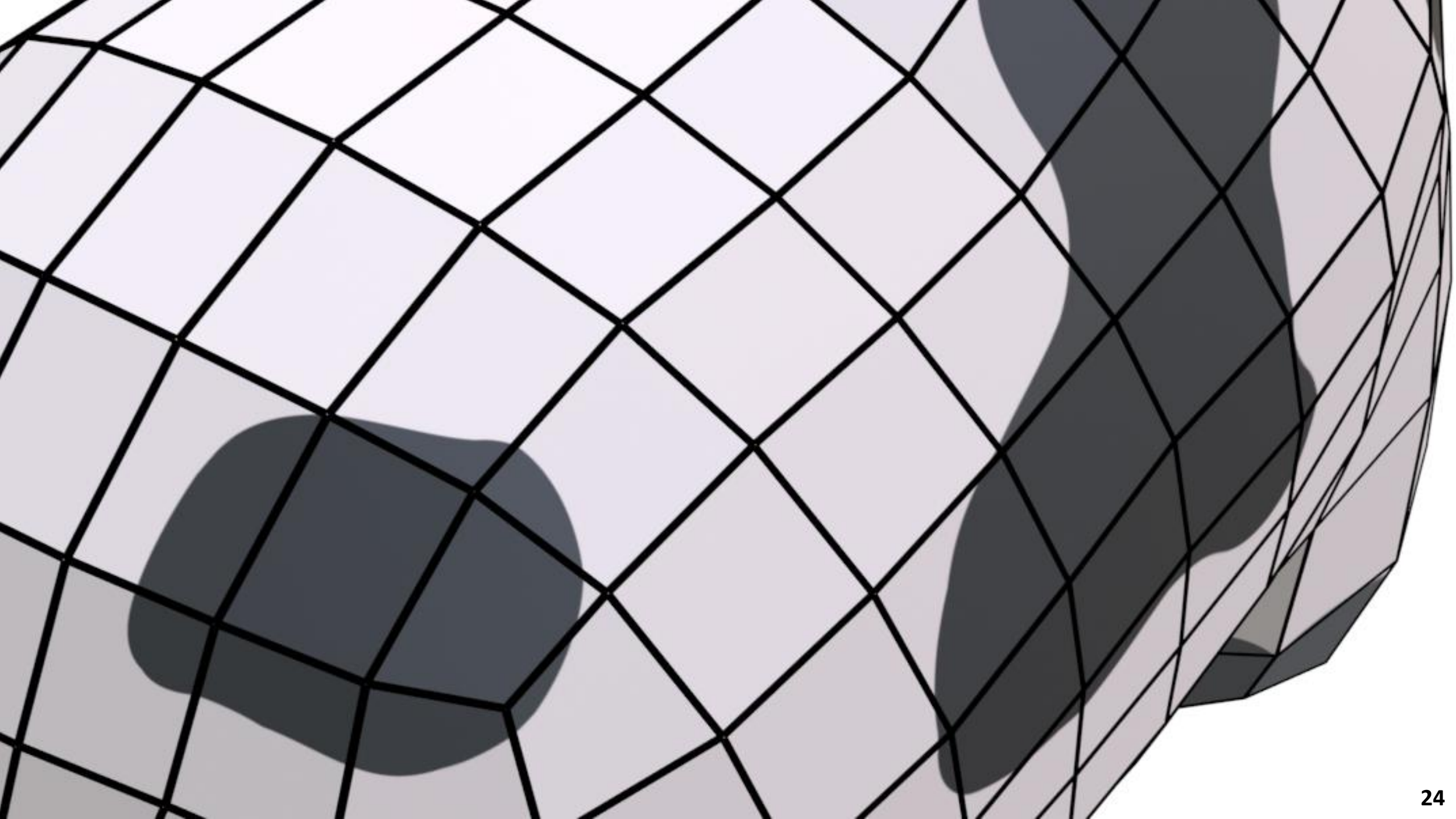
[DV21]

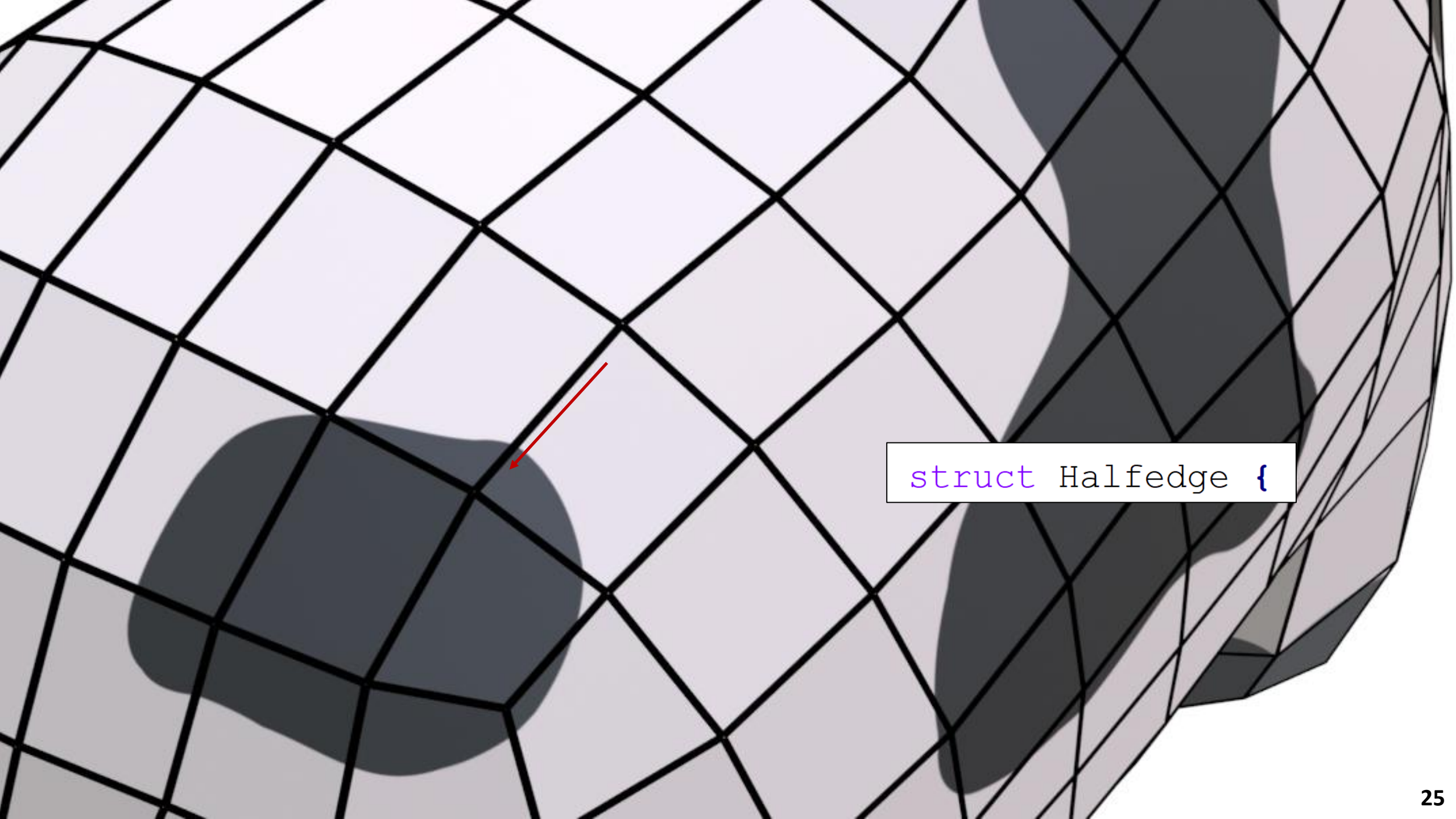


every halfedge turns into a quad

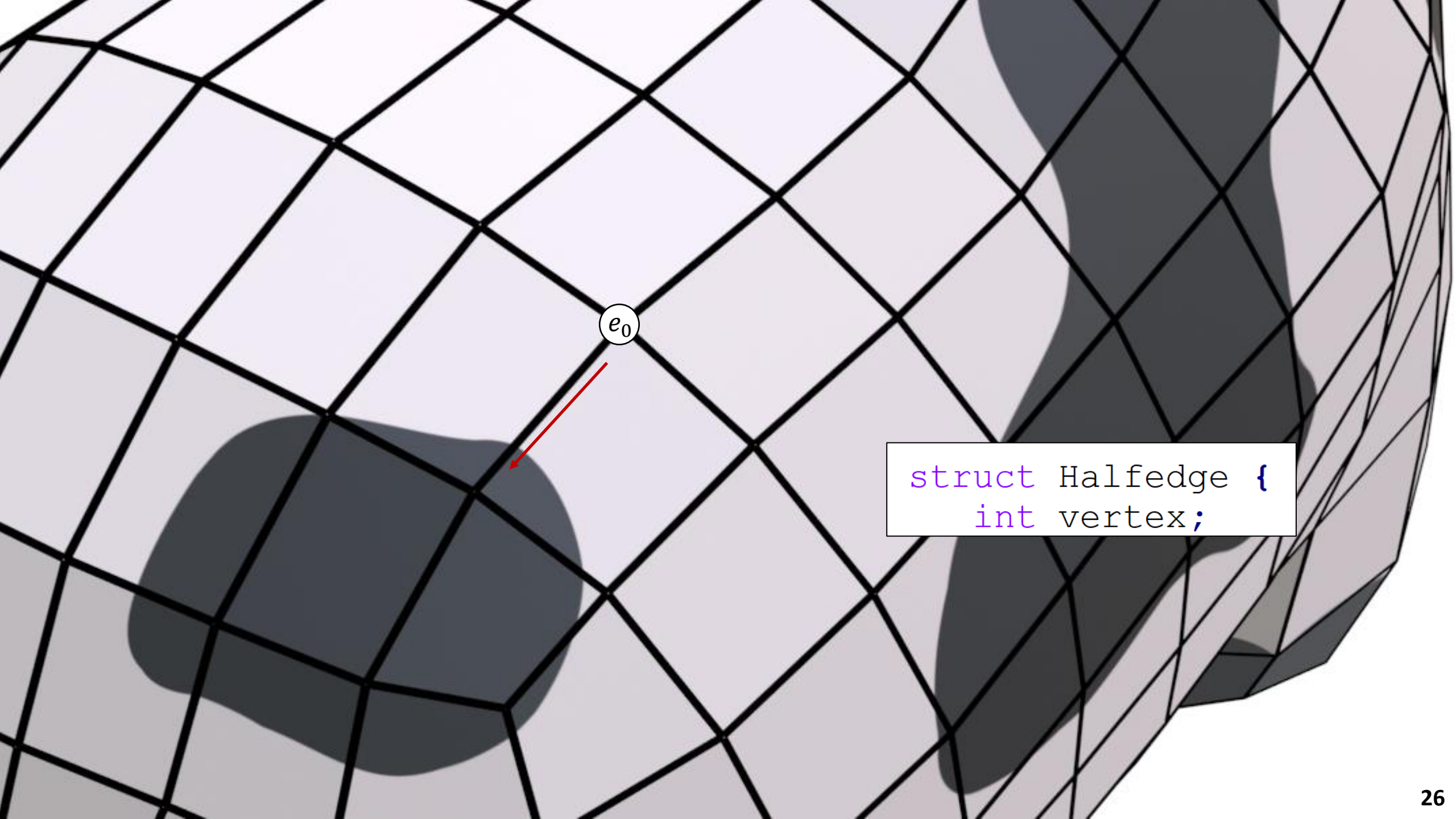
- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible



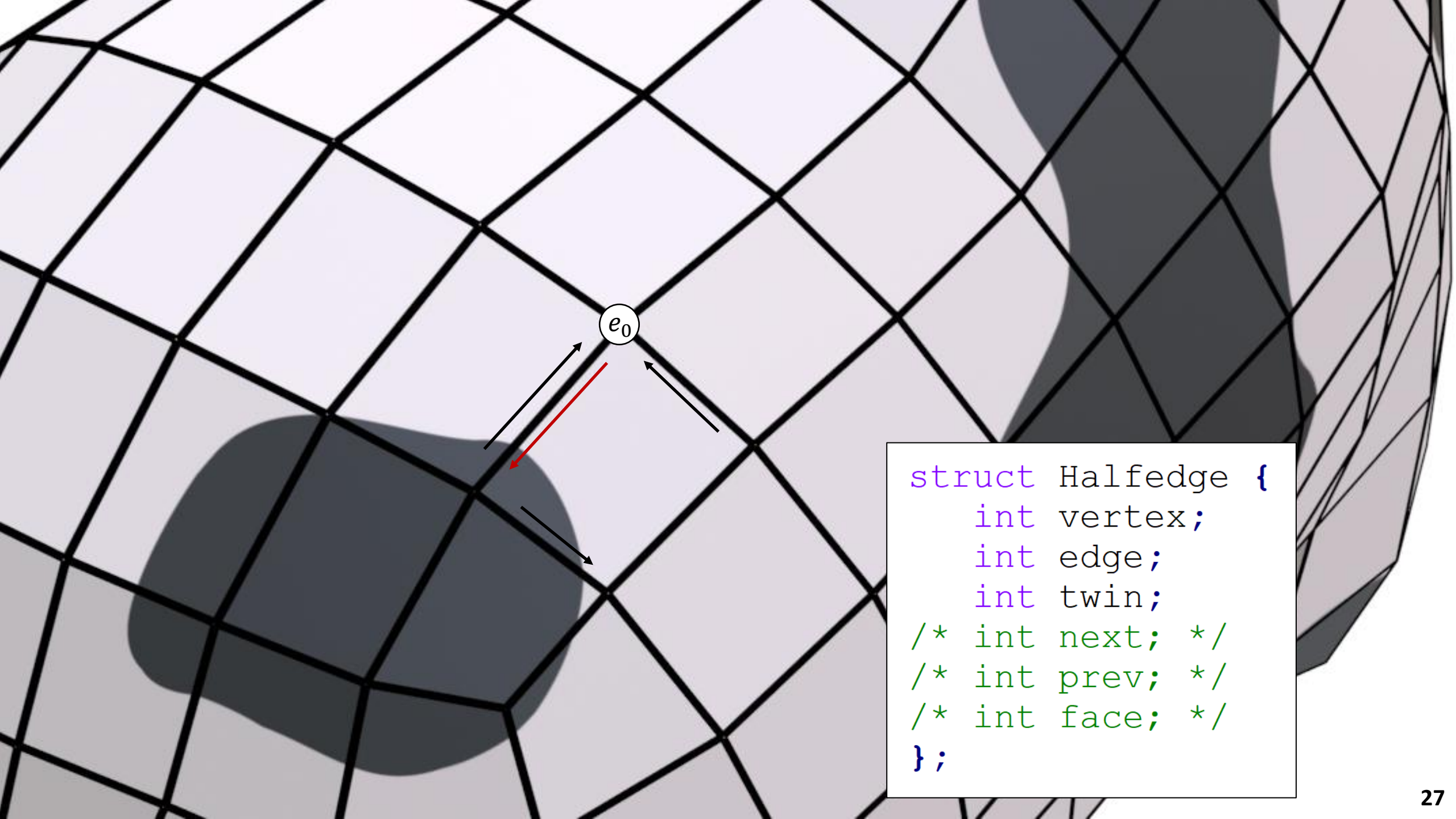




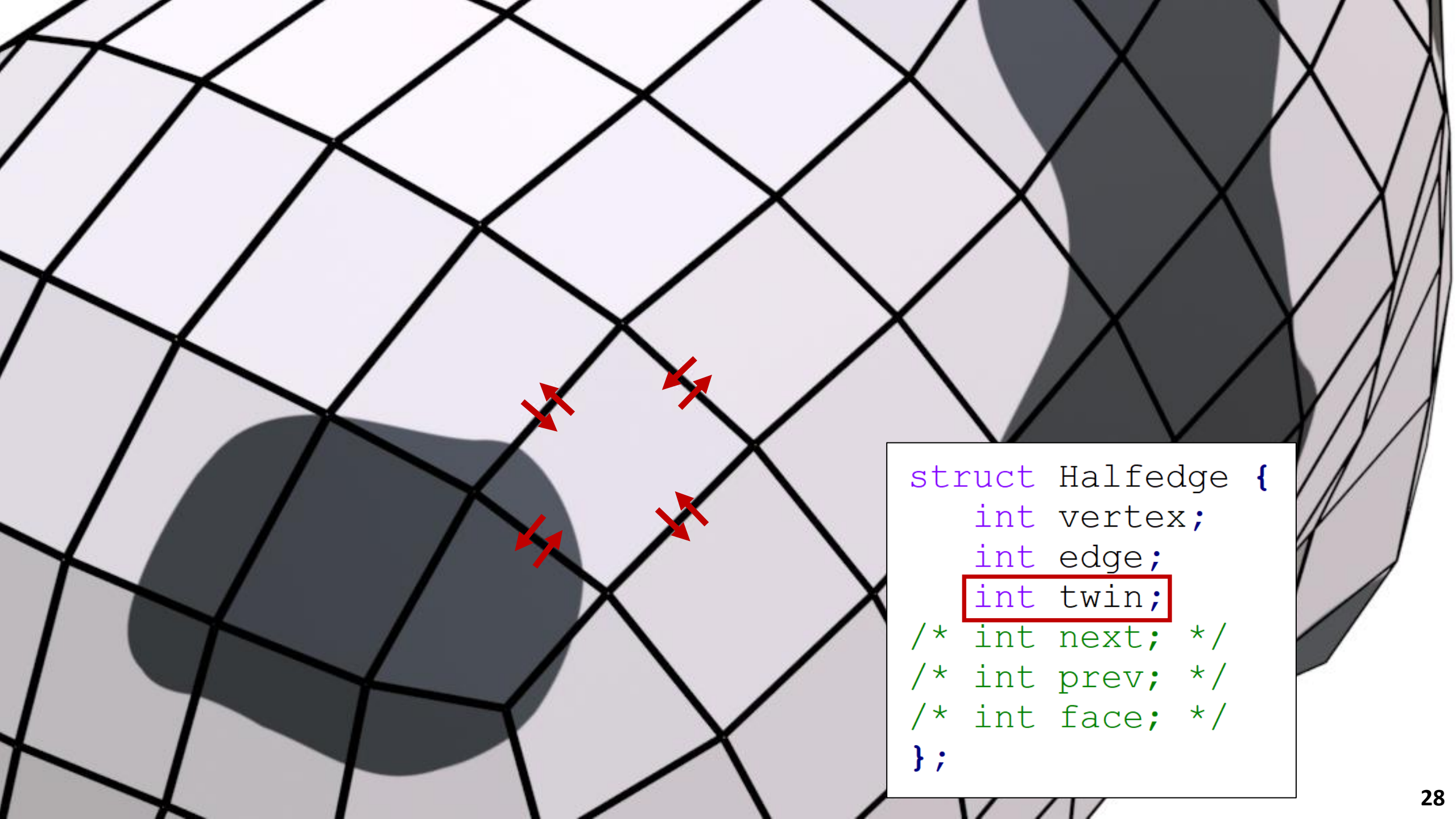
```
struct Halfedge {
```



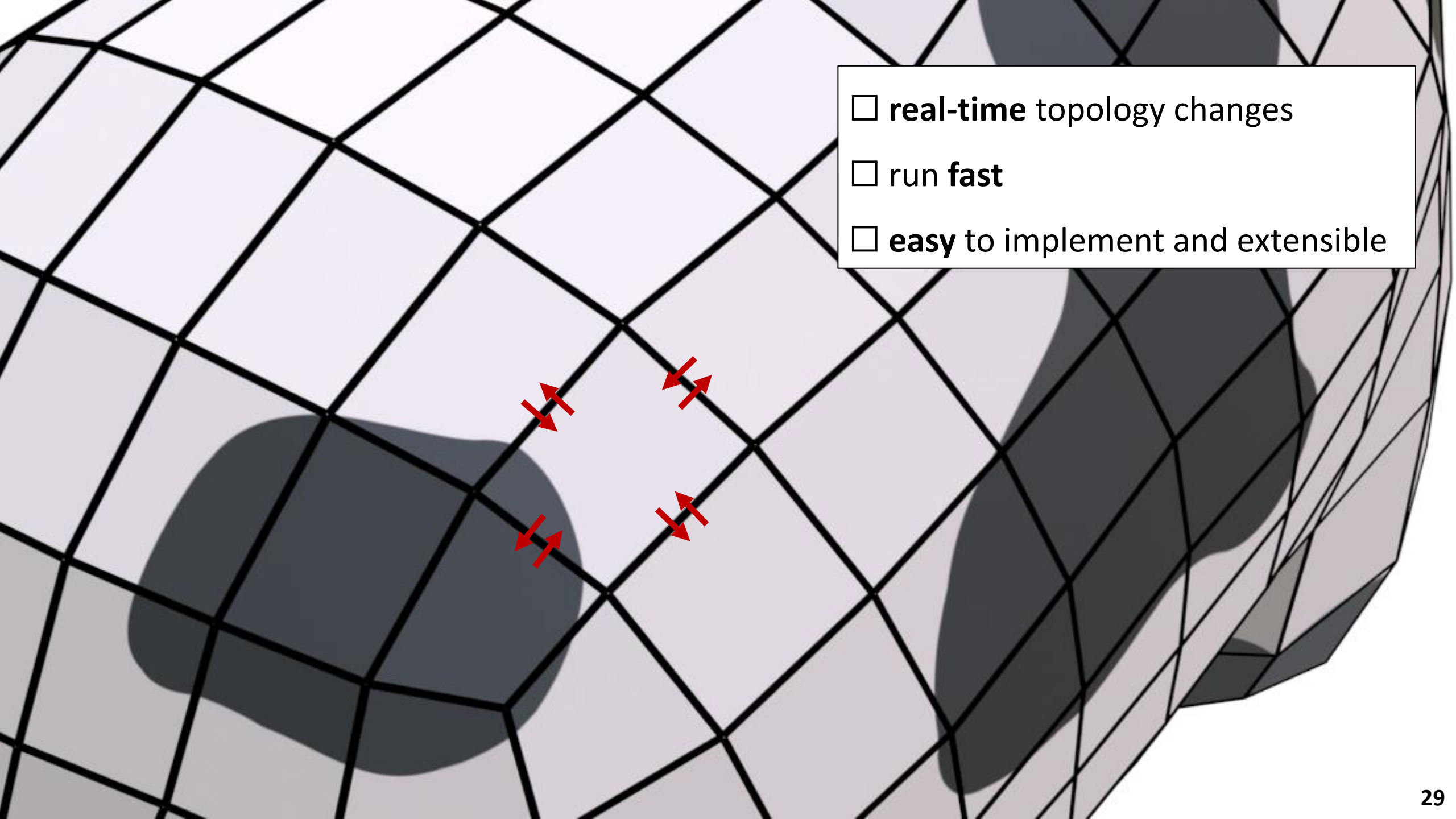
```
struct Halfedge {  
    int vertex;
```

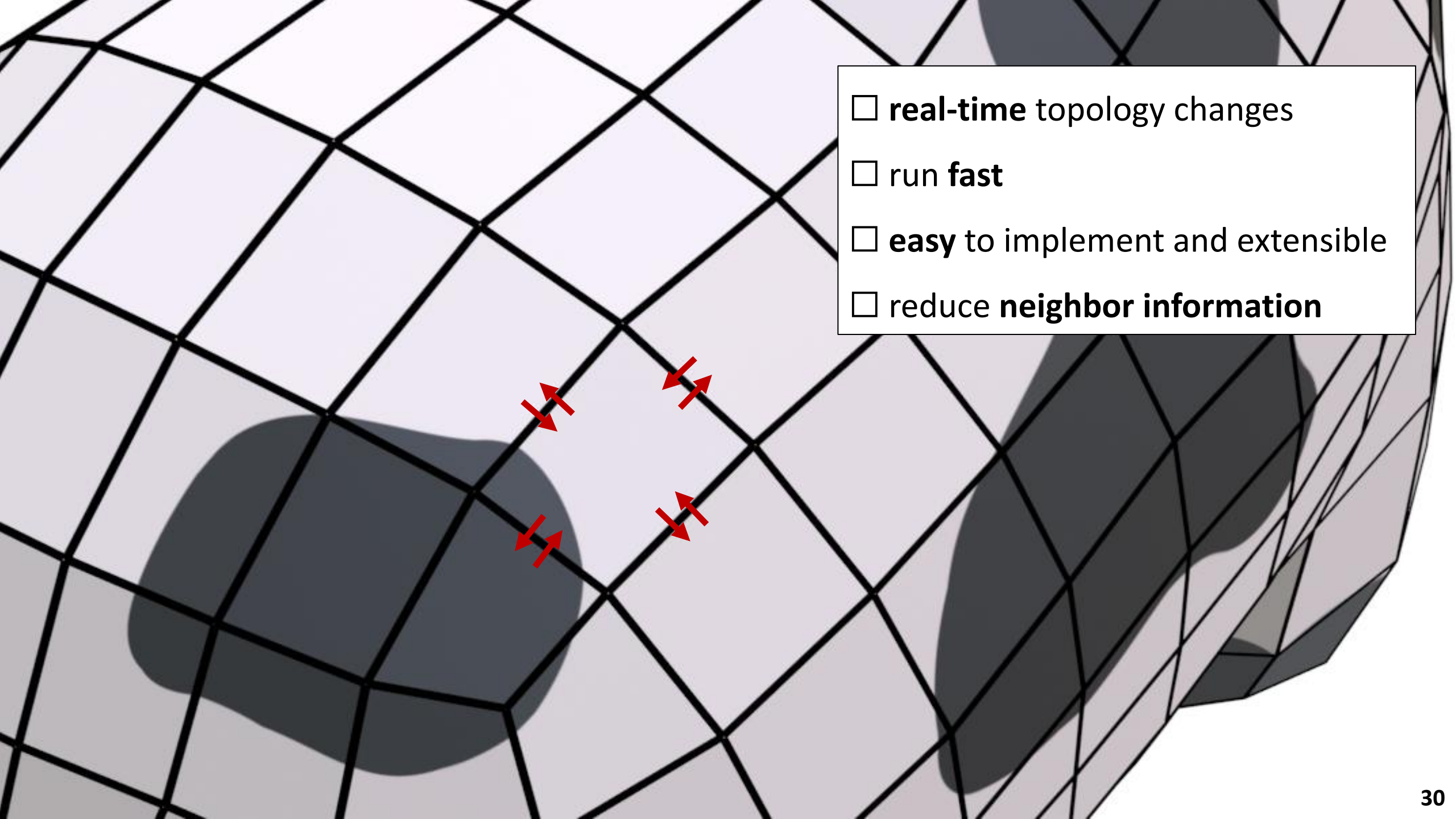



```
struct Halfedge {  
    int vertex;  
    int edge;  
    int twin;  
    /* int next; */  
    /* int prev; */  
    /* int face; */  
};
```



```
struct Halfedge {  
    int vertex;  
    int edge;  
    int twin;  
    /* int next; */  
    /* int prev; */  
    /* int face; */  
};
```

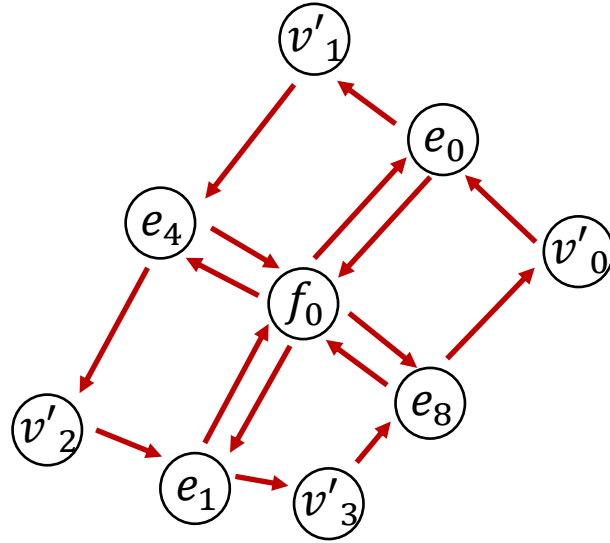

- 
- ☐ **real-time** topology changes
 - ☐ run **fast**
 - ☐ **easy** to implement and extensible

- 
- ☐ **real-time** topology changes
 - ☐ run **fast**
 - ☐ **easy** to implement and extensible
 - ☐ reduce **neighbor information**

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

```

[DV21]

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

```

for all  $h \in [0, H_d)$  do
  // apply rule Fig. 3 (a):
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$ 
  // apply rules Fig. 3 (b, ..., f):
  ...
end for

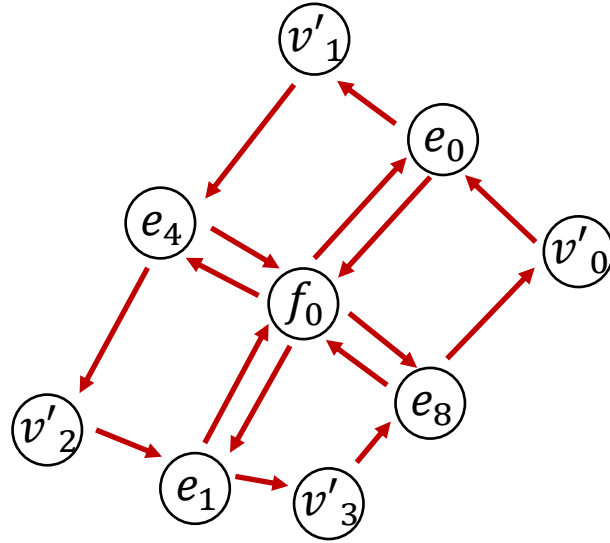
```



```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

```

[DV21]

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

for all $h \in [0, H_d)$ **do**

// apply rule Fig. 3 (a):

$\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$

// apply rules Fig. 3 (b, ..., f):

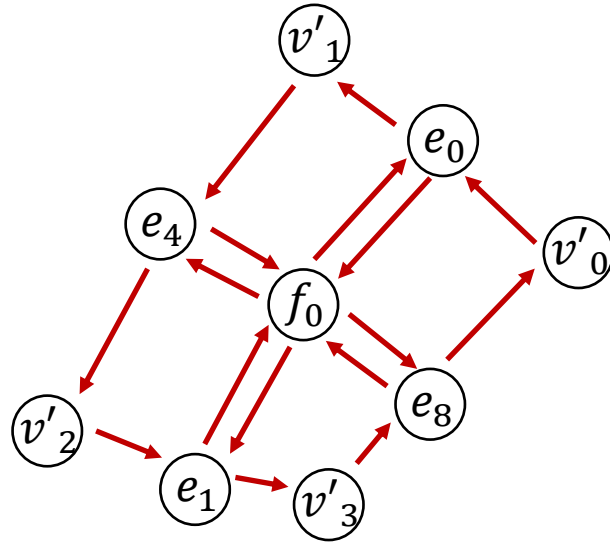
...

end for

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do

```

- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information

[DV21]

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

```

for all  $h \in [0, H_d)$  do

```

// apply rule Fig. 3 (a):

$\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$

// apply rules Fig. 3 (b, ..., f):

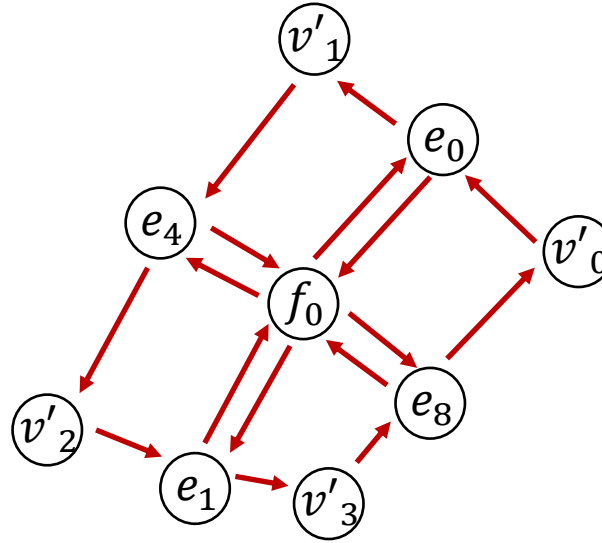
...

end for

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do

```

- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information
- ☐ merge to **single dispatch**

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

```

for all  $h \in [0, H_d)$  do

```

// apply rule Fig. 3 (a):

$\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$

$\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$

// apply rules Fig. 3 (b, ..., f):

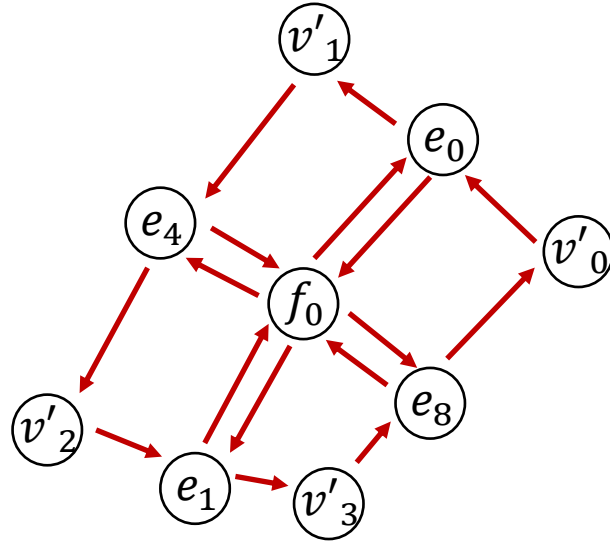
...

end for

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

```

[DV21]

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

```

for all  $h \in [0, H_d)$  do
  // apply rule Fig. 3 (a):
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$ 
  // apply rules Fig. 3 (b, ..., f):
  ...
end for

```



```
#include <iostream>
#include <bit>
int main() {

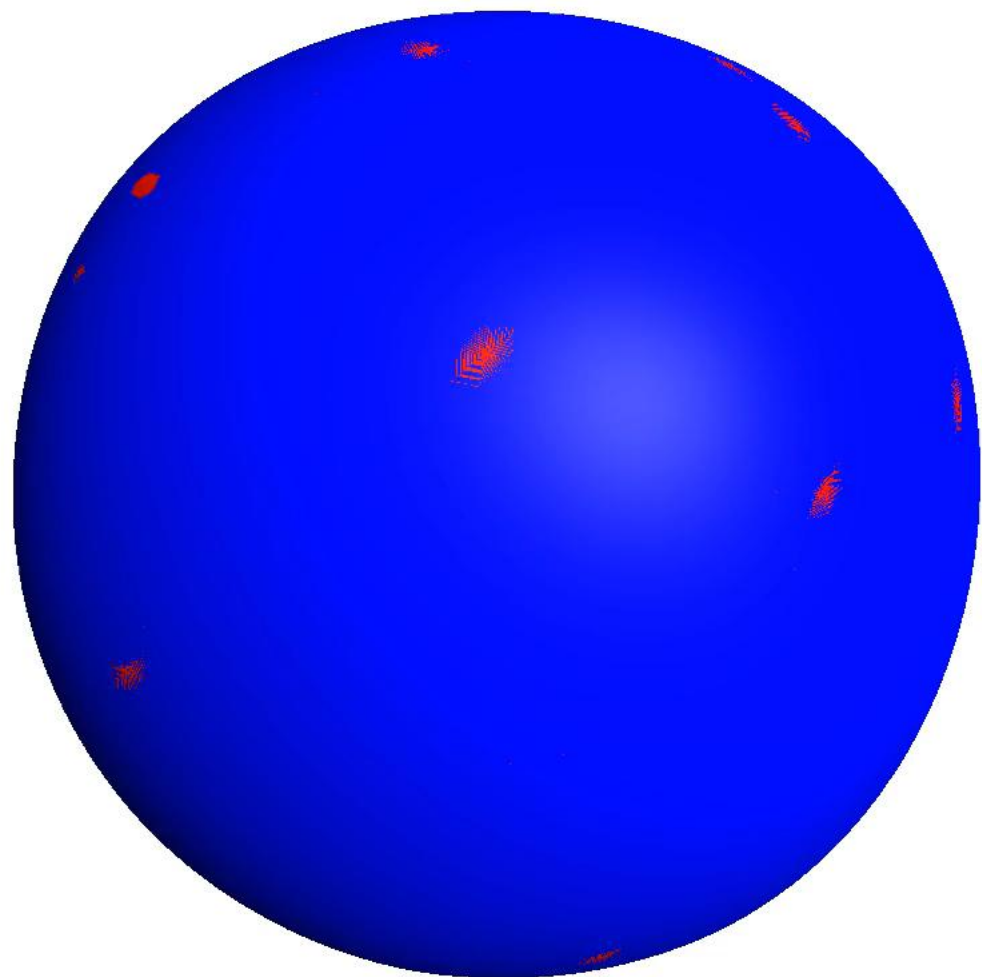
}
```

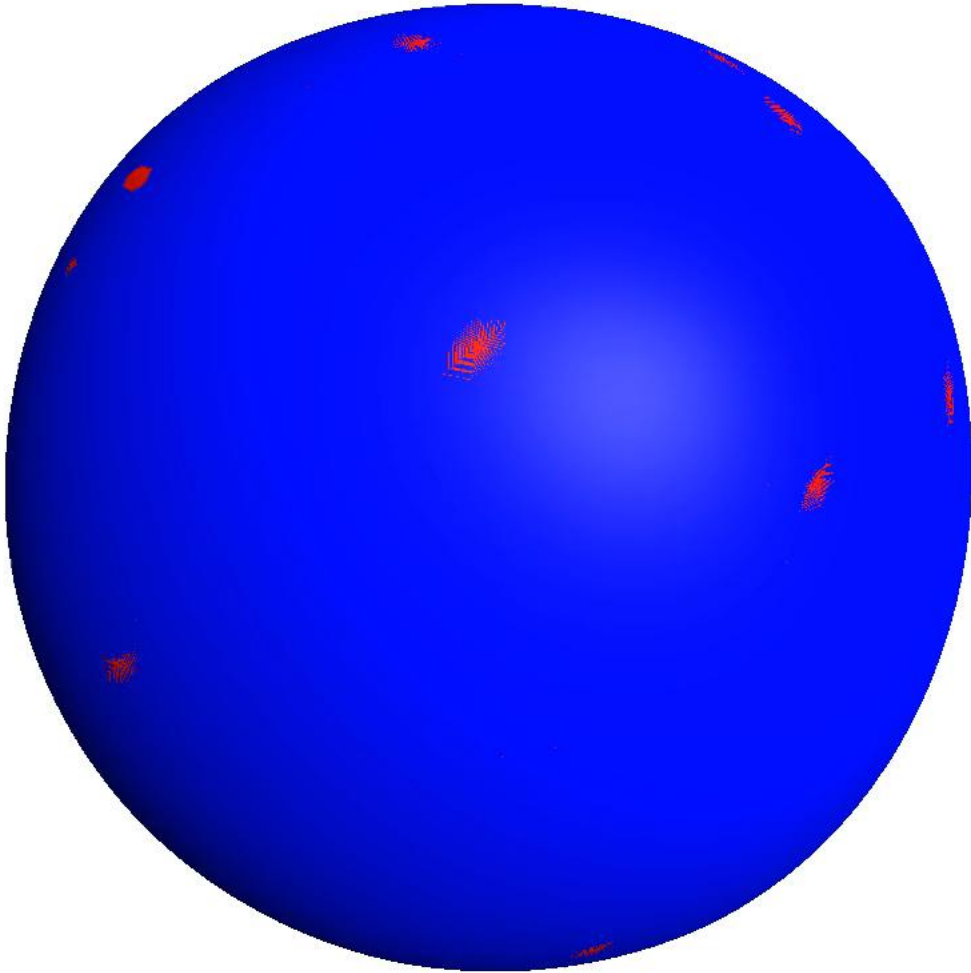
```
#include <iostream>
#include <bit>
int main() {
    float a = 0.1f;
    float b = 0.2f;
    float c = 1.0f;

}
```

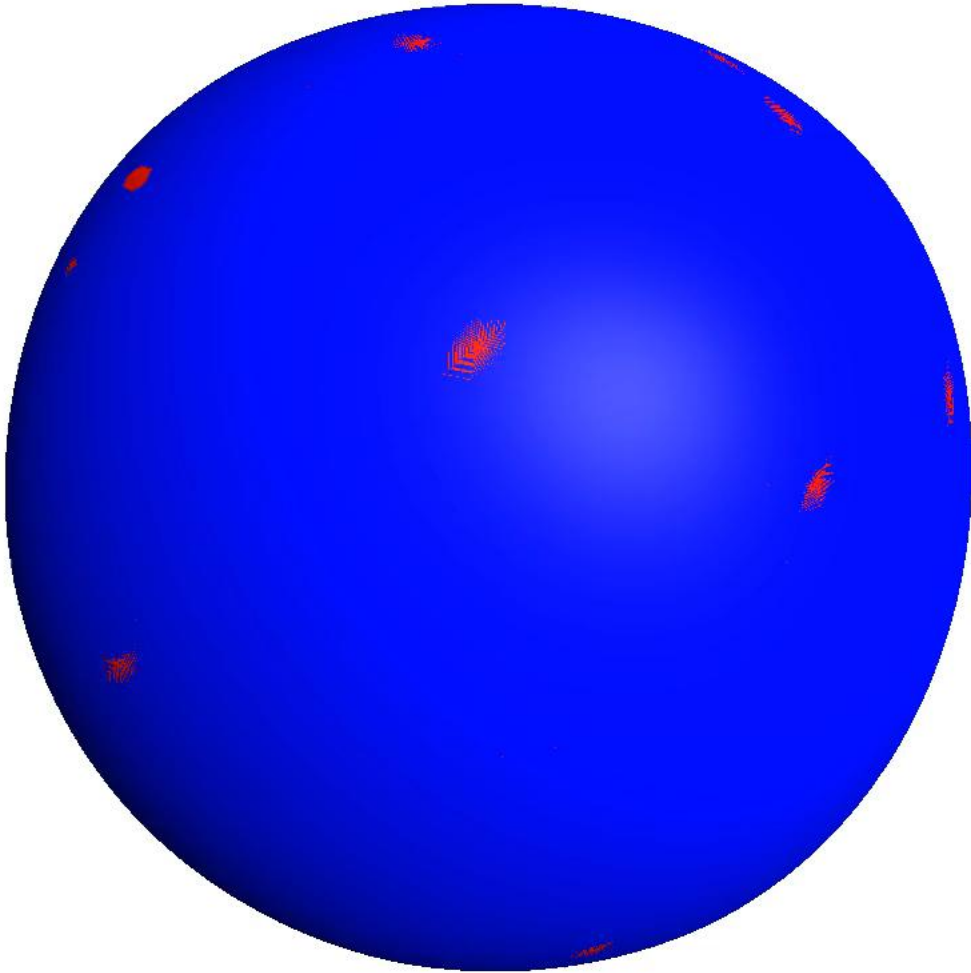
```
#include <iostream>
#include <bit>
int main() {
    float a = 0.1f;
    float b = 0.2f;
    float c = 1.0f;
    std::cout << std::bit_cast<unsigned>((a+b)+c) << '\n';
    std::cout << std::bit_cast<unsigned>(a+(b+c)) << '\n';
}
```

```
#include <iostream>
#include <bit>
int main() {
    float a = 0.1f;
    float b = 0.2f;
    float c = 1.0f;
    std::cout << std::bit_cast<unsigned>((a+b)+c) << '\n'; // 1067869798
    std::cout << std::bit_cast<unsigned>(a+(b+c)) << '\n'; // 1067869799
}
```



- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information
- ☐ merge to **single dispatch**

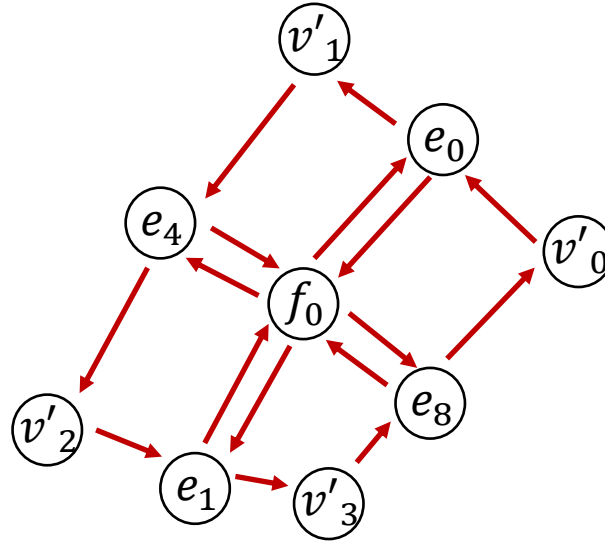


- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information
- ☐ merge to **single dispatch**
- ☐ remove all **atomic float** arithmetic

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

```

[DV21]

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

```

```

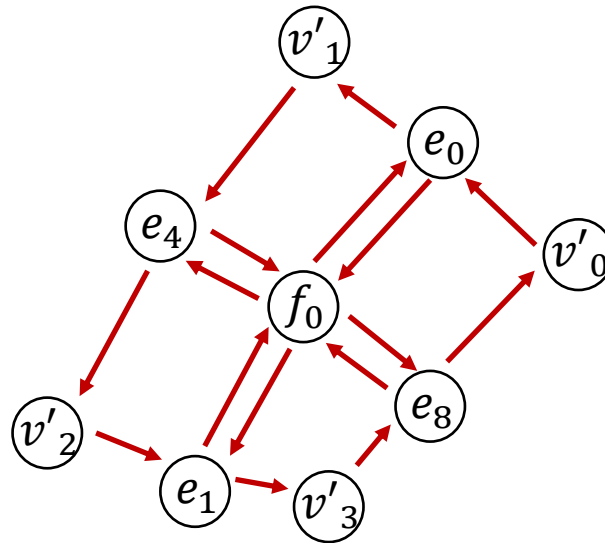
for all  $h \in [0, H_d)$  do
  // apply rule Fig. 3 (a):
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+0]) \leftarrow 4\text{NEXT}(\text{TWIN}(\mathcal{H}^d[h])) + 3$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+1]) \leftarrow 4\text{NEXT}(\mathcal{H}^d[h]) + 2$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+2]) \leftarrow 4\text{PREV}(\mathcal{H}^d[h]) + 1$ 
   $\text{TWIN}(\mathcal{H}^{d+1}[4h+3]) \leftarrow 4\text{TWIN}(\text{PREV}(\mathcal{H}^d[h])) + 0$ 
  // apply rules Fig. 3 (b, ..., f):
  ...
end for

```

```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

```

- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information
- ☐ merge to **single dispatch**
- ☐ remove all **atomic float** arithmetic

for

```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

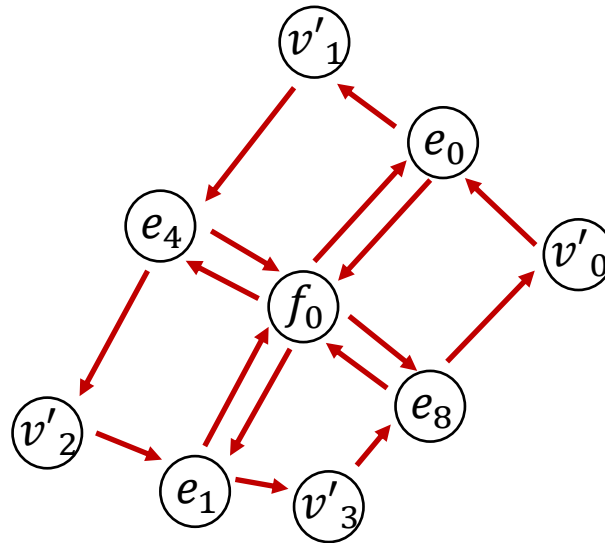
```

end for


```

1: procedure FACEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $m \leftarrow \text{CYCLELENGTH}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $\mathcal{V}^{d+1}[i] \leftarrow \mathcal{V}^{d+1}[i] + \frac{\mathcal{V}^d[v]}{m}$ 
7:   end for
8: end procedure

```



```

1: procedure EDGEPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $v \leftarrow \text{VERT}(h)$ 
4:      $i \leftarrow V_d + \text{FACE}(h)$ 
5:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
6:      $\mathcal{V}^{d+1}[j] \leftarrow \mathcal{V}^{d+1}[j] + \frac{\mathcal{V}^d[v] + \mathcal{V}^{d+1}[i]}{4}$ 
7:   end for
8: end procedure

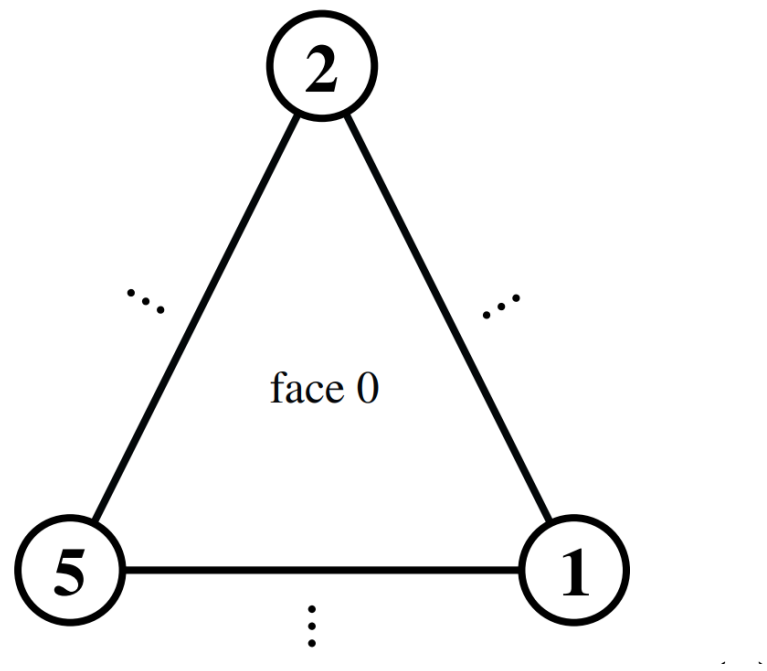
```

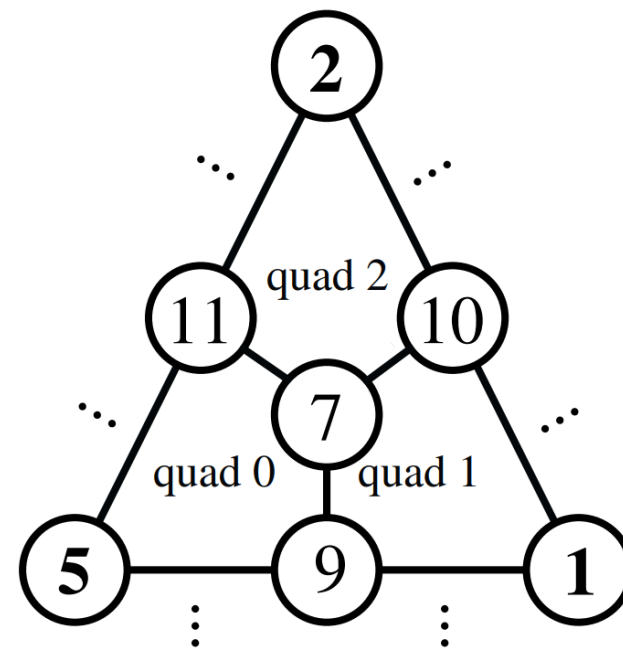
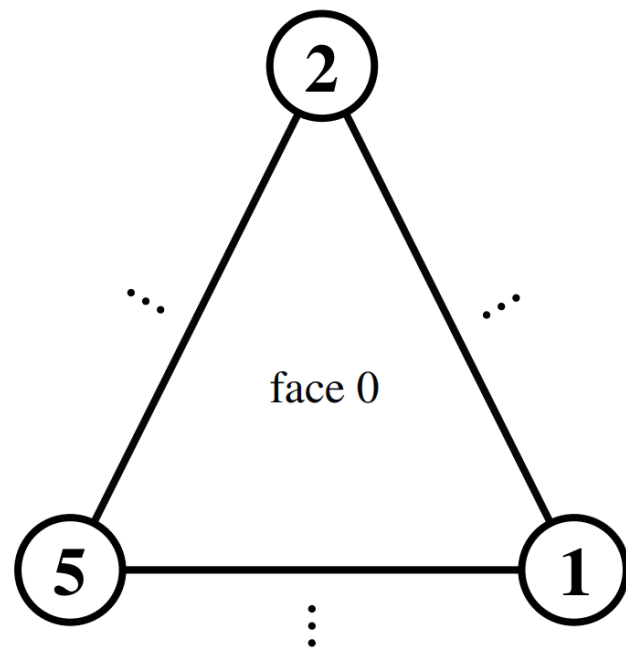
- ☐ **real-time** topology changes
- ☐ run **fast**
- ☐ **easy** to implement and extensible
- ☐ **reduce** neighbor information
- ☐ merge to **single dispatch**
- ☐ remove all **atomic float** arithmetic
- ☐ improve **memory layout**

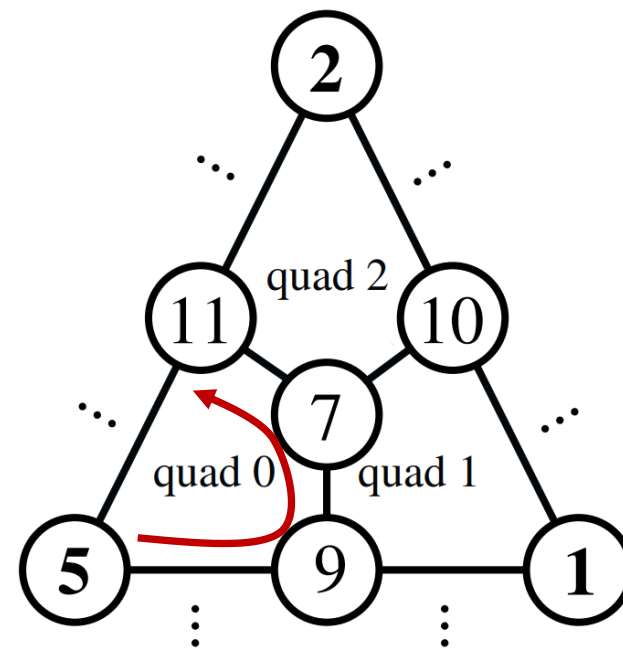
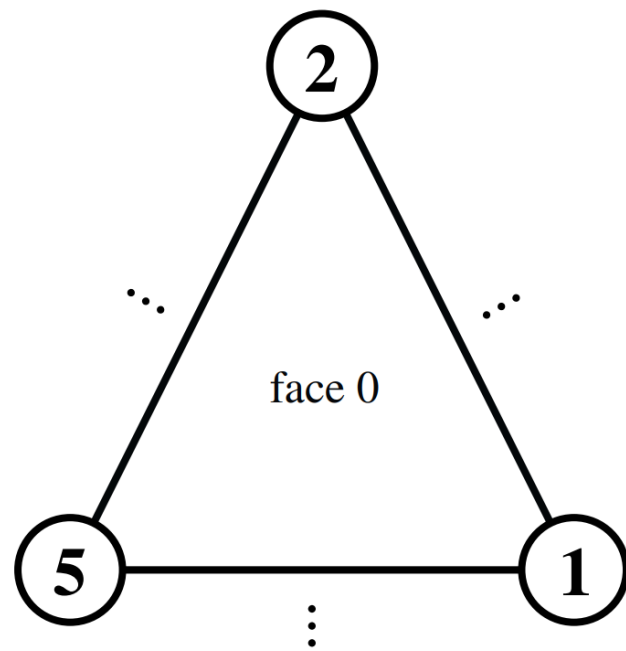
```

1: procedure VERTEXPOINTS( $\mathcal{S}^d$ : input mesh,  $\mathcal{V}^{d+1}$ : points)
2:   for all  $h \in [0, H_d)$  do
3:      $n \leftarrow \text{VALENCE}(\mathcal{S}^d, h)$ 
4:      $v \leftarrow \text{VERT}(h)$ 
5:      $i \leftarrow V_d + \text{FACE}(h)$ 
6:      $j \leftarrow V_d + F_d + \text{EDGE}(h)$ 
7:      $\mathcal{V}^{d+1}[v] \leftarrow \mathcal{V}^{d+1}[v] + \frac{4\mathcal{V}^{d+1}[j] - \mathcal{V}^{d+1}[i] + (n-3)\mathcal{V}^d[v]}{n^2}$ 
8:   end for
9: end procedure

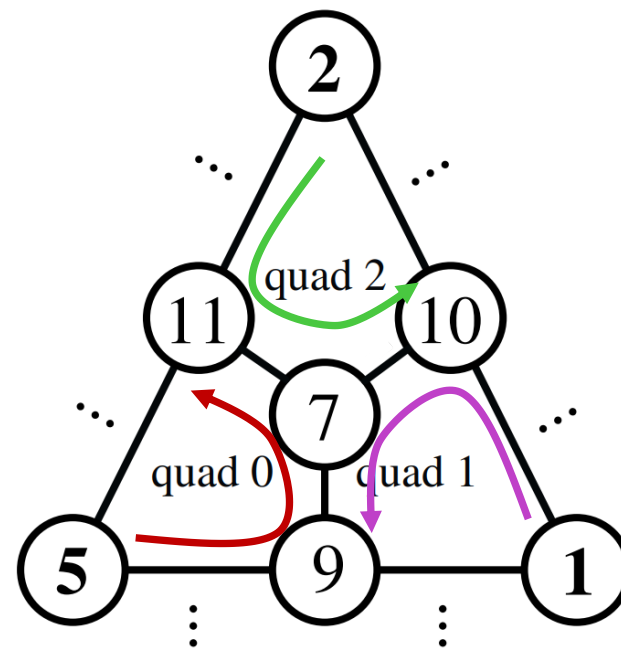
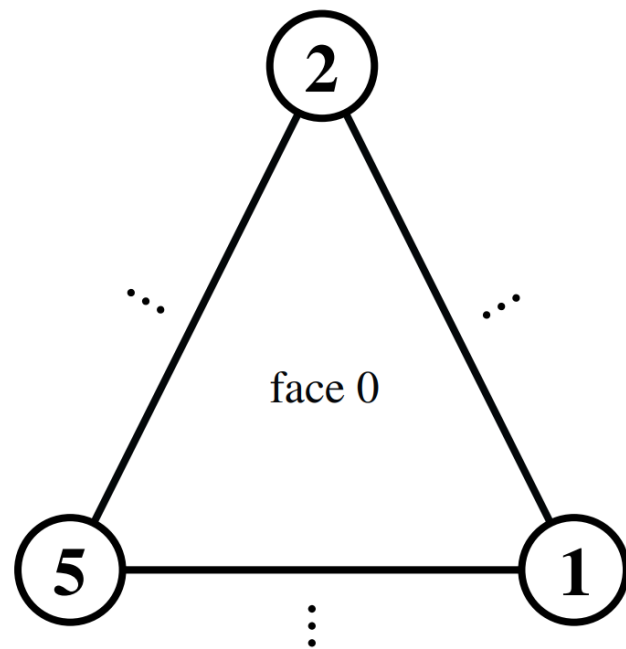
```



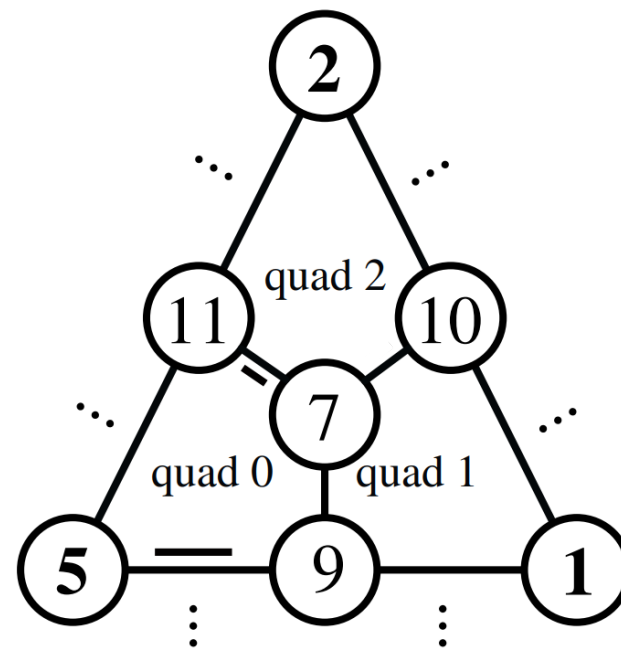
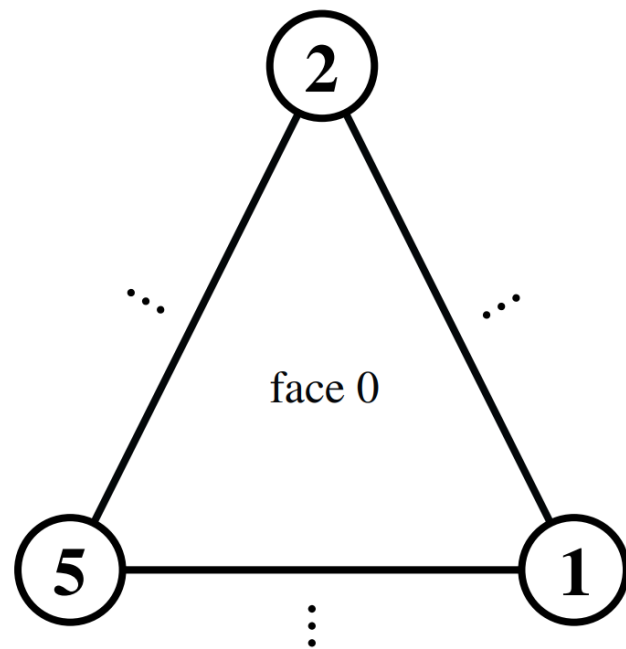




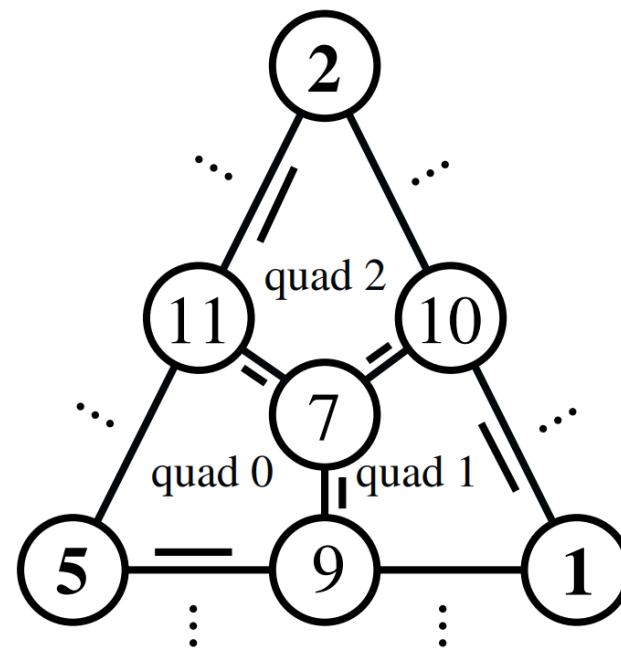
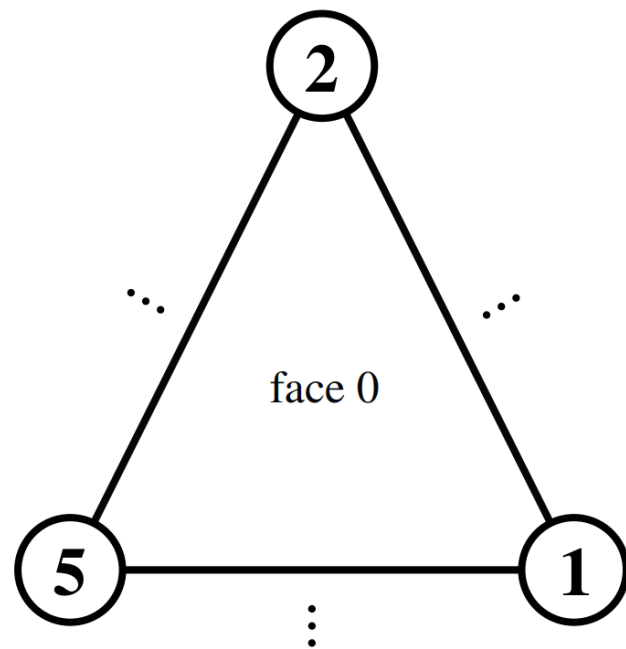
c	0	1	2	3
$\mathcal{I}_d$	5	9	7	11
quad 0				



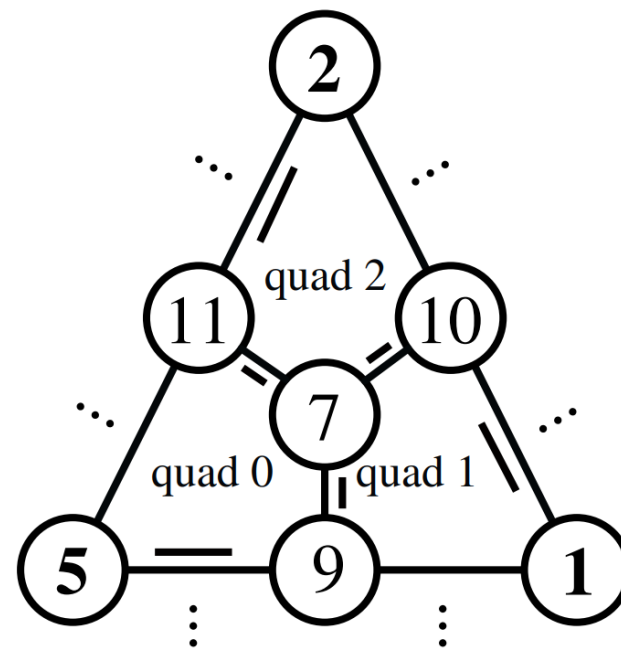
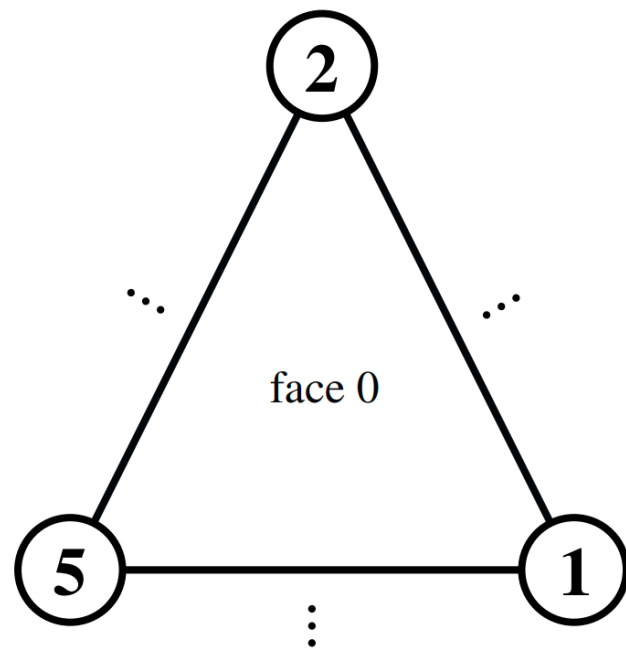
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
													...
	quad 0				quad 1				quad 2				...



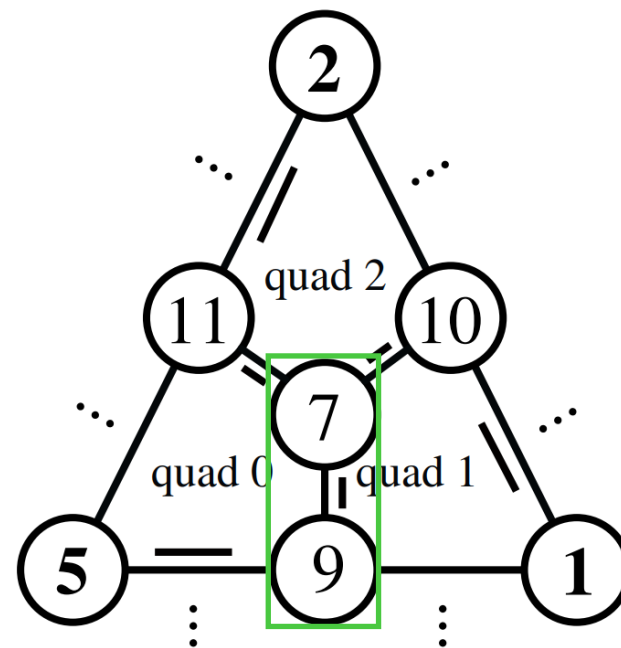
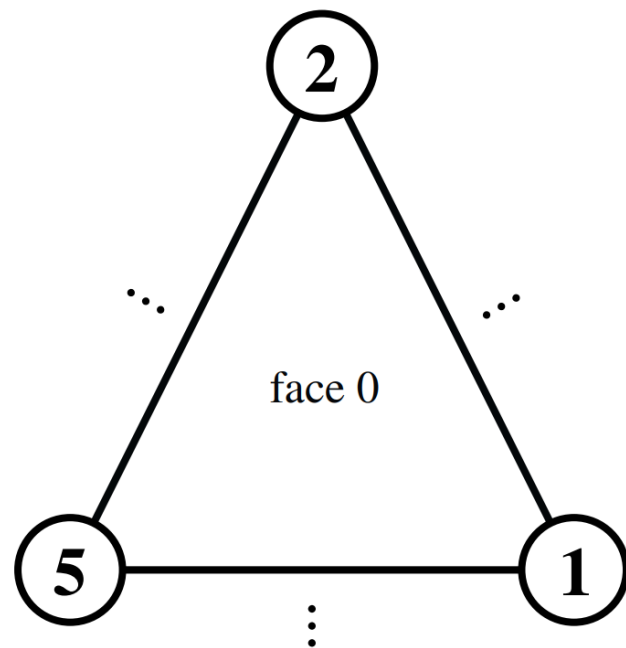
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
	edge 0		edge 1										...
	quad 0				quad 1				quad 2				



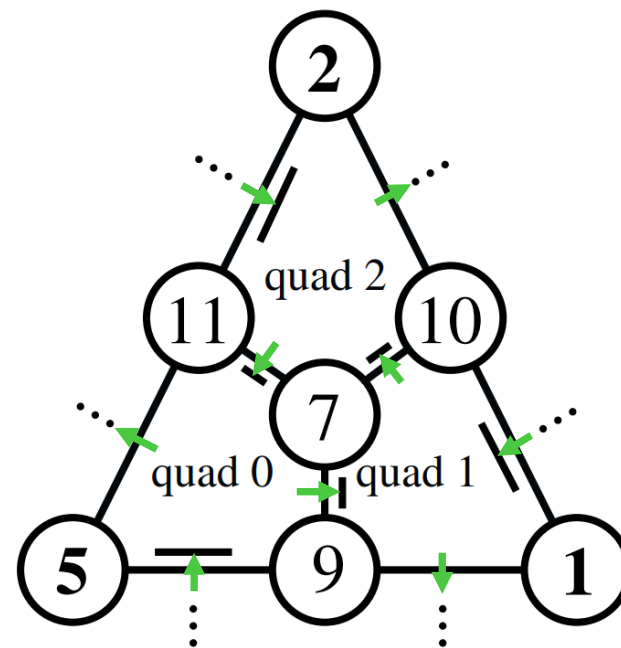
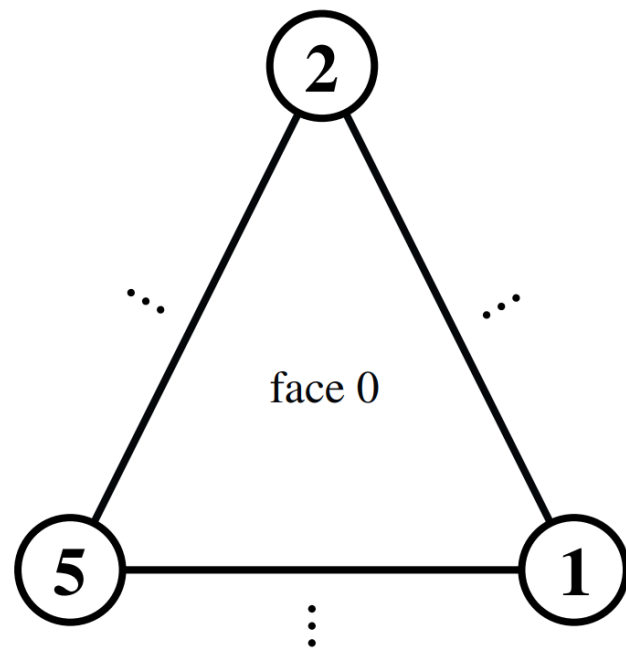
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				



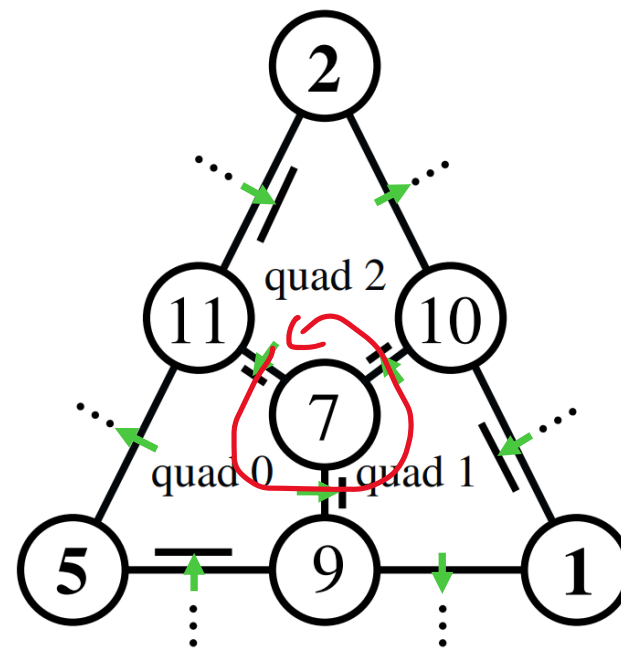
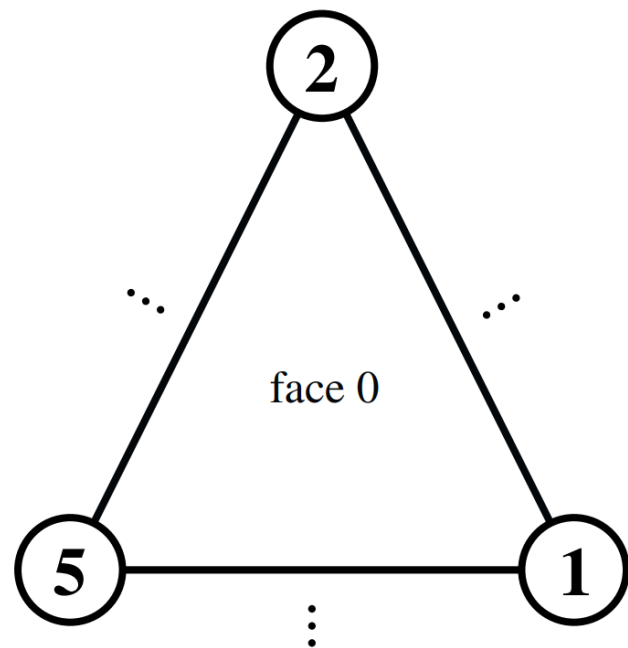
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				
$\mathcal{G}_d$	edge 3		...		edge 5		...		edge 1		...		...



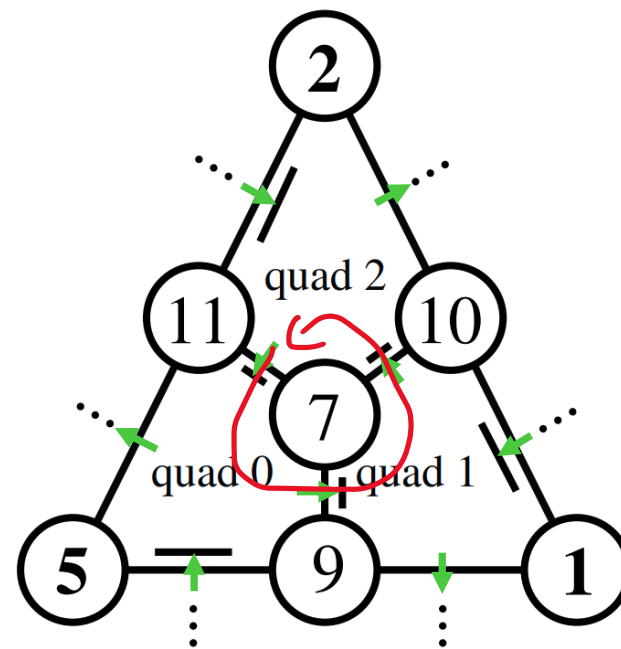
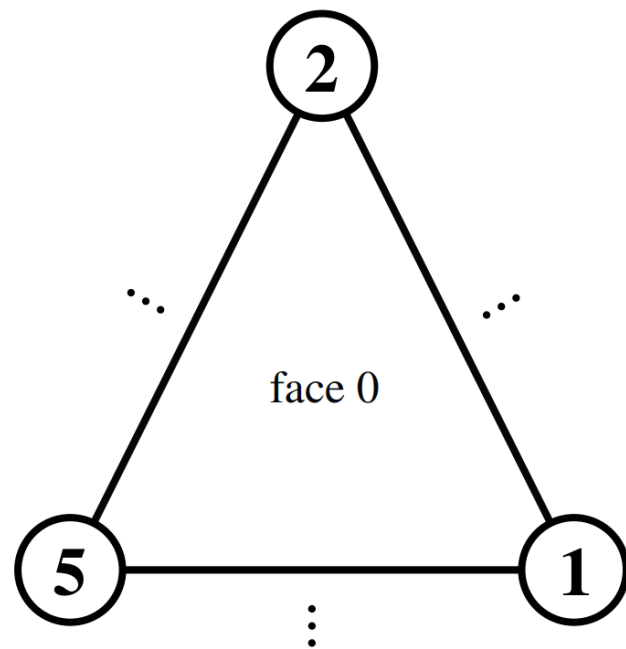
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				
$\mathcal{G}_d$	edge 3		...		edge 5		...		edge 1		...		...



c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				
$\mathcal{G}_d$	edge 3		...		edge 5		...		edge 1		...		...

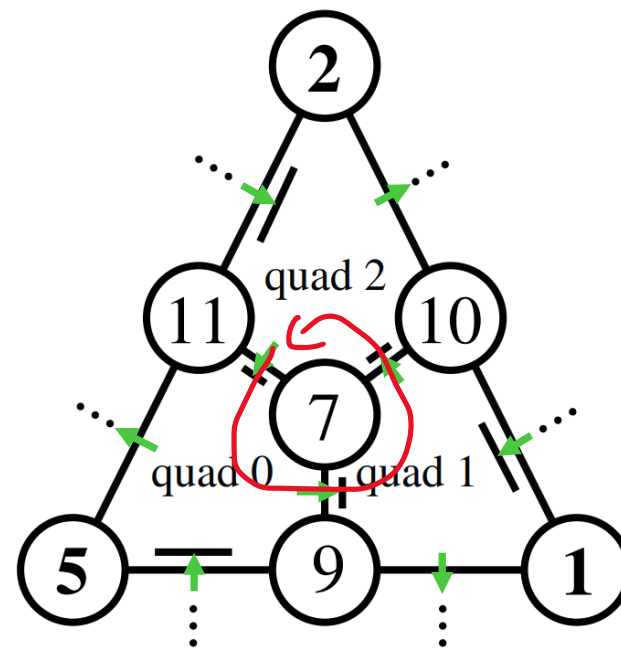
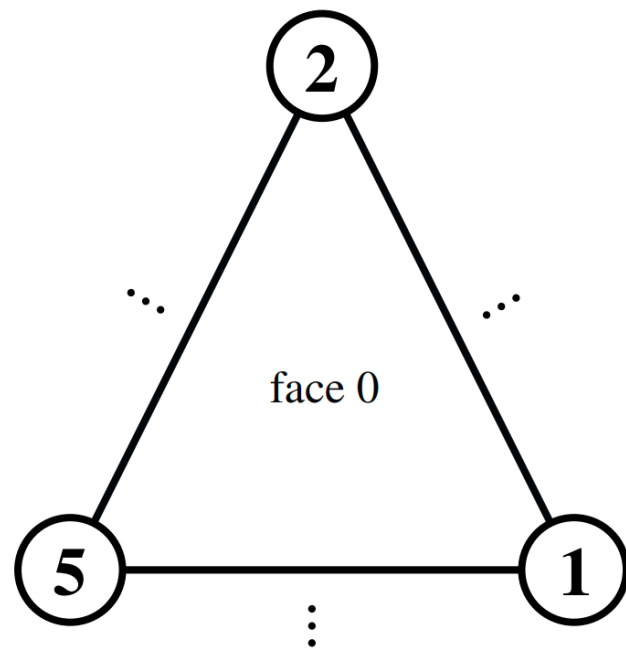


c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	



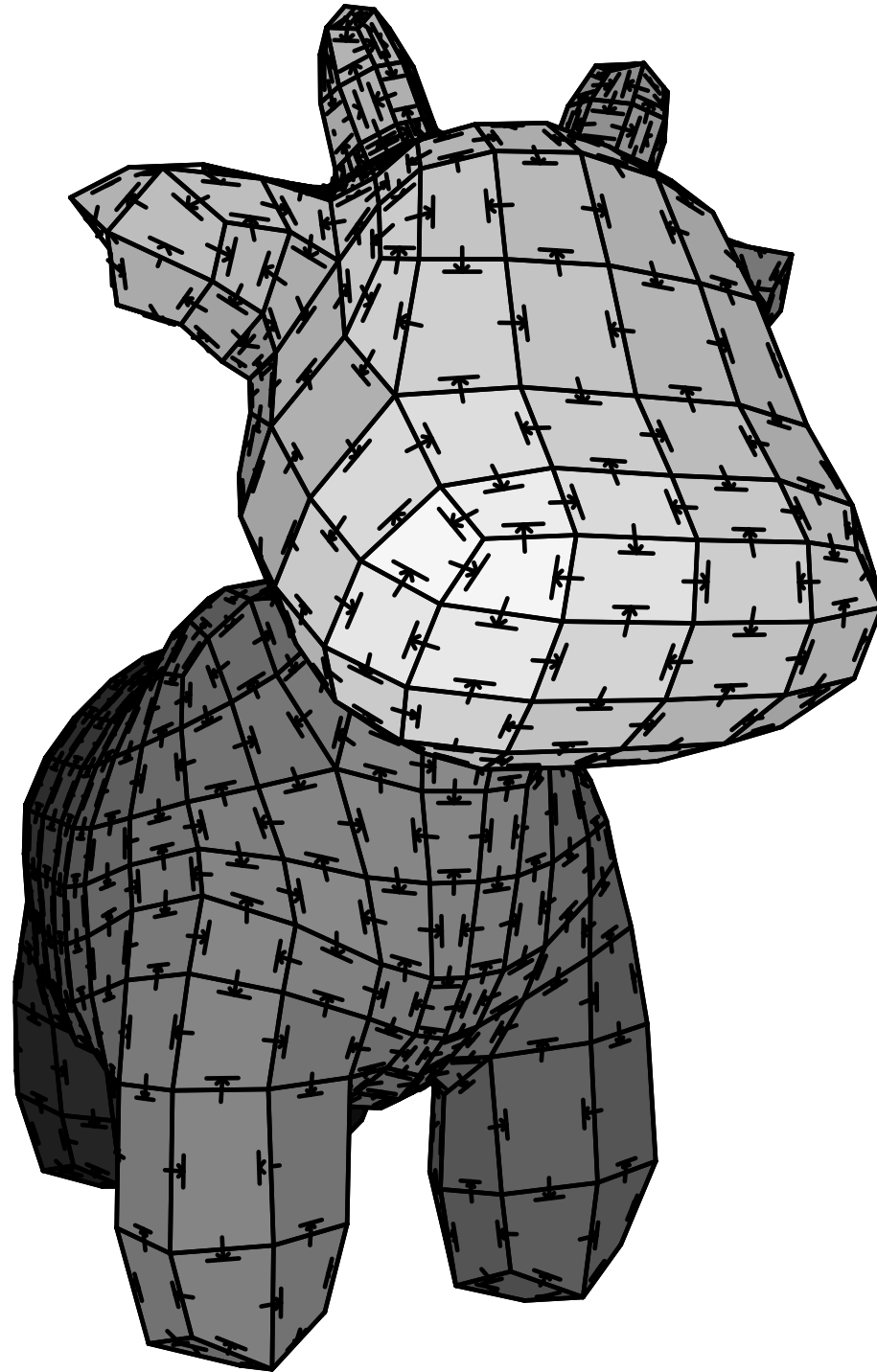
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	

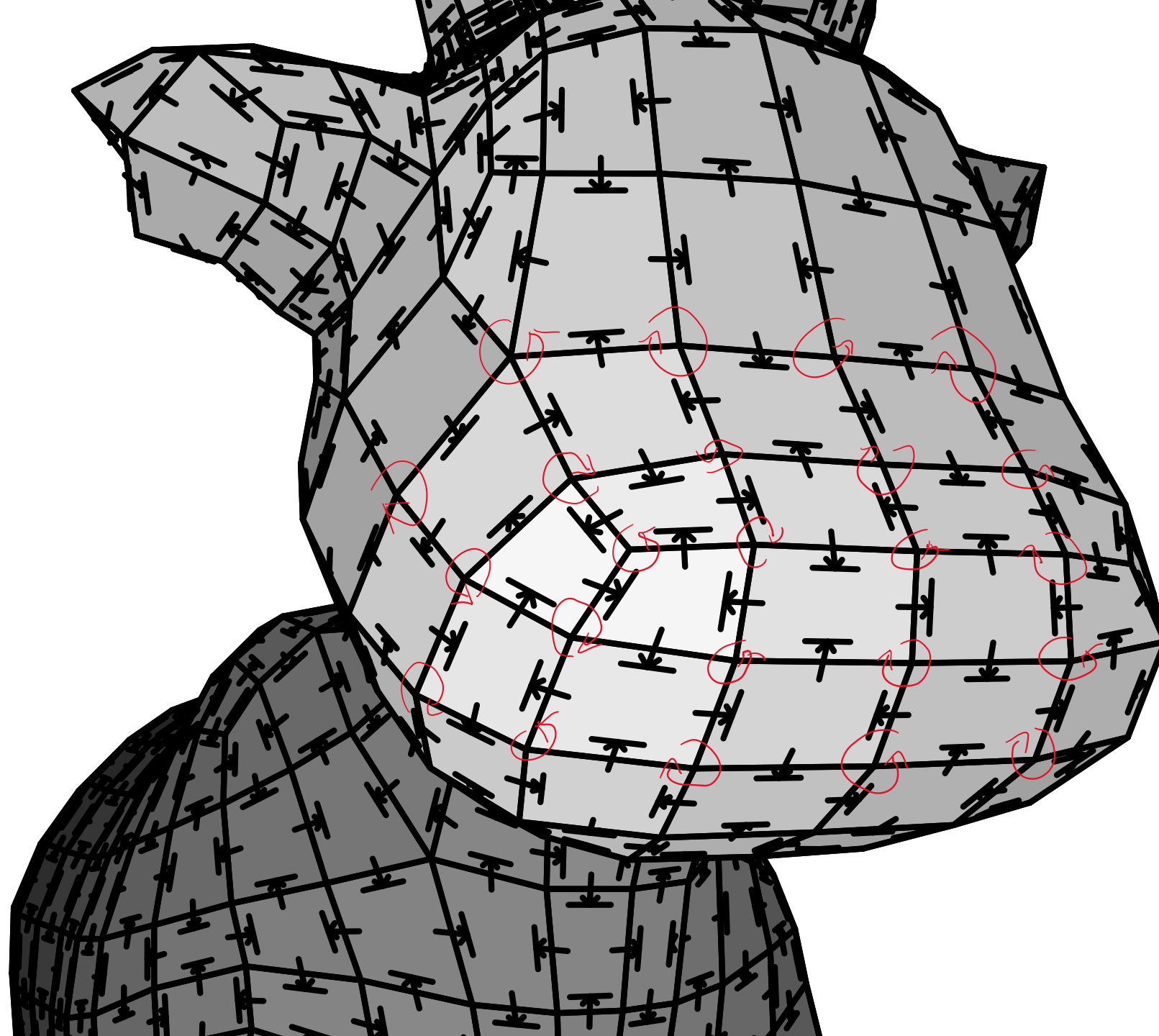
	...	1	2	...	5	...	7	...	9	10	11	...
$\mathcal{L}_d$		4	8		0		2		1	5	3	

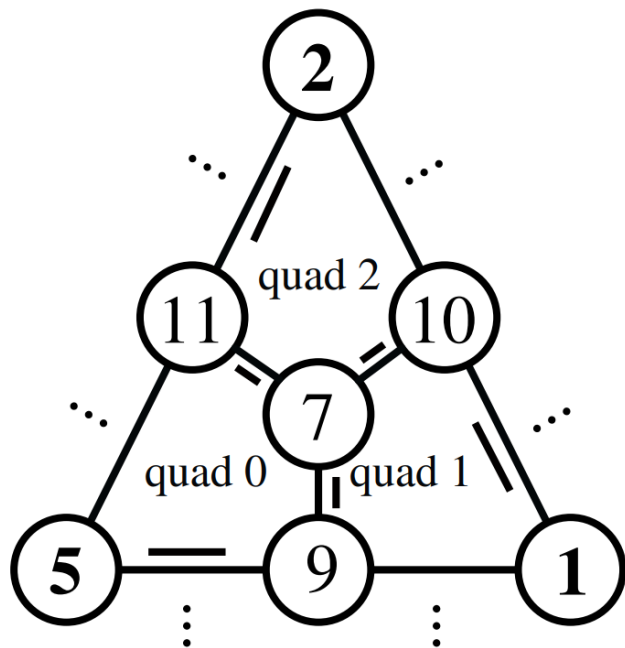


c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	

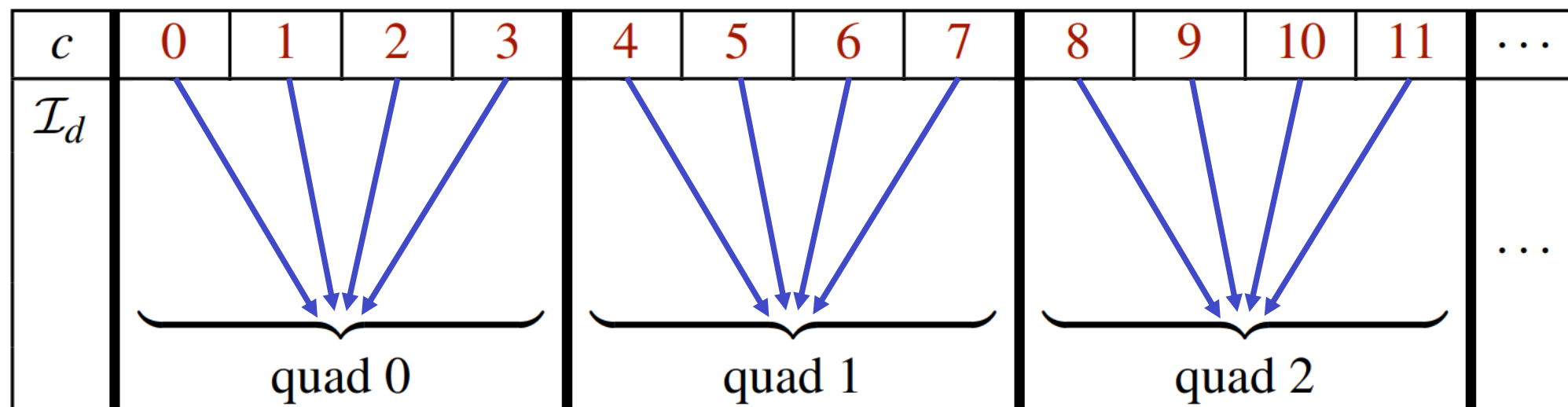
	...	1	2	...	5	...	7	...	9	10	11	...
$\mathcal{L}_d$		4	8		0		2		1	5	3	

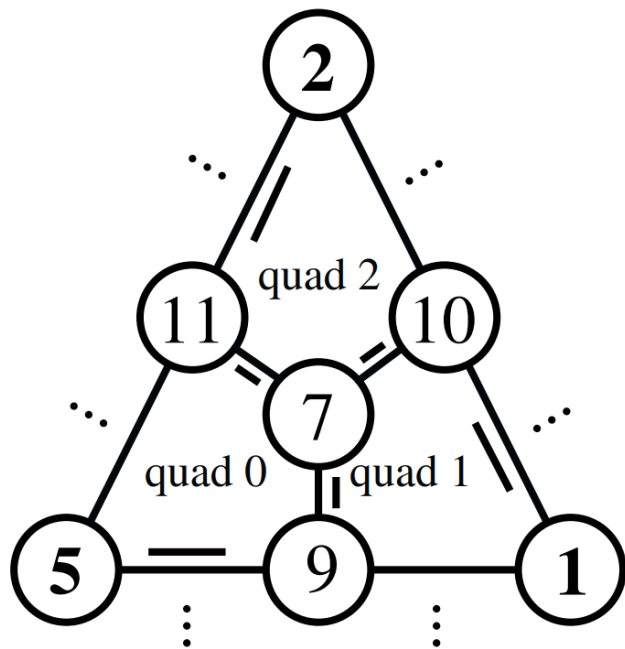






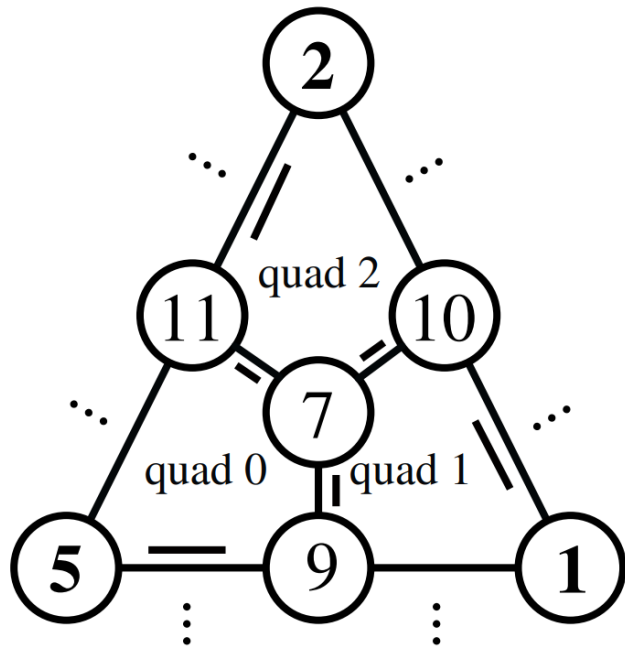
- $\text{CQUAD}(c) = \lfloor \frac{c}{4} \rfloor$ yields the quad index of a corner,





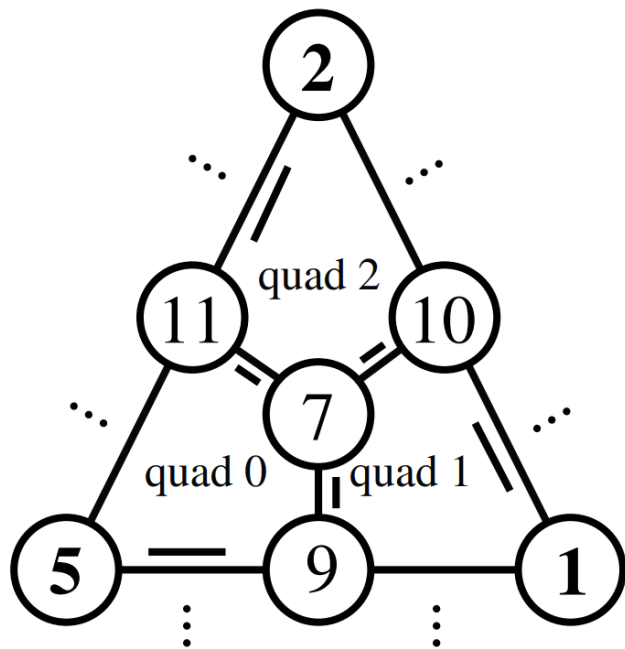
- $\text{CQUAD}(c) = \left\lfloor \frac{c}{4} \right\rfloor$ yields the quad index of a corner,
- $\text{EQUAD}(e) = \left\lfloor \frac{e}{2} \right\rfloor$ yields the quad index of an edge,

c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				



- $\text{CQUAD}(c) = \left\lfloor \frac{c}{4} \right\rfloor$ yields the quad index of a corner,
- $\text{EQUAD}(e) = \left\lfloor \frac{e}{2} \right\rfloor$ yields the quad index of an edge,
- $\text{DIAG}(c) = c \oplus 2$ yields the corner index across the quad, and

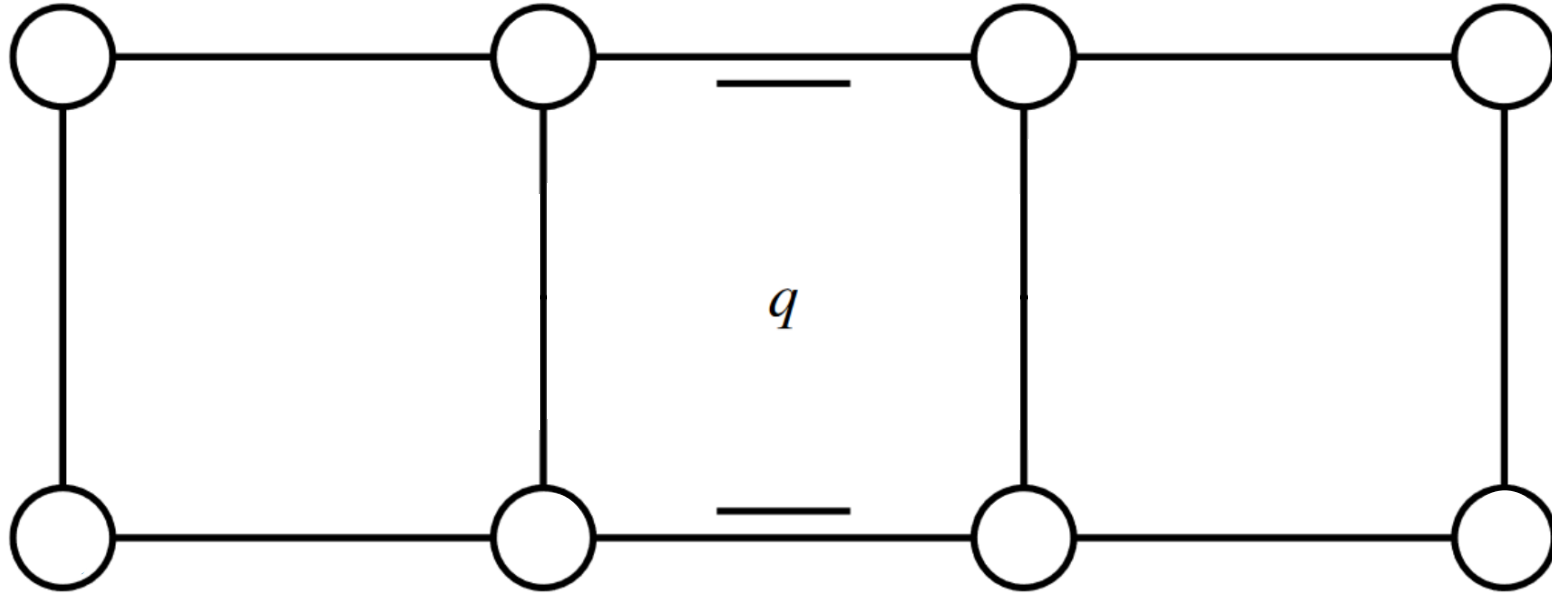
c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				



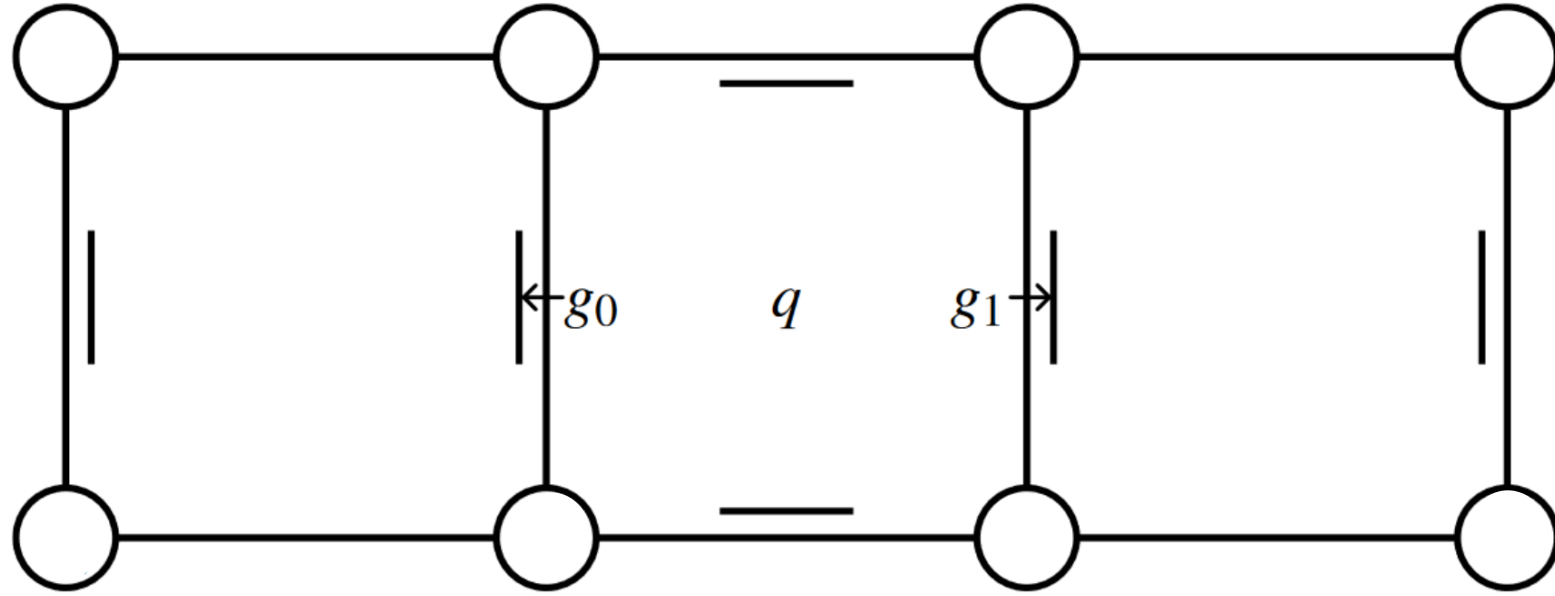
- $\text{CQUAD}(c) = \left\lfloor \frac{c}{4} \right\rfloor$ yields the quad index of a corner,
- $\text{EQUAD}(e) = \left\lfloor \frac{e}{2} \right\rfloor$ yields the quad index of an edge,
- $\text{DIAG}(c) = c \oplus 2$ yields the corner index across the quad, and
- $\text{OFF}(c) = c \oplus 3$ yields the corner index along the off-edge.

c	0	1	2	3	4	5	6	7	8	9	10	11	...
$\mathcal{I}_d$	5	9	7	11	1	10	7	9	2	11	7	10	...
	edge 0		edge 1		edge 2		edge 3		edge 4		edge 5		...
	quad 0				quad 1				quad 2				

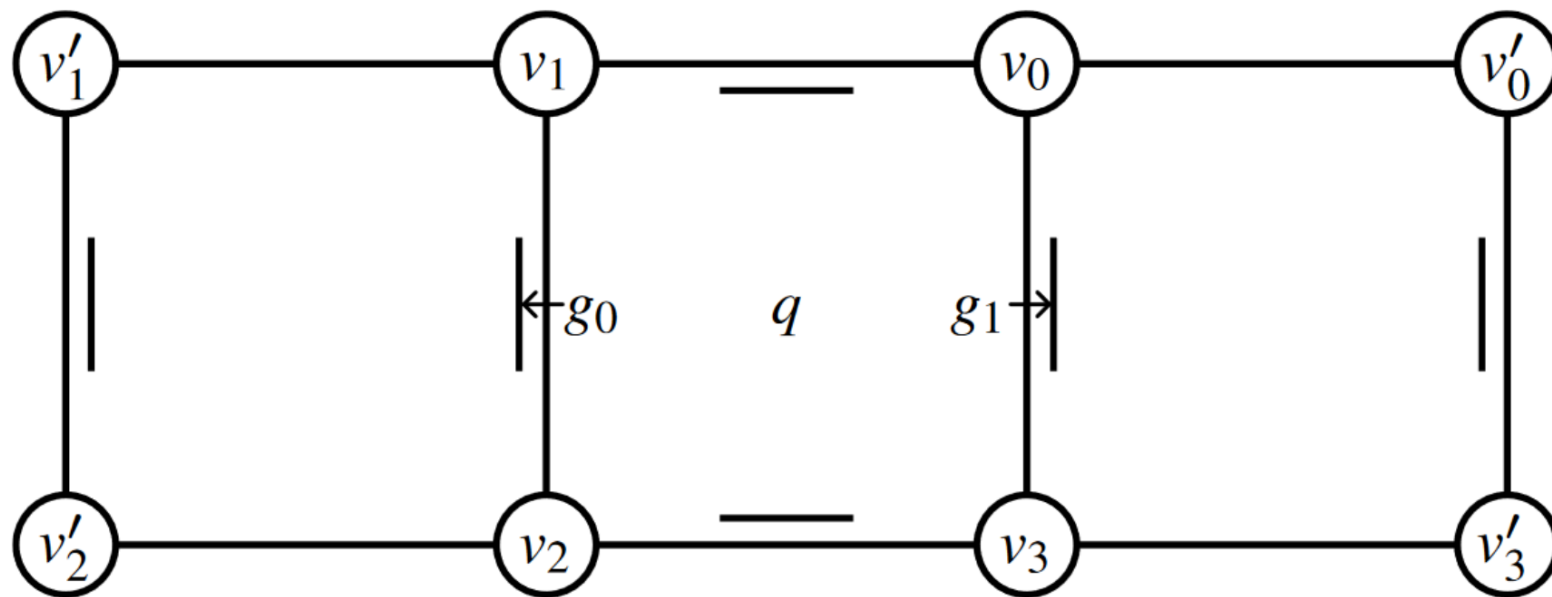
Quad-Task



Quad-Task



Quad-Task



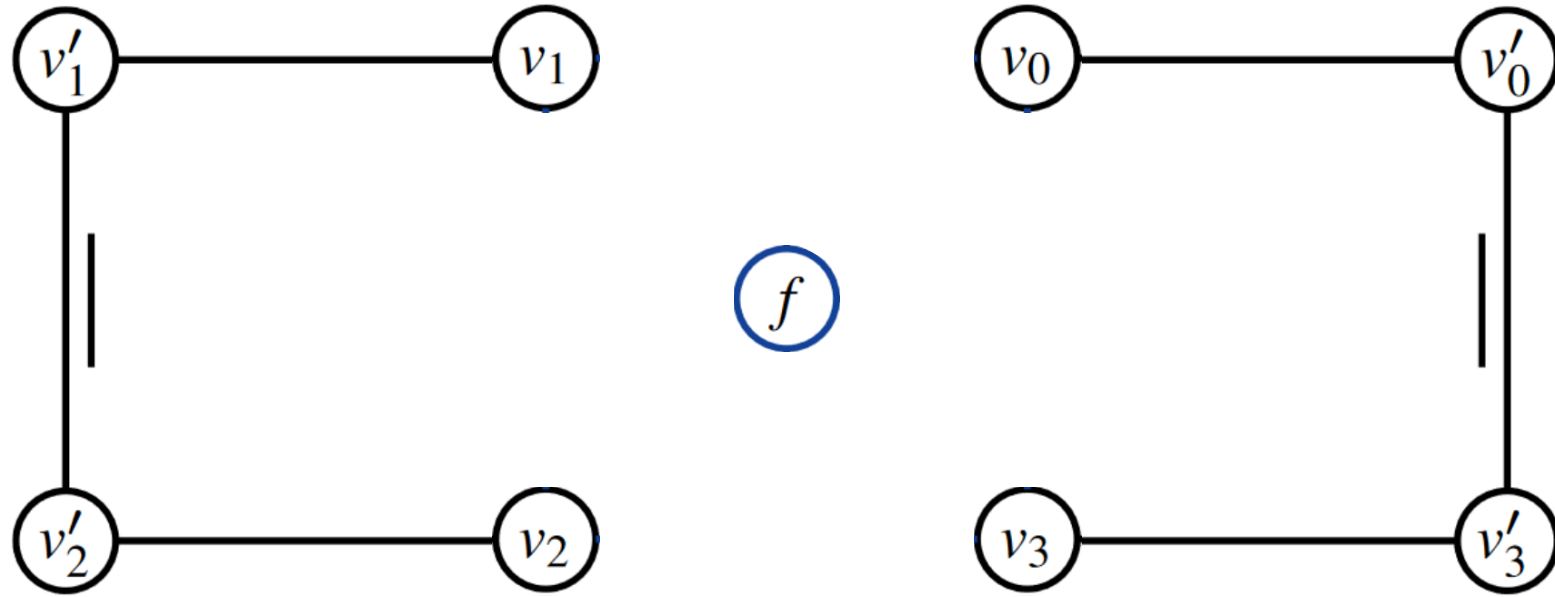
$$v_0 = \mathcal{I}_d[2g_1 + 0], \quad v'_0 = \mathcal{I}_d[\text{OFF}(2g_1 + 0)],$$

$$v_1 = \mathcal{I}_d[2g_0 + 1], \quad v'_1 = \mathcal{I}_d[\text{OFF}(2g_0 + 1)],$$

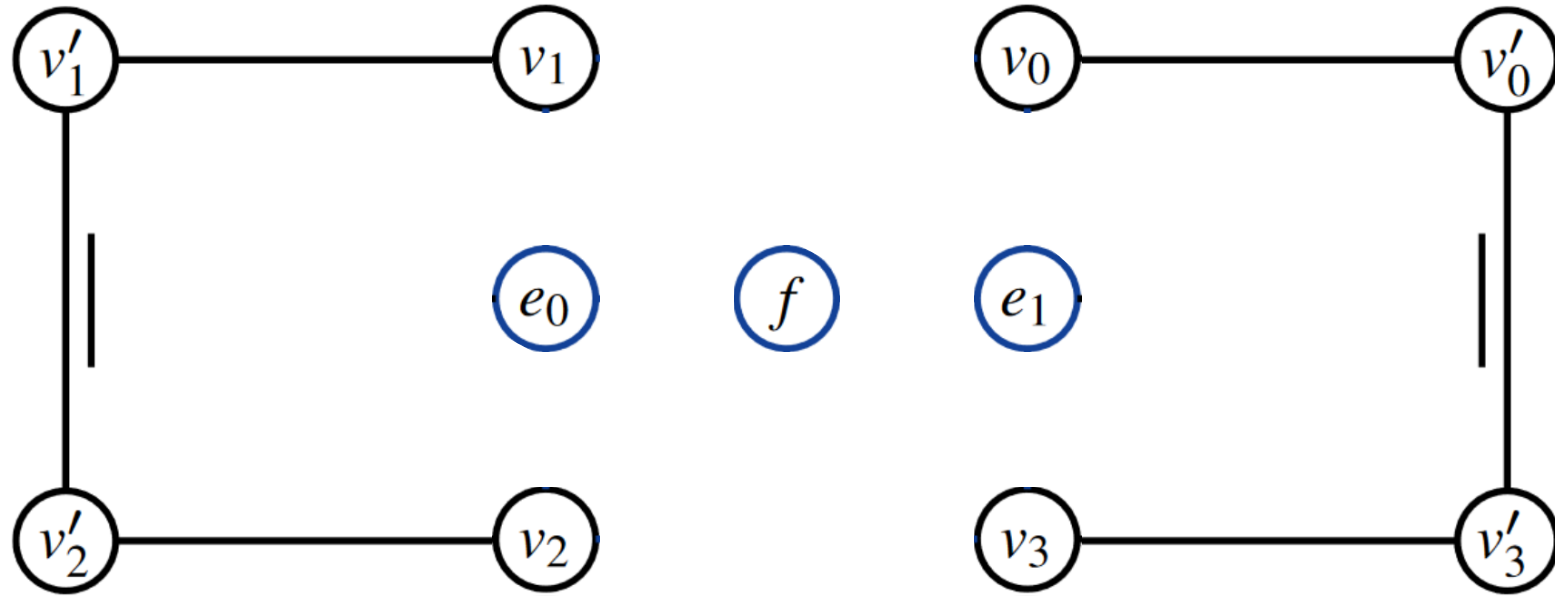
$$v_2 = \mathcal{I}_d[2g_0 + 0], \quad v'_2 = \mathcal{I}_d[\text{OFF}(2g_0 + 0)],$$

$$v_3 = \mathcal{I}_d[2g_1 + 1], \quad v'_3 = \mathcal{I}_d[\text{OFF}(2g_1 + 1)].$$

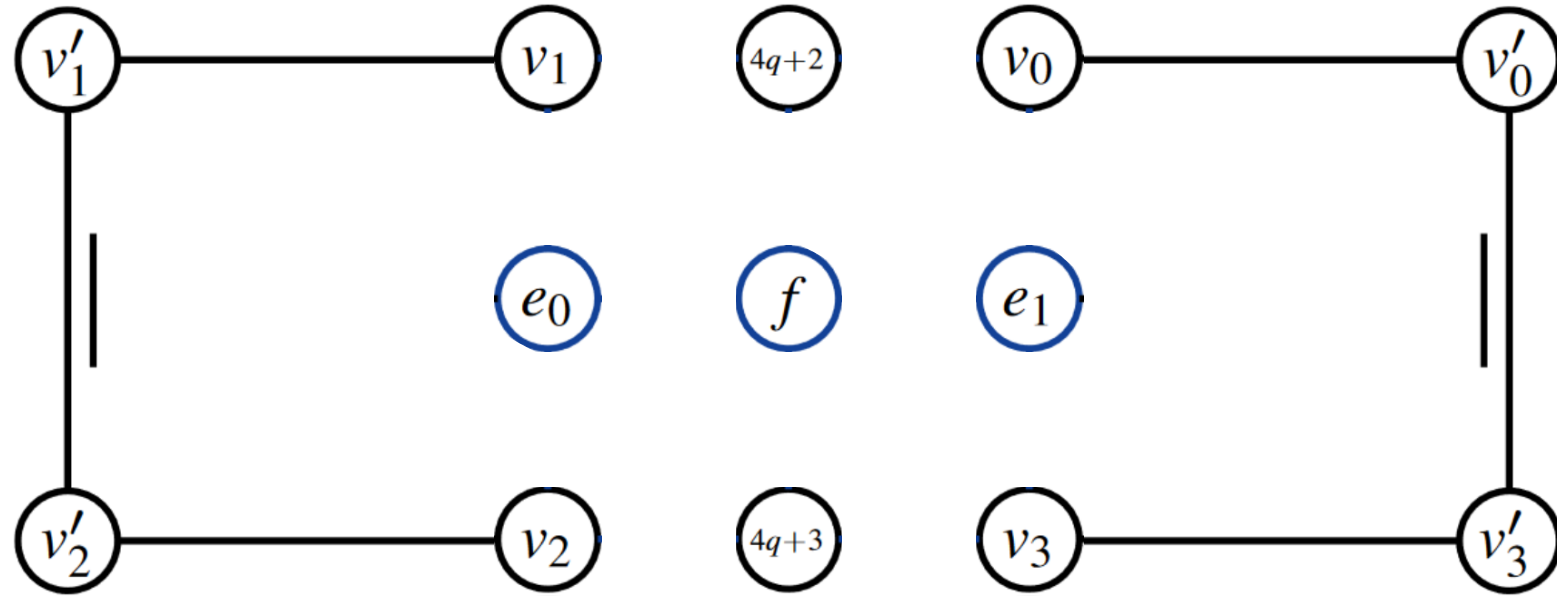
Quad-Task



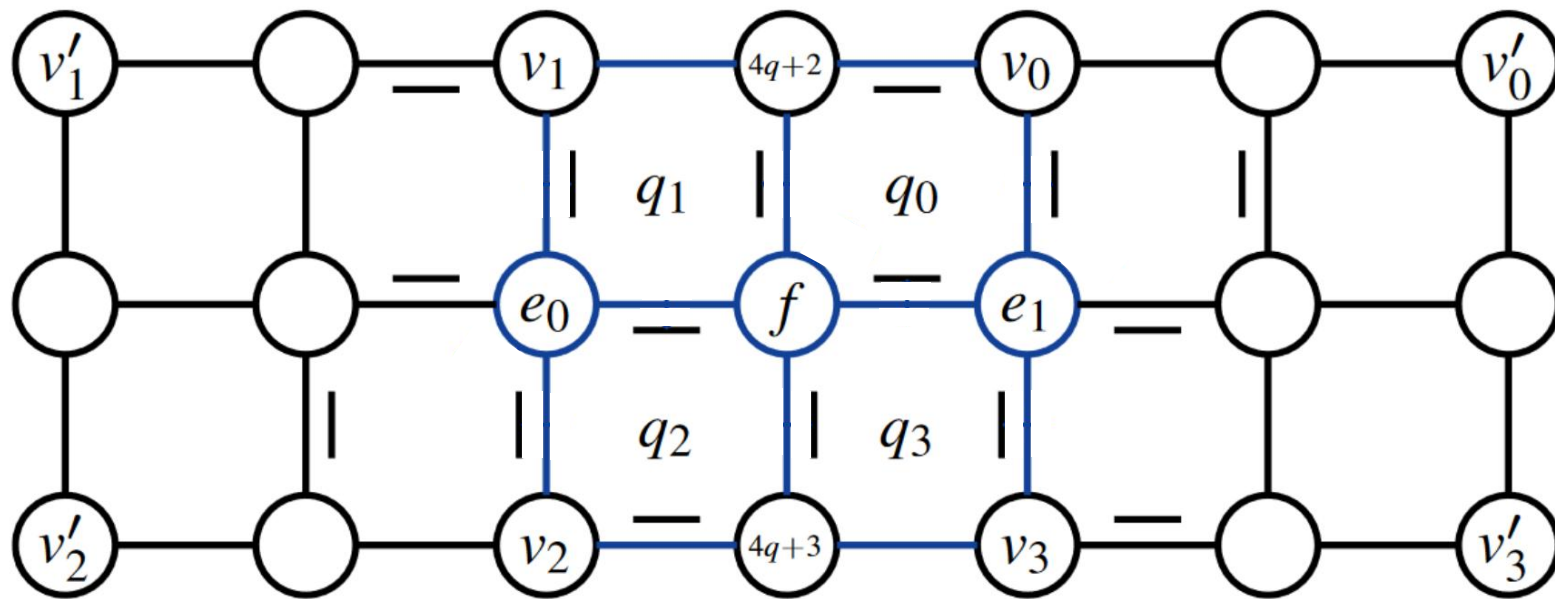
Quad-Task



Quad-Task



Quad-Task



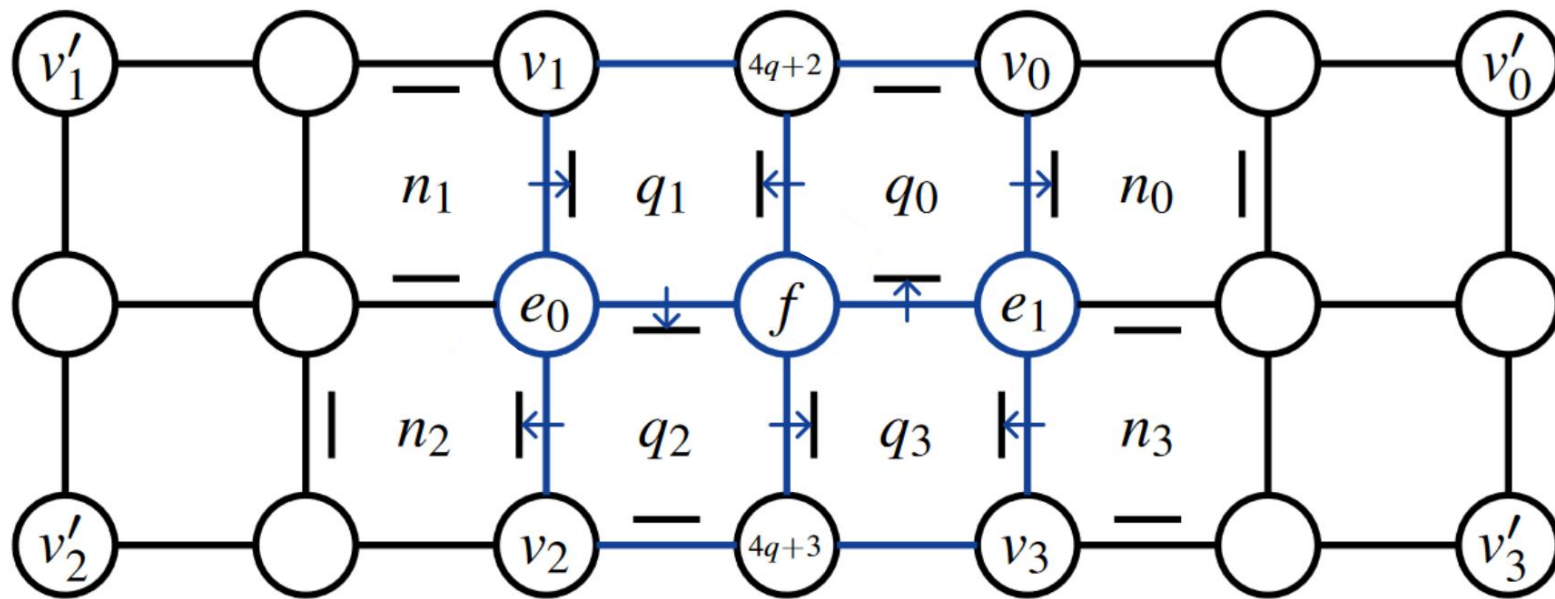
$$\mathcal{I}_{d+1}[4q_0 : 4q_1] \leftarrow (w_0, 4q + 2, f, e_1),$$

$$\mathcal{I}_{d+1}[4q_1 : 4q_2] \leftarrow (w_1, e_0, f, 4q + 2),$$

$$\mathcal{I}_{d+1}[4q_2 : 4q_3] \leftarrow (w_2, 4q + 3, f, e_0),$$

$$\mathcal{I}_{d+1}[4q_3 : 4q_3 + 4] \leftarrow (w_3, e_1, f, 4q + 3),$$

Quad-Task



$$n_0 = 4\text{EQUAD}(g_1) + 2(g_1 \bmod 2) + 0,$$

$$n_1 = 4\text{EQUAD}(g_0) + 2(g_0 \bmod 2) + 1,$$

$$n_2 = 4\text{EQUAD}(g_0) + 2(g_0 \bmod 2) + 0,$$

$$n_3 = 4\text{EQUAD}(g_1) + 2(g_1 \bmod 2) + 1,$$

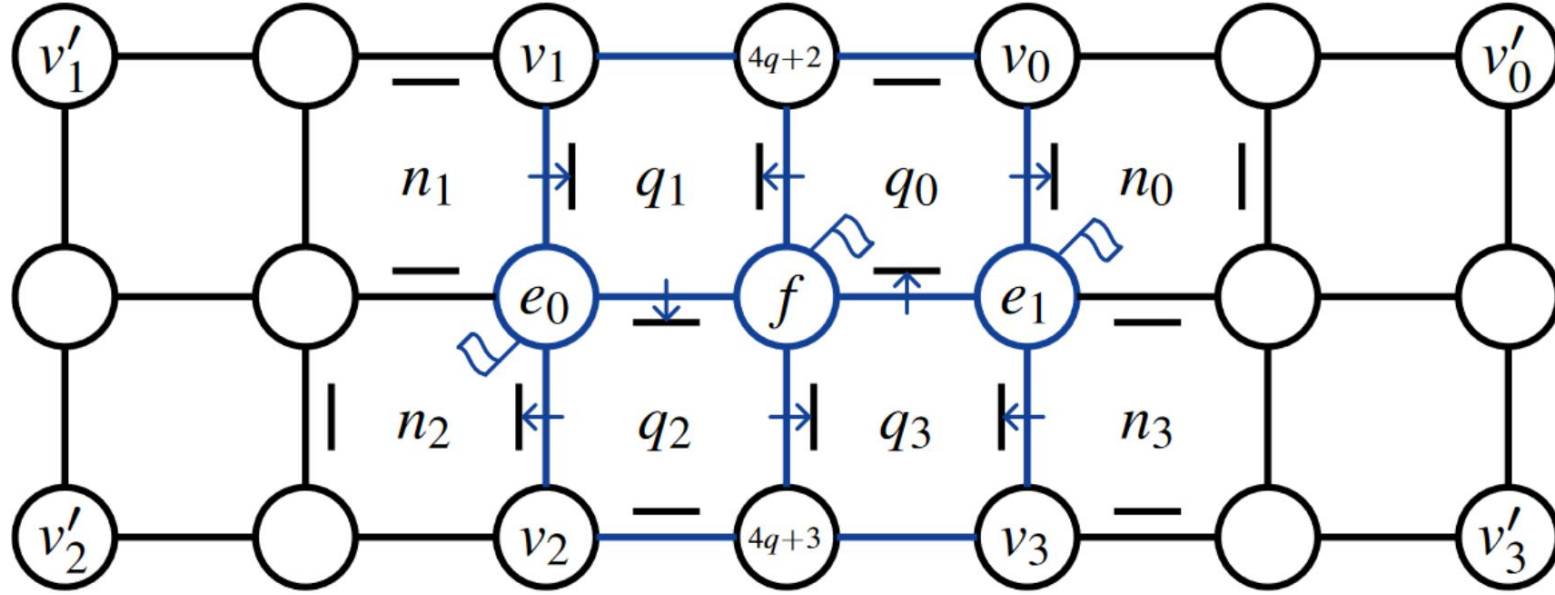
$$\mathcal{G}_{d+1}[q_0]_0 \leftarrow 2q_1 + 1, \quad \mathcal{G}_{d+1}[q_0]_1 \leftarrow 2n_0 + 0,$$

$$\mathcal{G}_{d+1}[q_1]_0 \leftarrow 2q_2 + 1, \quad \mathcal{G}_{d+1}[n_1]_1 \leftarrow 2q_1 + 0,$$

$$\mathcal{G}_{d+1}[q_2]_0 \leftarrow 2q_3 + 1, \quad \mathcal{G}_{d+1}[q_2]_1 \leftarrow 2n_2 + 0,$$

$$\mathcal{G}_{d+1}[q_3]_0 \leftarrow 2q_0 + 1, \quad \mathcal{G}_{d+1}[n_3]_1 \leftarrow 2q_3 + 0.$$

Quad-Task

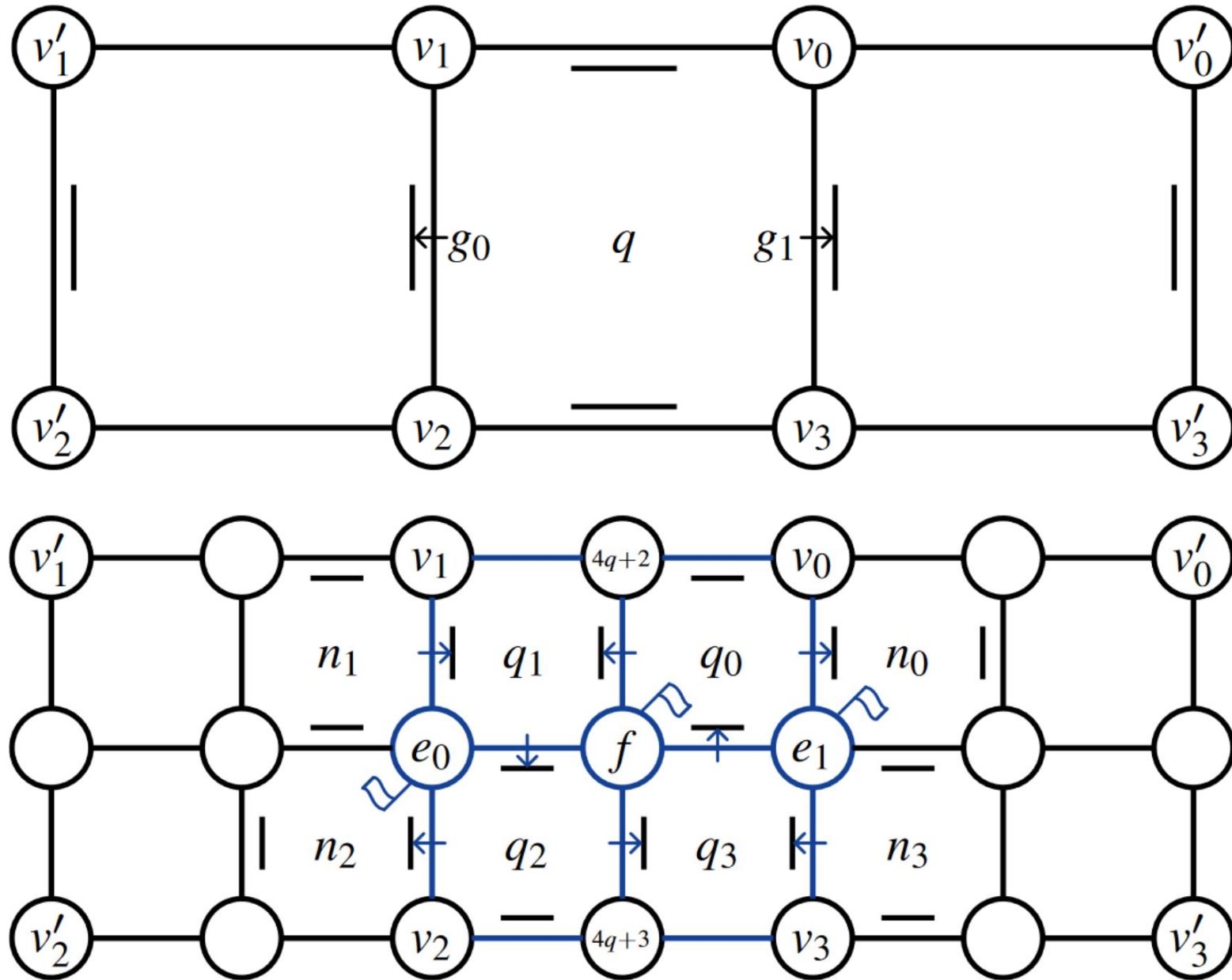


$$\mathcal{L}_{d+1}[f] \leftarrow 4q_0 + 2,$$

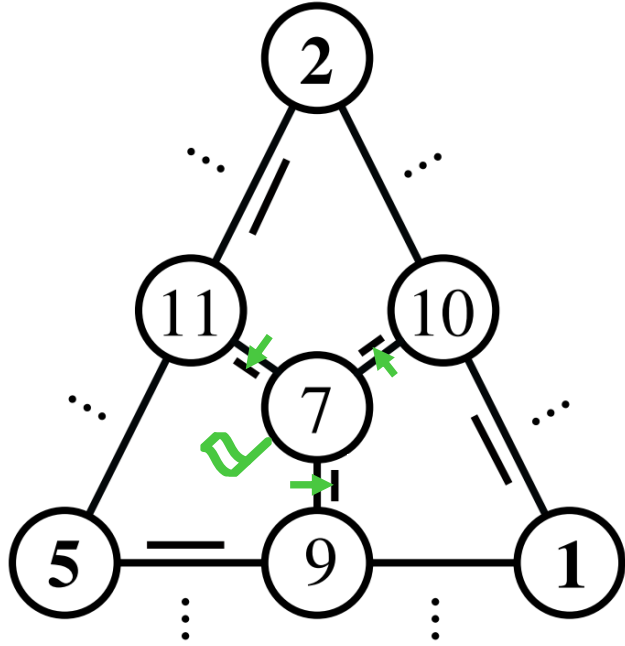
$$\mathcal{L}_{d+1}[e_0] \leftarrow 4n_2 + 1,$$

$$\mathcal{L}_{d+1}[e_1] \leftarrow 4n_0 + 1.$$

Quad-Task



Vertex-Task



	...	①	②	...	⑤	...	⑦	...	⑨	⑩	⑪	...
$\mathcal{L}_d$		4	8		0		2		1	5	3	

Task-Merge



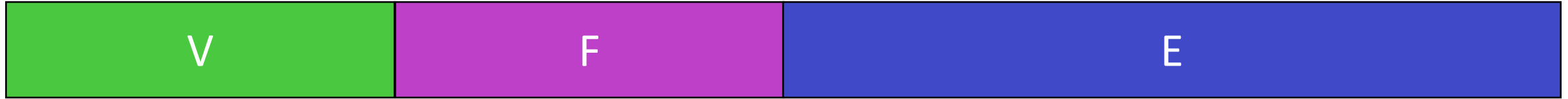
$$V - E + F = 2$$

$$E = 2F$$

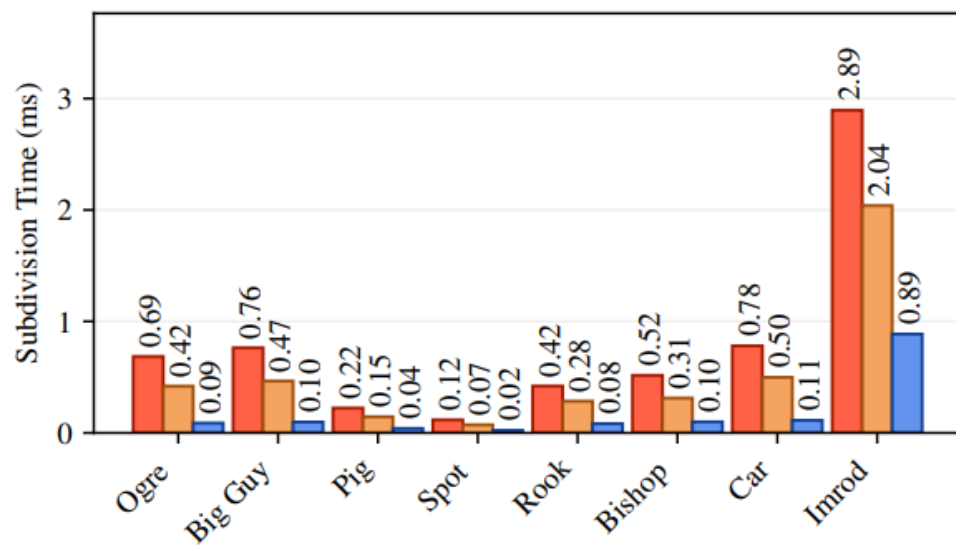
$$V = F + 2$$

$$V \approx F$$

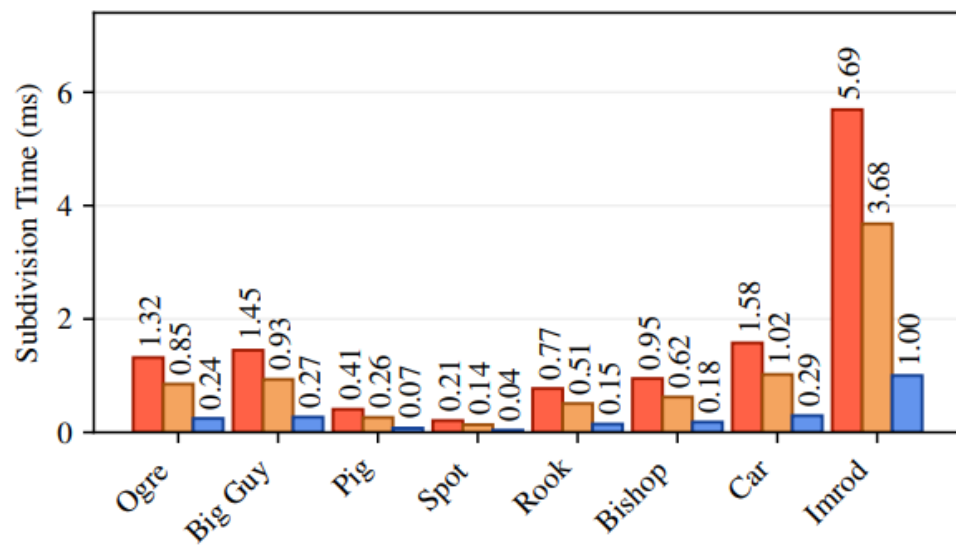
Memory Layout



AMD Radeon RX 7900 XTX

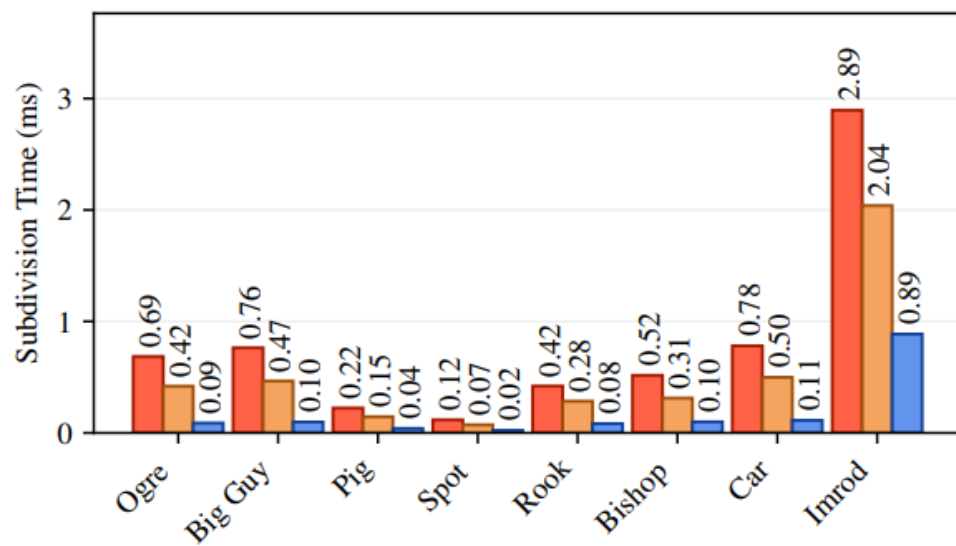
To $d = 5$ 

NVIDIA GeForce RTX 4080

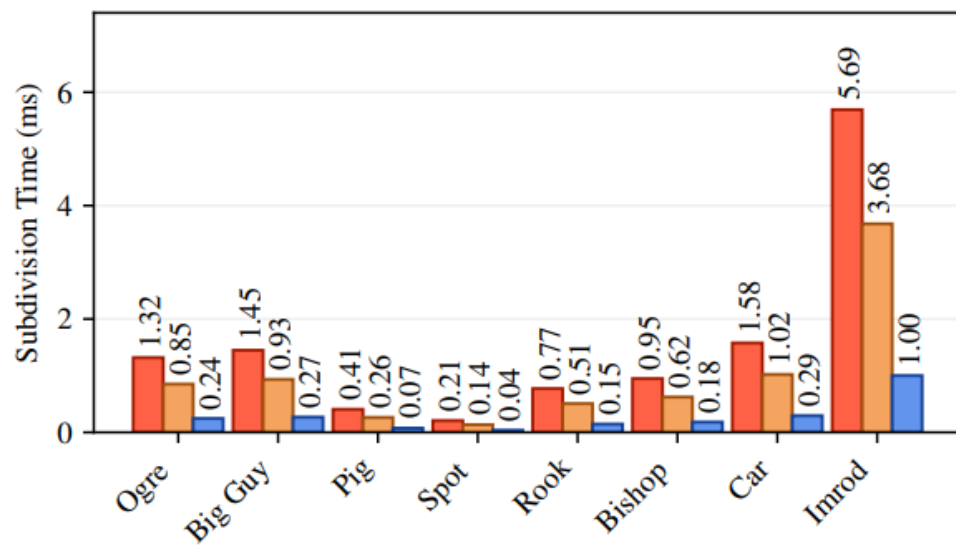
To $d = 5$ 

[DV21] (w/o hardware FP32-atomics) [DV21] Ours

AMD Radeon RX 7900 XTX

To $d = 5$ 

NVIDIA GeForce RTX 4080

To $d = 5$ 

■ [DV21] (w/o hardware FP32-atomics)
 ■ [DV21]
 ■ Ours

- ✓ **real-time** topology changes
- ✓ run **fast**
- ✓ **easy** to implement and extensible
- ✓ reduce **neighbor information**
- ✓ merge to **single dispatch**
- ✓ remove all **atomic float** arithmetic
- ✓ improve **memory layout**

COPYRIGHT AND DISCLAIMER

©2023 Advanced Micro Devices, Inc. All rights reserved.

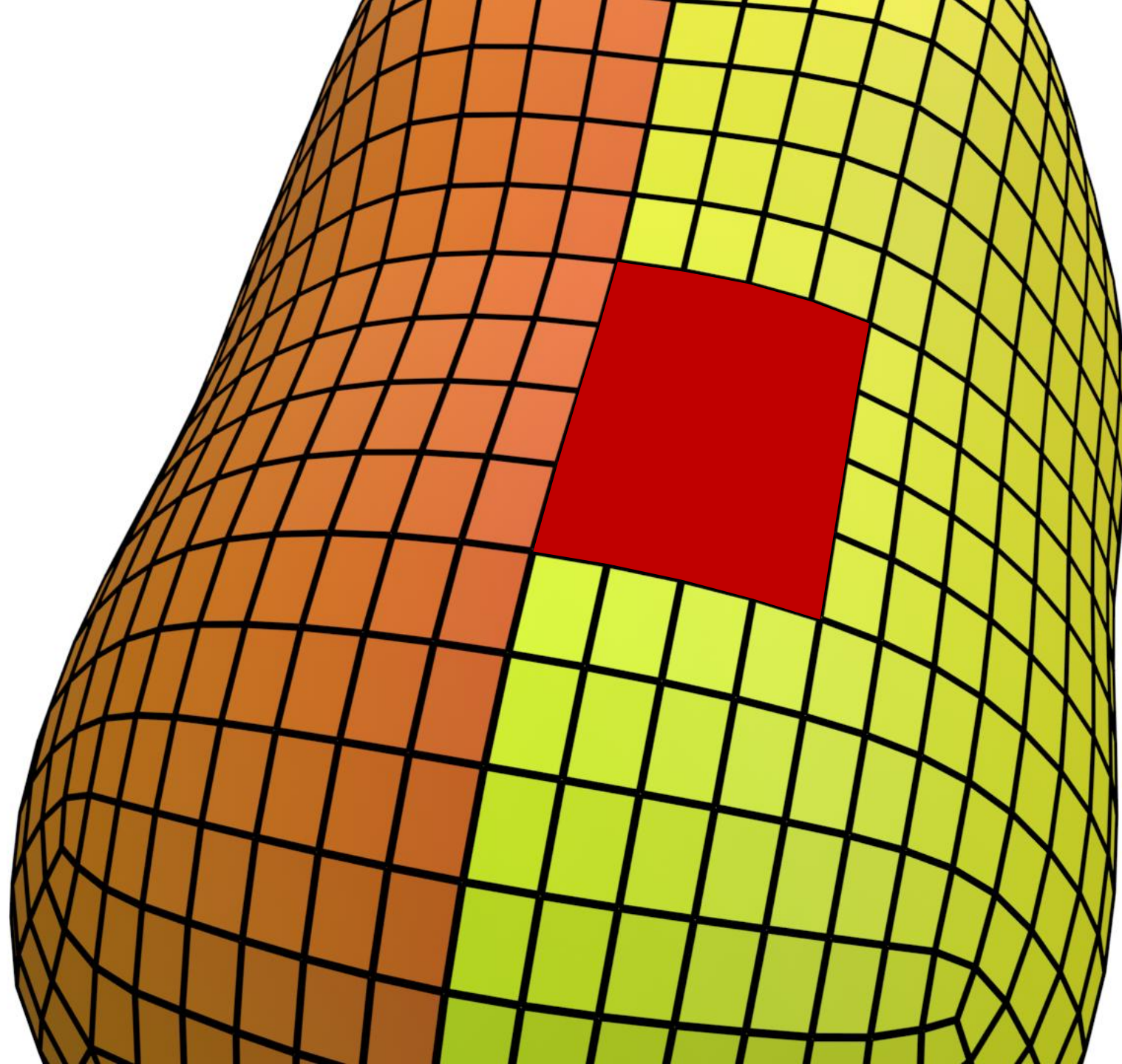
AMD, the AMD Arrow logo and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

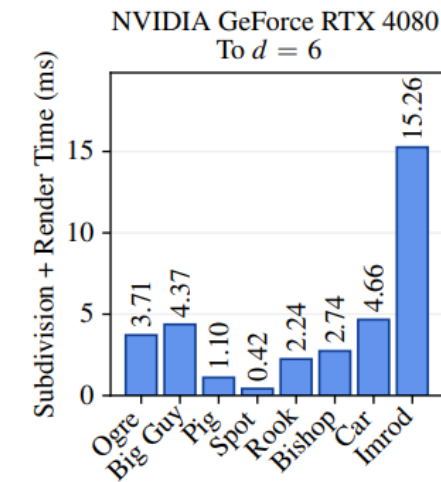
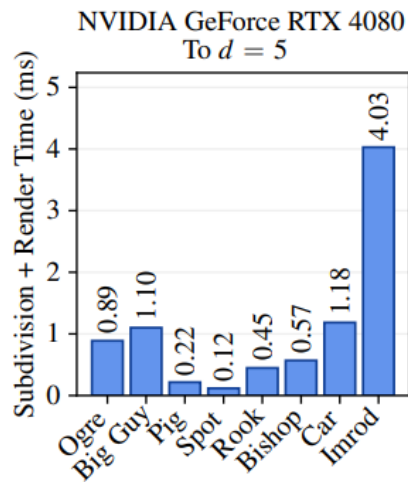
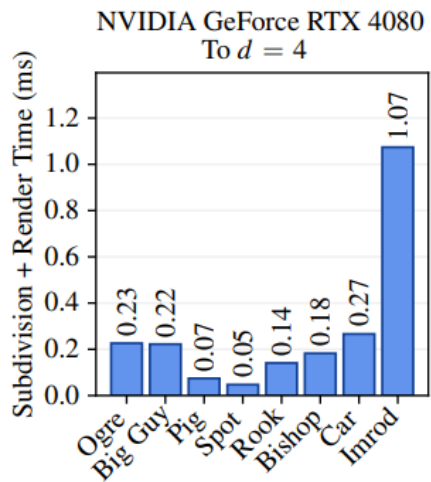
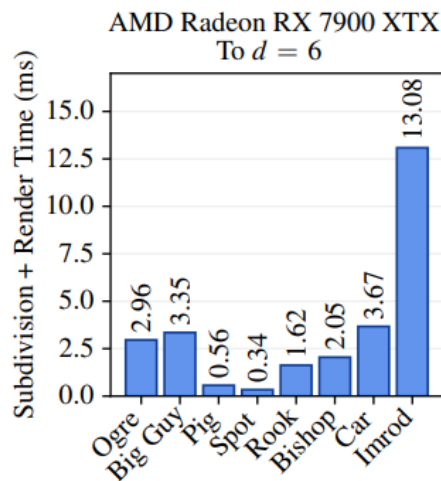
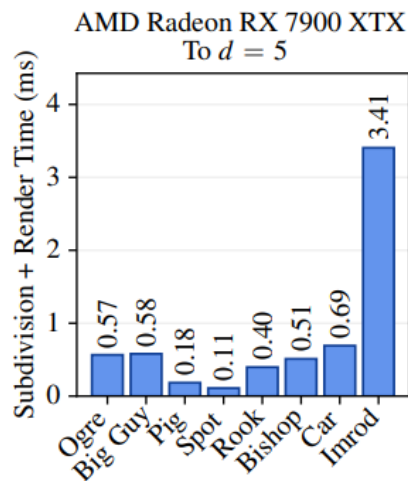
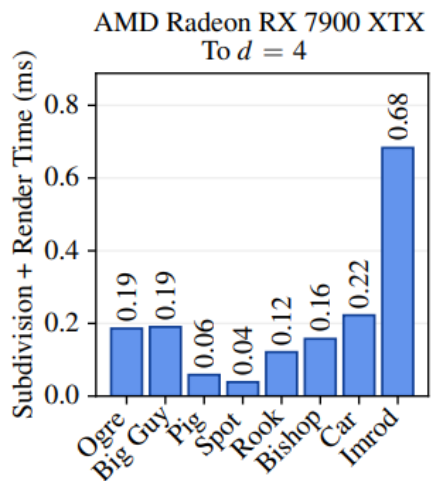
The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate releases, for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.









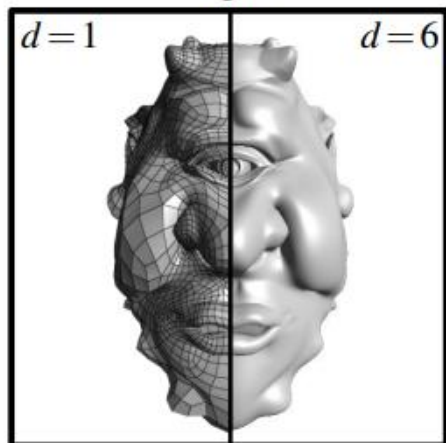
Meshes	Ogre	Big Guy	Pig	Spot	Rook	Bishop	Car	Imrod
[DV21]	386.0	424.8	111.6	53.6	224.4	273.9	461.5	1567.4
Ours	308.8	339.8	89.3	42.9	180.9	220.5	372.7	1266.9

Table 2: Temporary Memory Requirements for six subdivisions.

Meshes	Ogre	Big Guy	Pig	Spot	Rook	Bishop	Car	Imrod
Time (ms)	1.28	0.99	0.26	0.42	0.78	0.82	1.81	2.93

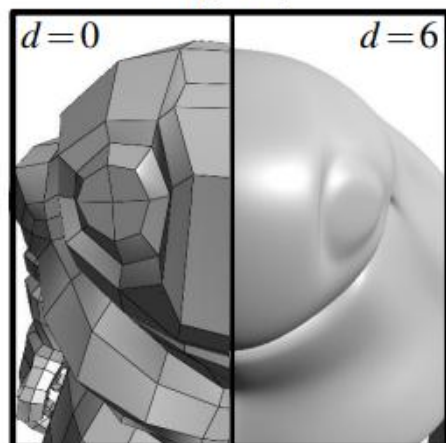
Table 1: Pre-processing Time.

Ogre



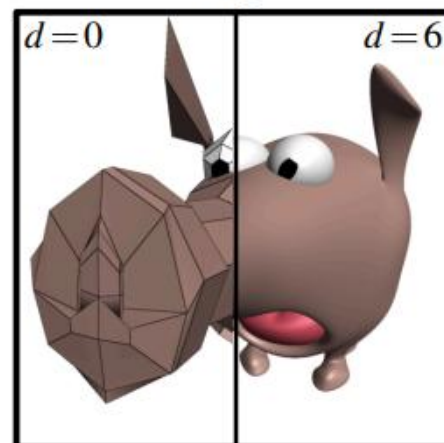
$F_1 = 5,270$ $F_6 = 5,396,480$
 $V_1 = 5,272$ $V_6 = 5,396,482$

Big Guy



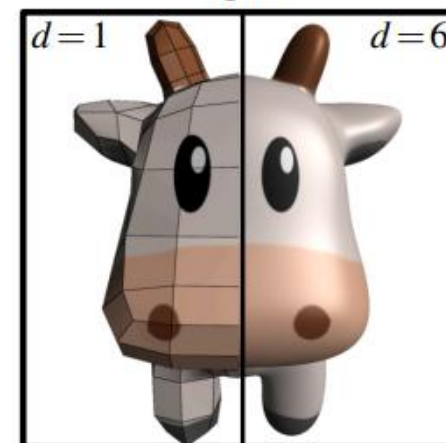
$F_0 = 1,450$ $F_6 = 5,939,200$
 $V_0 = 1,452$ $V_6 = 5,939,202$

Pig



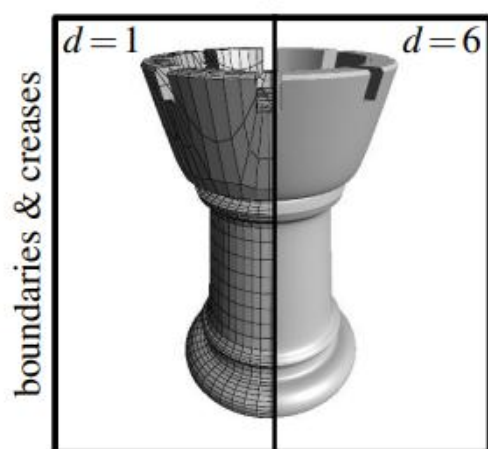
$F_0 = 381$ $F_6 = 1,560,576$
 $V_0 = 389$ $V_6 = 1,560,584$

Spot



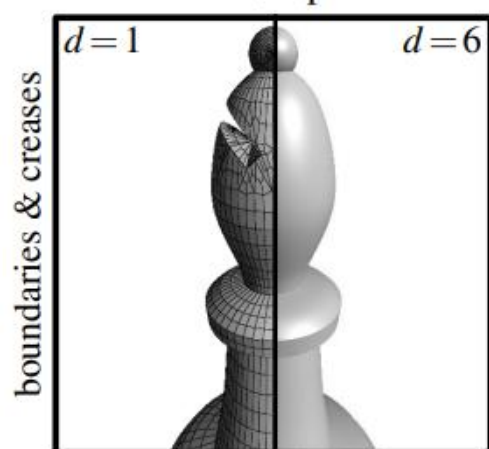
$F_1 = 732$ $F_6 = 749,568$
 $V_1 = 734$ $V_6 = 749,570$

Rook



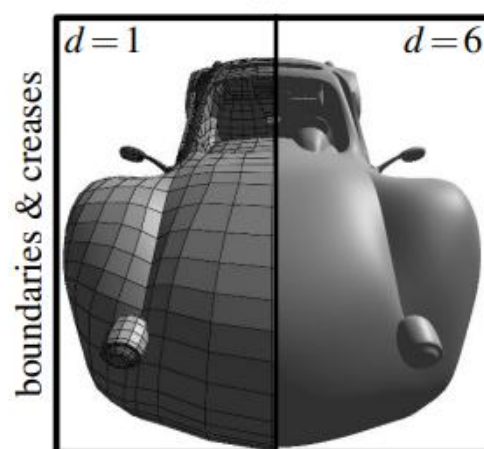
$F_1 = 3,064$ $F_6 = 3,137,536$
 $V_1 = 3,089$ $V_6 = 3,138,305$
 $G_1 = 24$ $G_6 = 24,576$
 $W_1 = 1$ $W_6 = 23,809$

Bishop



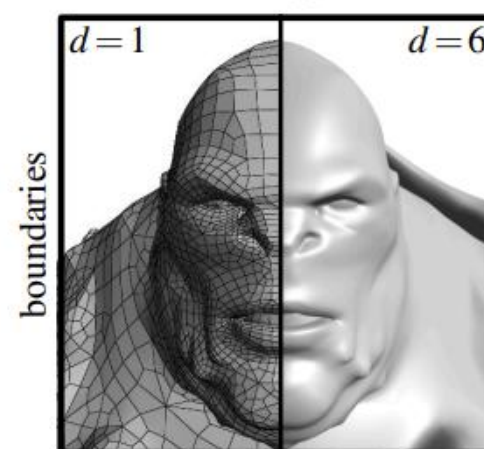
$F_1 = 3,740$ $F_6 = 3,829,760$
 $V_1 = 3,767$ $V_6 = 3,830,531$
 $G_1 = 24$ $G_6 = 24,576$
 $W_1 = 1$ $W_6 = 23,809$

Car



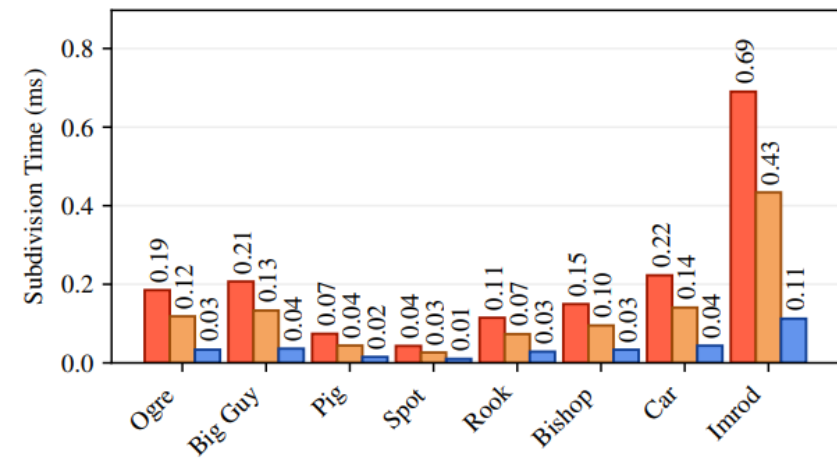
$F_1 = 6,300$ $F_6 = 6,451,200$
 $V_1 = 6,397$ $V_6 = 6,453,157$
 $G_1 = 60$ $G_6 = 61,440$
 $W_1 = 11$ $W_6 = 59,531$

Imrod

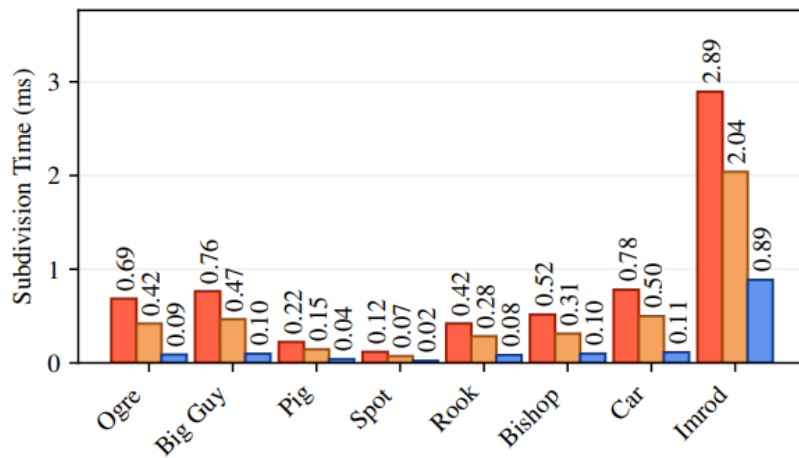


$F_1 = 21,399$ $F_6 = 21,912,576$
 $V_1 = 21,643$ $V_6 = 21,919,733$
 $G_1 = 223$ $G_6 = 228,352$
 $W_1 = 19$ $W_6 = 221,235$

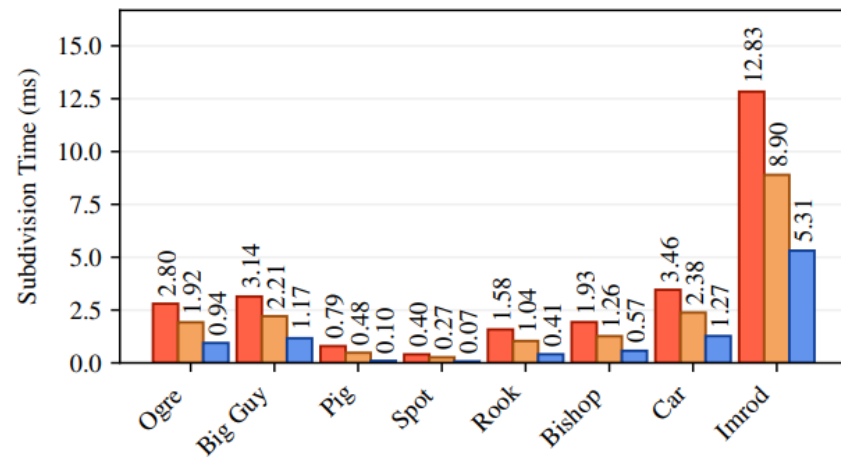
AMD Radeon RX 7900 XTX
To $d = 4$



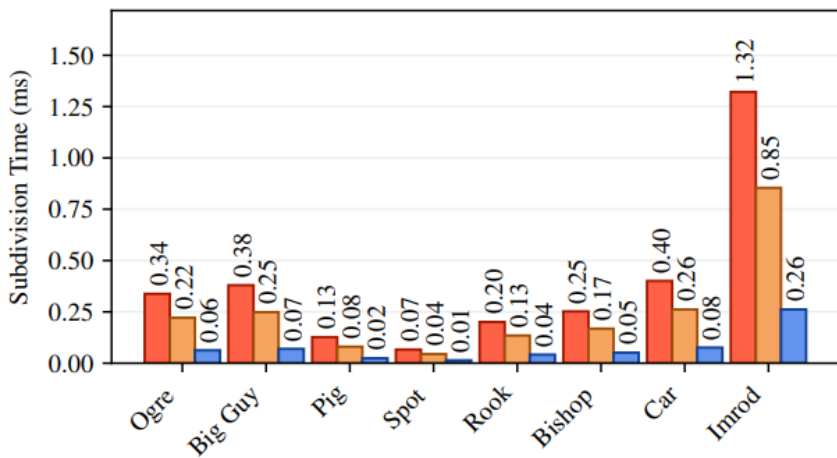
AMD Radeon RX 7900 XTX
To $d = 5$



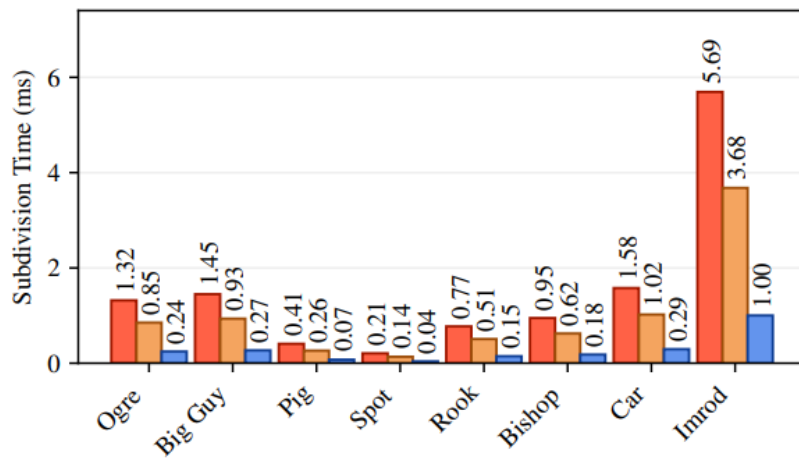
AMD Radeon RX 7900 XTX
To $d = 6$



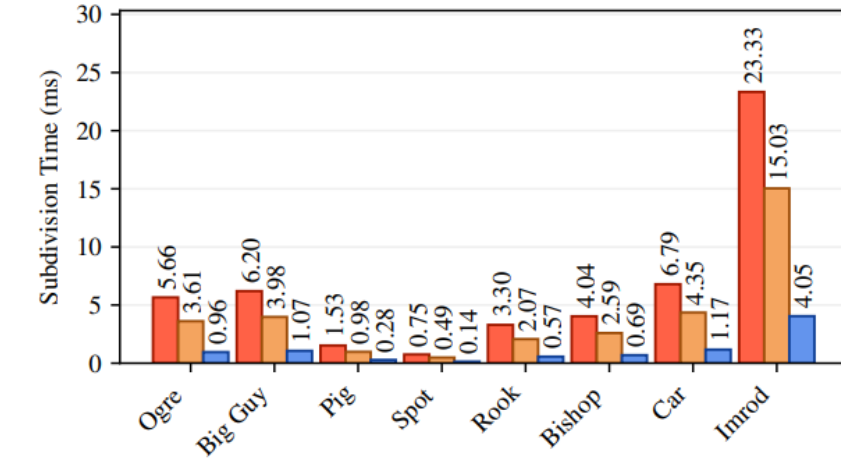
NVIDIA GeForce RTX 4080
To $d = 4$



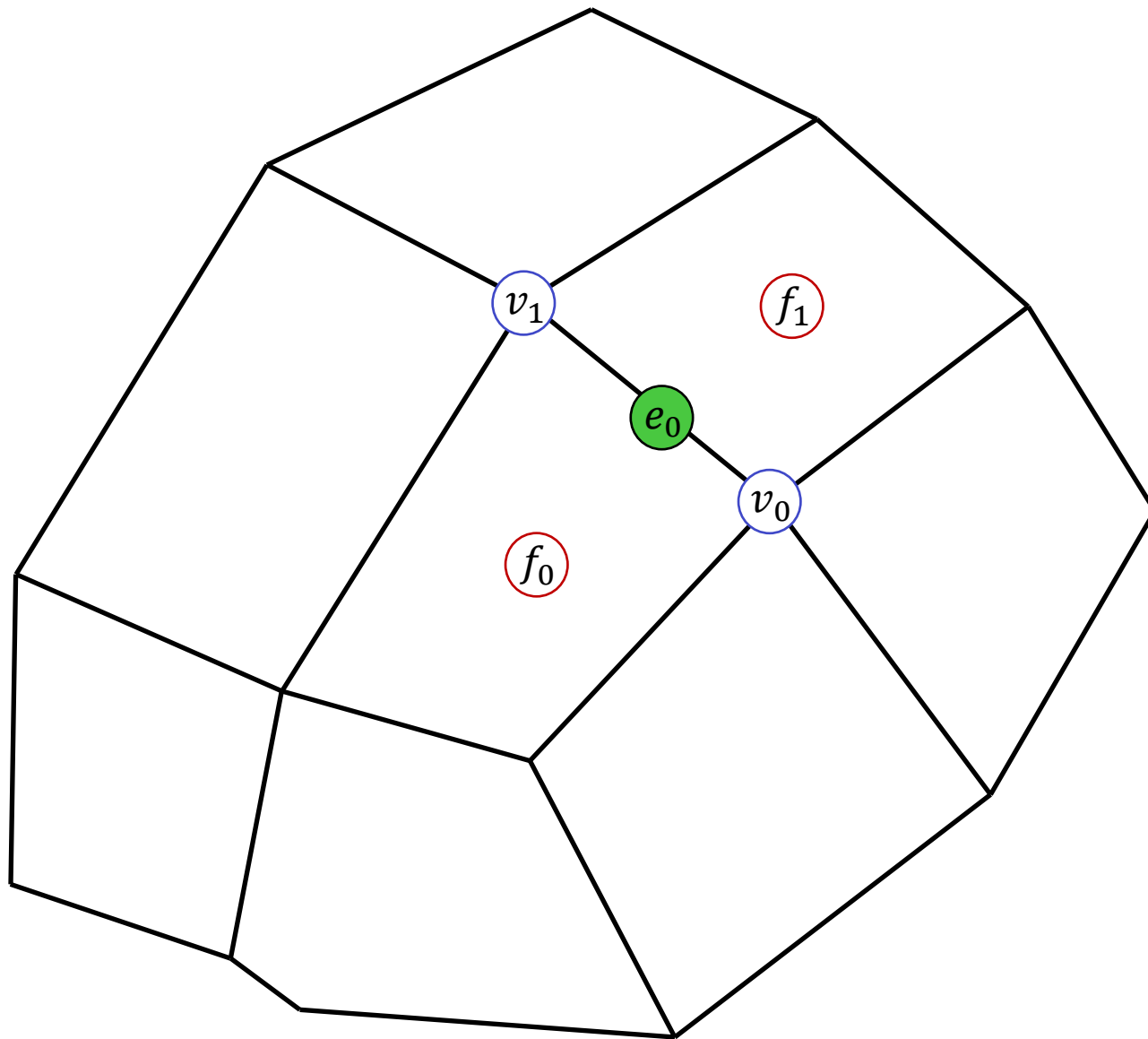
NVIDIA GeForce RTX 4080
To $d = 5$



NVIDIA GeForce RTX 4080
To $d = 6$



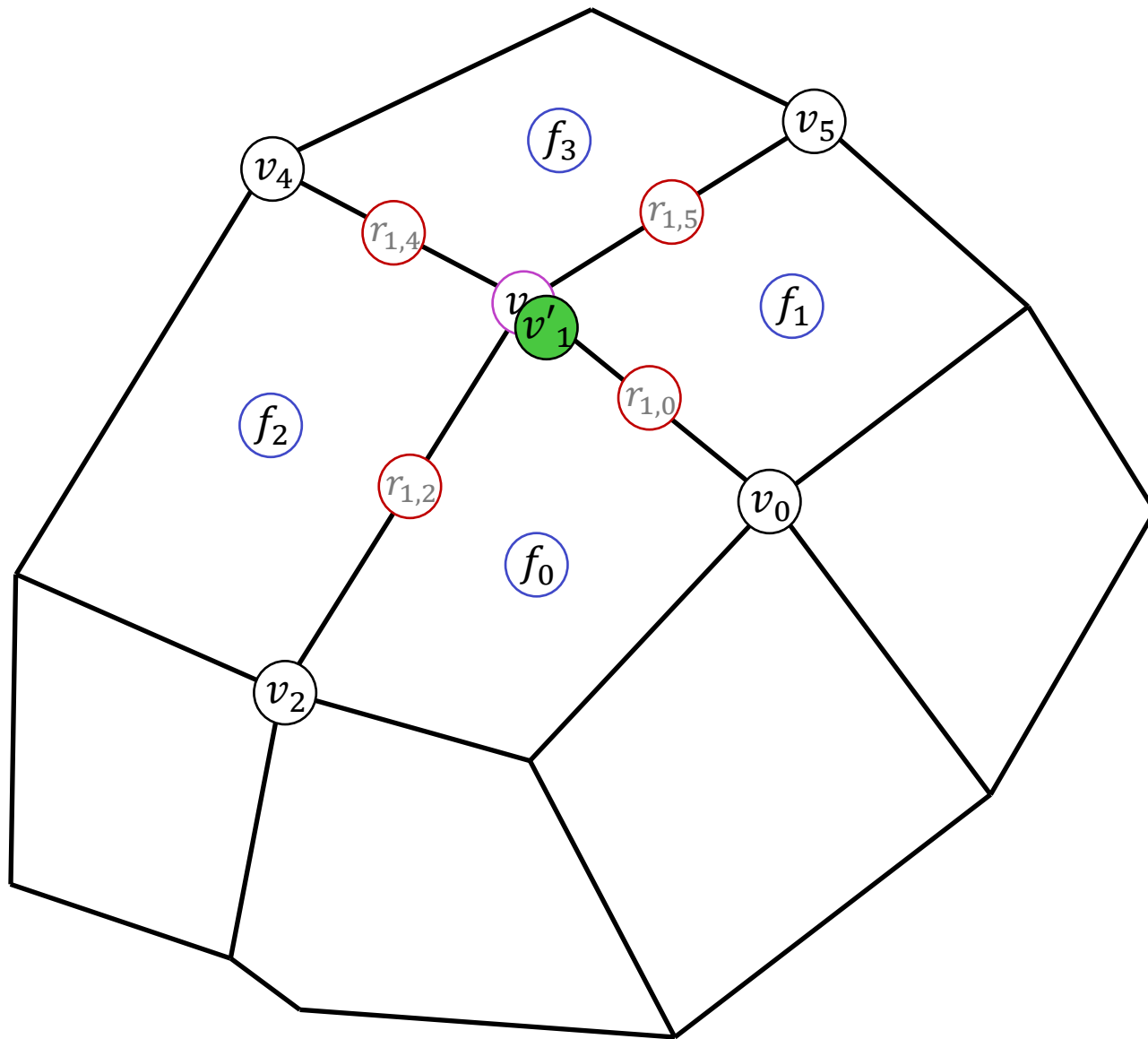
■ [DV21] (w/o hardware FP32-atomics)
 ■ [DV21]
 ■ Ours



$$f_0 = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

$$e_0 = \frac{1}{4}(v_0 + v_1 + f_0 + f_1)$$

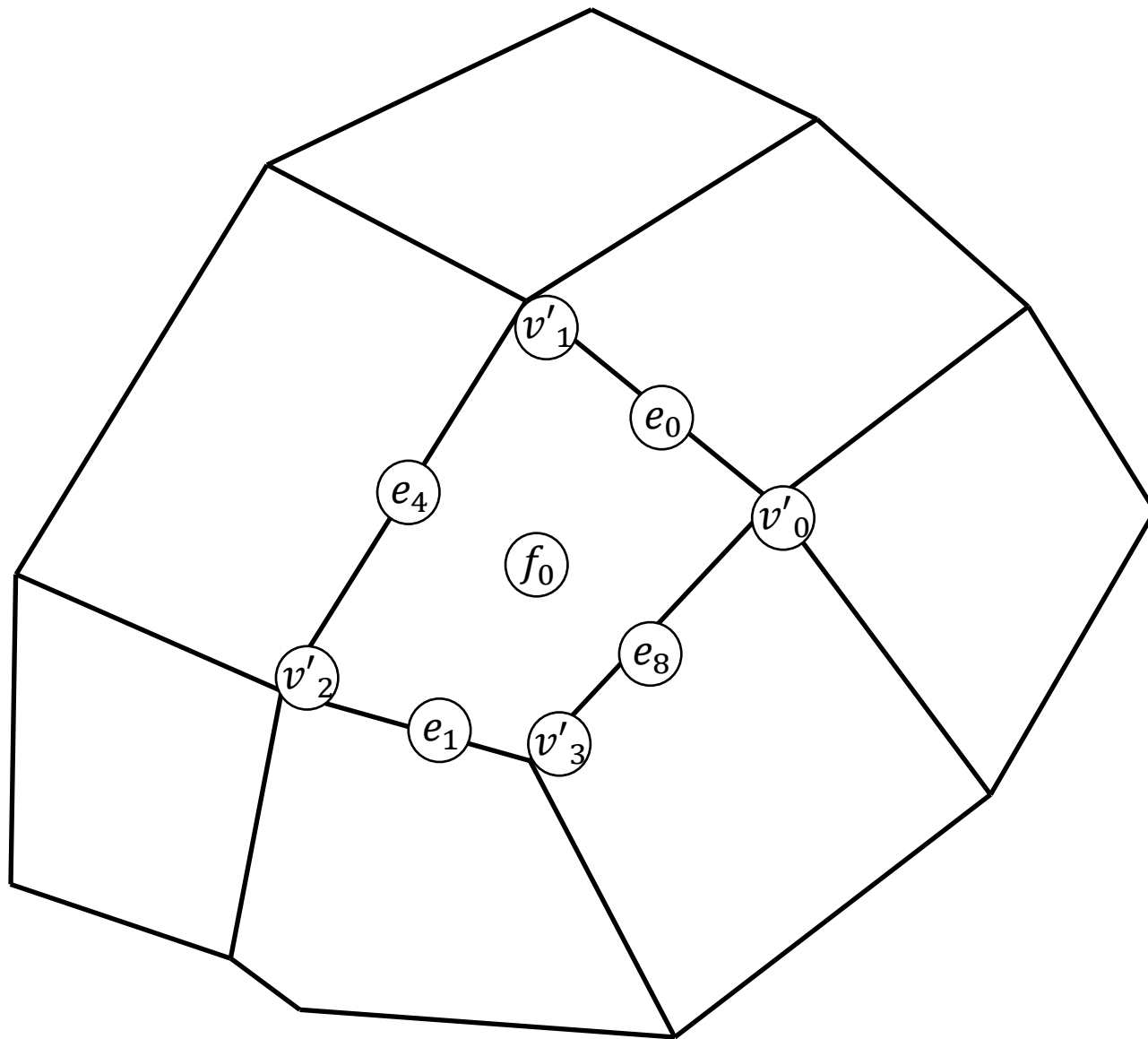
$$v'_1 = \frac{1}{n}(Q + 2R + (n - 3)v)$$



$$f_0 = \frac{1}{4}(v_0 + v_1 + v_2 + v_3)$$

$$e_0 = \frac{1}{4}(v_0 + v_1 + f_0 + f_1)$$

$$v'_1 = \frac{1}{n}(Q + 2R + (n-3)v)$$



$$f_0 = \frac{1}{4}(\mathbf{v}_0 + \mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3)$$

$$e_0 = \frac{1}{4}(\mathbf{v}_0 + \mathbf{v}_1 + \mathbf{f}_0 + \mathbf{f}_1)$$

$$\mathbf{v}'_1 = \frac{1}{n}(Q + 2R + (n - 3)\mathbf{v})$$